

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂY ĐÔ
KHOA KỸ THUẬT – CÔNG NGHỆ**



Trí tuệ - Sáng tạo - Năng động – Đổi mới

TIÊU LUẬN
HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ SPORTNEXUS

NGÀNH CÔNG NGHỆ THÔNG TIN

SVTH: Nguyễn Chí Nguyễn – MSSV: 227060172

Cần Thơ – tháng 8 năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂY ĐÔ
KHOA KỸ THUẬT – CÔNG NGHỆ**



Trí tuệ - Sáng tạo - Năng động – Đổi mới

TIỂU LUẬN

HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ SPORTNEXUS

NGÀNH CÔNG NGHỆ THÔNG TIN

SVTH: Nguyễn Chí Nguyễn – MSSV: 227060172

GVHD: THS. NGUYỄN CHÍ CƯỜNG

Cần Thơ – tháng 8 năm 2026

MỤC LỤC

MỤC LỤC	i
MỤC LỤC HÌNH ẢNH	vi
MỤC LỤC BẢNG	viii
LỜI CẢM ƠN.....	x
LỜI CAM ĐOAN	xi
TÓM TẮT.....	xii
ABSTRACT.....	xiii
ĐÁNH GIÁ KẾT QUẢ THỰC HIỆN.....	xiv
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	xv
CHƯƠNG I. MỞ ĐẦU	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu cần đạt được	1
1.3 Phạm vi nghiên cứu.....	2
1.3.1 Nghiên cứu đối tượng sử dụng	2
1.3.2 Chức năng chính cho các đối tượng	2
1.3.3 Phạm vi công nghệ	3
1.3.4 Phạm vi triển khai.....	3
1.4 Phương pháp nghiên cứu.....	3
1.4.1 Nghiên cứu tài liệu và cơ sở lý thuyết.....	3
1.4.2 Phân tích yêu cầu và thiết kế hệ thống	4
1.4.2.1. Yêu cầu chức năng theo nhóm đối tượng	4
1.4.2.2. Yêu cầu phi chức năng.....	5
1.4.3. Kiểm thử và đánh giá hệ thống.....	5
1.4.3.1 Kiểm thử chức năng thủ công (Manual Functional Testing).....	5
1.4.3.2. Kiểm thử tự động và kiểm thử tích hợp API (API & End-to-End Testing)...	5
1.4.3.3 Đánh giá và ghi nhận hạn chế	5
1.5 Hướng giải quyết.....	5

1.6 Kế hoạch thực hiện.....	6
CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	8
2.1 Mô hình kinh doanh B2C.....	8
2.2 Mô hình Client – Server.....	8
2.2.1 Khái niệm	8
2.2.2 Lợi ích của mô hình client–server	9
2.2.3 Sơ đồ kiến trúc tổng thể trong SportNexus	9
2.2.4 Phân tích sâu việc áp dụng trong SportNexus	10
2.3 Kiến trúc 3 lớp (Three-Layer Architecture).....	11
2.3.1 Tổng quan kiến trúc.....	11
2.3.2 Ưu điểm của kiến trúc 3 lớp	12
2.3.3 Ví dụ minh họa luồng nghiệp vụ (Luồng tạo đơn hàng)	12
2.3.4 Nhược điểm và phương án khắc phục của kiến trúc 3 lớp	16
2.4 Công nghệ Frontend.....	16
2.4.1 React 19	16
2.4.2 Vite	17
2.4.3 React Router 7	17
2.4.4 TanStack Query	18
2.4.5 Tailwind CSS.....	19
2.4.6 Các thư viện hỗ trợ Frontend khác	19
2.5 Công nghệ Backend và Cơ sở dữ liệu.....	20
2.5.1 Javascript	20
2.5.1.1 Tổng quan	20
2.5.1.2 NodeJS và Express 5.....	20
2.5.2 Prisma ORM.....	21
2.5.3 MySQL	21
2.5.4 JWT (JSON Web Token)	22
2.5.5 Các thư viện hỗ trợ Backend khác.....	24

2.5.6 Các phần mềm hỗ trợ.....	24
2.5.7 Lợi ích thực tiễn.....	26
CHƯƠNG III. KẾT QUẢ THỰC HIỆN	27
3.1 Cơ sở dữ liệu	27
3.2 Nhóm tài khoản và địa chỉ	31
3.2.1 users (Người dùng).....	31
3.2.2 useraddresses (Sổ địa chỉ)	32
3.3 Nhóm phân quyền	33
3.3.1 permissions (Quyền hạn).....	33
3.3.2 roles (Vai trò)	34
3.4 Nhóm phân loại sản phẩm.....	34
3.4.1 categories (Danh mục).....	34
3.4.2 collections (Bộ sưu tập).....	35
3.4.3 brands (Thương hiệu)	36
3.4.4 suppliers (Nhà cung cấp)	36
3.5 Nhóm hình ảnh và thuộc tính sản phẩm.....	37
3.5.1 productimages (Hình ảnh sản phẩm).....	37
3.5.2 attributekeys (Khóa thuộc tính)	37
3.5.3 variableattributes (Giá trị thuộc tính biến thể)	38
3.5.4 productattributekeys (Thuộc tính hiển thị của sản phẩm)	39
3.6 Nhóm mã giảm giá.....	39
3.6.1 coupons (Mã giảm giá).....	39
3.6.2 user_coupons (Mã giảm giá đã lưu):	41
3.7 Nhóm đơn hàng và thanh toán	41
3.7.1 invoices (Hóa đơn)	41
3.7.2 payment_transactions (Lịch sử thanh toán).....	43
3.7.3 orders (Đơn hàng).....	44
3.7.4 orderitems (Dòng chi tiết đơn hàng).....	45

3.8 Nhóm đánh giá	46
3.8.1 reviews (Đánh giá sản phẩm)	46
3.9 Nhóm kho và nhập hàng	47
3.9.1 purchaseorders (Đơn nhập hàng).....	47
3.9.2 purchaseorderitems (Chi tiết đơn nhập)	48
3.9.3 stockmovements (Biến động tồn kho):.....	48
3.10 Nhóm chương trình thành viên và tích điểm.....	49
3.10.1 membership_tiers (Hạng thành viên)	49
3.10.2 tier_rewards (Phần thưởng đổi điểm).....	50
3.10.3 point_transactions (Nhật ký tích điểm)	51
3.11 Nhóm sản phẩm và giỏ hàng.....	52
3.11.1 productvariants (Biến thể sản phẩm)	52
3.11.2 products (Sản phẩm).....	52
3.11.3 carts (Giỏ hàng)	53
3.11.4 cartitems (Dòng giỏ hàng)	54
3.12 Nhóm nhật ký hệ thống	54
3.12.1 systemlogs (Nhật ký hoạt động).....	54
3.13 Mối quan hệ thực thể chi tiết.....	55
3.14 Nguyên tắc toàn vẹn dữ liệu và khóa	59
3.15 Biểu đồ use case tổng quát	60
3.15.1 Use case — Khách hàng.....	60
3.15.2 Use case — Quản trị viên.....	61
3.16 Phân tích các quy trình nghiệp vụ chính	62
3.16.1 Quy trình nhập kho	62
3.16.2 Quy trình bán hàng	64
3.17 Giao diện và chức năng (Client)	67
3.17.1 Trang chủ.....	67
3.17.2 Danh sách sản phẩm – tìm kiếm và lọc	71

3.17.3 Chi tiết sản phẩm và chọn biến thể.....	73
3.17.4 Giỏ hàng	73
3.17.5 Đặt hàng và thanh toán (Checkout)	75
3.17.6 Theo dõi đơn hàng.....	78
3.17.7 Đăng ký và đăng nhập	78
3.18 Giao diện và chức năng (Quản trị viên)	80
3.18.1 Dashboard thống kê.....	81
3.18.2 Quản lý sản phẩm, biến thể và thuộc tính	81
3.18.3 Quản lý danh mục, thương hiệu, bộ sưu tập và nhà cung cấp.....	83
3.18.4 Đơn nhập hàng và quản lý kho	83
3.18.5 Xử lý đơn hàng và hóa đơn	86
3.18.6 Quản lý tài khoản, vai trò và phân quyền.....	87
3.18.7 Khuyến mãi, chương trình thành viên và đánh giá.....	88
KẾT LUẬN – ĐÁNH GIÁ	96
TÀI LIỆU THAM KHẢO.....	100

MỤC LỤC HÌNH ẢNH

Hình 2. 1. Tổng hợp lợi ích của mô hình Client – Server.....	10
Hình 2. 2 Sơ đồ luồng nghiệp vụ tạo đơn hàng	14
Hình 2. 3 Kết quả của giải mã một JWT ví dụ	23
Hình 3. 1 Mô hình MCD của hệ thống	27
Hình 3. 2 Mô hình MLD của hệ thống	28
Hình 3. 3 Mô hình MPD của hệ thống.....	29
Hình 3. 4. Sơ đồ use case của actor khách hàng	60
Hình 3. 5. Sơ đồ use case của actor quản trị viên (Admin)	61
Hình 3. 6. Lưu đồ chi tiết quy trình nghiệp vụ Nhập kho.....	62
Hình 3. 7. Lưu đồ luồng data của quy trình nhập kho	63
Hình 3. 8. Lưu đồ luồng data của quy trình bán hàng	65
Hình 3. 9 Giao diện trang chủ.....	68
Hình 3. 10 Giao diện menu ở header	68
Hình 3. 11 – Lưu đồ lấy dữ liệu trang chủ.....	70
Hình 3. 12 Giao diện trang danh sách sản phẩm – tìm kiếm và lọc	72
Hình 3. 13 – Lưu đồ lọc và tìm kiếm sản phẩm.....	72
Hình 3. 14 Giao diện trang chi tiết sản phẩm	73
Hình 3. 15 Giao diện trang giỏ hàng.....	74
Hình 3. 16 Lưu đồ thêm vào giỏ hàng	74
Hình 3. 17 Gaio diện trang thanh toán.....	76
Hình 3. 18 Giao diện QR thanh toán của payOS khi chọn thanh toán qua app ngân hàng .	76
Hình 3. 19 Giao diện QR thanh toán của payOS khi chọn thanh toán qua app ngân hàng (mobile).....	76
Hình 3. 20 Thông tin đơn hàng sẽ được tự động điện sẵn	77
Hình 3. 21 Giao diện khi thanh toán thành công	77
Hình 3. 22 – Lưu đồ đặt hàng và thanh toán.....	77
Hình 3. 23 Giao diện trang theo dõi đơn hàng.....	78
Hình 3. 24 Giao diện trang đăng nhập	79
Hình 3. 25 Giao diện trang đăng ký	79
Hình 3. 26 – Lưu đồ đăng ký tài khoản	79
Hình 3. 27 – Lưu đồ đăng nhập	80
Hình 3. 28 Giao diện trang dashboard thống kê kinh doanh	81

Hình 3. 29 form thêm biến thể cho sản phẩm	82
Hình 3. 30 Giao diện trang quản lý sản phẩm	82
Hình 3. 31 – Lưu đồ thêm sản phẩm mới	82
Hình 3. 32 Giao diện quản lý danh mục	83
Hình 3. 33 Giao diện quản lý thương hiệu.....	83
Hình 3. 34 Giao diện trang nhập hàng	84
Hình 3. 35 Giao diện trang quản lý kho.....	84
Hình 3. 36 – Lưu đồ nhận hàng - nhập kho	85
Hình 3. 37 – Lưu đồ xuất kho, giao hàng và hủy đơn	85
Hình 3. 38 Giao diện trang quản lý đơn hàng.....	86
Hình 3. 39 Giao diện trang quản lý người dùng	87
Hình 3. 40 Giao diện trang cấp quyền	87
Hình 3. 41 – Lưu đồ gán vai trò và quyền cho người dùng.....	88
Hình 3. 42 Giao diện trang thêm khuyến mãi.....	89
Hình 3. 43 Giao diện trang tặng khuyến mãi.....	89
Hình 3. 44 – Lưu đồ tạo và tặng mã giảm giá	90
Hình 3. 45 Giao diện trang quản lý đánh giá.....	91
Hình 3. 46 Giao diện model trả lời đánh giá.....	91
Hình 3. 47 – Lưu đồ duyệt và phản hồi đánh giá.....	92
Hình 3. 48 Giao diện menu tích điểm.....	93
Hình 3. 49 Giao diện trang quản lý hạng thành viên	93
Hình 3. 50 Giao diện trang quản lý đổi quà.....	93
Hình 3. 51 Giao diện trang cấu hình điểm.....	94
Hình 3. 52 Giao diện trang quản lý người dùng và điểm.....	94
Hình 3. 53 – Lưu đồ điều chỉnh điểm thành viên	95

MỤC LỤC BẢNG

Bảng 1. 1 Nội dung công việc.....	7
Bảng 2. 1 Lợi ích mô hình client – server.....	9
Bảng 2. 2. Khía cạnh áp dụng mô hình Client – Server trong hệ thống SportNexus	11
Bảng 2. 3. Ưu điểm của mô hình kiến trúc 3 lớp trong hệ thống Backend	12
Bảng 2. 4. Nhược điểm của kiến trúc 3 lớp và giải pháp khắc phục trong hệ thống	16
Bảng 2. 5. Tổng hợp các thư viện hỗ trợ phát triển phía Frontend	20
Bảng 2. 6. Phân loại và thời hạn của chiến lược Token trong SportNexus	23
Bảng 2. 7. Tổng hợp các thư viện hỗ trợ phía Backend.....	24
Bảng 3. 1 Chi tiết thực thể users	32
Bảng 3. 2 Chi tiết thực thể useraddresses	33
Bảng 3. 3 Chi tiết thực thể permissions	33
Bảng 3. 4 Chi tiết thực thể roles	34
Bảng 3. 5 Chi tiết thực thể categories	35
Bảng 3. 6 Chi tiết thực thể collections	36
Bảng 3. 7 Chi tiết thực thể brands	36
Bảng 3. 8 Chi tiết thực thể suppliers.....	37
Bảng 3. 9 Chi tiết thực thể productimages.....	37
Bảng 3. 10 Chi tiết thực thể attributekeys	38
Bảng 3. 11 Chi tiết thực thể variableattributes	38
Bảng 3. 12 Chi tiết thực thể productattributekeys	39
Bảng 3. 13 Chi tiết thực thể coupons.....	40
Bảng 3. 14 Chi tiết thực thể user_coupons	41
Bảng 3. 15 Chi tiết thực thể invoices.....	42
Bảng 3. 16 Chi tiết thực thể payment_transactions	43
Bảng 3. 17 Chi tiết thực thể orders	45
Bảng 3. 18 Chi tiết thực thể orderitems	45
Bảng 3. 19 Chi tiết thực thể reviews.....	46
Bảng 3. 20 Chi tiết thực thể purchaseorders	47
Bảng 3. 21 Chi tiết thực thể purchaseorderitems.....	48
Bảng 3. 22 Chi tiết thực thể stockmovements	49
Bảng 3. 23 Chi tiết thực thể membership_tiers	50

Bảng 3. 24 Chi tiết thực thể tier_rewards	51
Bảng 3. 25 Chi tiết thực thể point_transactions.....	51
Bảng 3. 26 Chi tiết thực thể productvariants	52
Bảng 3. 27 Chi tiết thực thể products	53
Bảng 3. 28 Chi tiết thực thể carts.....	54
Bảng 3. 29 Chi tiết thực thể cartitems	54
Bảng 3. 30 Chi tiết thực thể systemlogs	55
Bảng 3. 31 liệt kê đầy đủ các cặp quan hệ chính trong hệ thống.....	58

LỜI CẢM ƠN

Lời đầu tiên, xin gửi lời cảm ơn chân thành tới Thầy Nguyễn Chí Cường, người đã tận tình hướng dẫn và chỉ bảo trong suốt quá trình thực hiện bài niên luận này. Thầy đã giúp vượt qua nhiều khó khăn và cung cấp những kiến thức quý báu, góp phần quan trọng vào việc hoàn thành nghiên cứu này. Bên cạnh đó, xin bày tỏ lòng biết ơn sâu sắc tới các thầy cô trong Khoa Kỹ Thuật Công Nghệ, Trường Đại học Tây Đô, vì đã tạo điều kiện thuận lợi trong quá trình học tập và nghiên cứu. Cảm ơn gia đình, cha mẹ và những người thân yêu đã luôn động viên, khích lệ cả về tinh thần lẫn vật chất trong suốt chặng đường học tập. Cuối cùng, xin cảm ơn bạn bè đã luôn hỗ trợ và đóng góp ý kiến quý báu, giúp hoàn thiện bài niên luận này. Những lời động viên và sự giúp đỡ của mọi người chính là nguồn động lực to lớn giúp hoàn thành tốt công việc nghiên cứu. Xin chân thành cảm ơn!

LỜI CAM ĐOAN

Tôi xin cam đoan rằng Tiểu luận này là do chính tôi thực hiện, không sao chép dưới bất kỳ hình thức nào hay thuê hoặc nhờ người khác thực hiện.

Dữ liệu và kết quả phân tích trong Tiểu luận đảm bảo tính chính xác, khách quan và trung thực, không có bất kỳ sự ngụy tạo và điều chỉnh kết quả nghiên cứu bằng sự chủ quan của tác giả.

Cần Thơ, ngày.....tháng 8 năm 2026

Sinh viên

Nguyễn Chí Nguyễn

TÓM TẮT

Hệ thống SportNexus là nền tảng thương mại điện tử chuyên biệt cho lĩnh vực thể thao, hỗ trợ chủ cửa hàng quản lý toàn diện hoạt động kinh doanh trên một nền tảng duy nhất, bao gồm quản lý sản phẩm đa biến thể, tồn kho, đơn hàng, thanh toán, vận chuyển và kết nối trực tiếp với khách hàng. Giao diện được thiết kế trực quan, dữ liệu được đồng bộ giữa các thiết bị, giúp giảm thời gian thao tác và hạn chế sai sót trong quá trình vận hành.

Hệ thống gồm ba nhóm đối tượng sử dụng chính: khách hàng, nhân viên quản lý và quản trị viên.

Đối với khách hàng, hệ thống cho phép duyệt và tìm kiếm sản phẩm theo danh mục, thương hiệu, giá và thuộc tính; xem chi tiết sản phẩm với nhiều biến thể (màu sắc, kích thước) kèm tồn kho và giá tương ứng. Khách hàng có thể thêm sản phẩm vào giỏ hàng, áp dụng mã giảm giá, tiến hành thanh toán đa phương thức (COD, chuyển khoản, MoMo, VNPAY, thẻ tín dụng) và theo dõi trạng thái đơn hàng cùng vận đơn. Sau khi nhận hàng, khách có thể đánh giá sản phẩm và tra cứu hóa đơn bất kỳ lúc nào. Thông tin cá nhân của khách hàng được bảo mật và có thể chỉnh sửa trong mục tài khoản.

Đối với nhân viên quản lý, hệ thống cho phép quản lý sản phẩm và biến thể, danh mục, thương hiệu, nhà cung cấp, mã giảm giá và bộ sưu tập. Nhân viên xử lý đơn hàng từ khâu chuẩn bị hàng đến giao thành công, quản lý tồn kho qua phiếu nhập – xuất, lập phiếu nhập hàng từ nhà cung cấp và tạo hóa đơn. Hệ thống cung cấp báo cáo thu – chi, thống kê doanh thu, đơn hàng và tồn kho giúp theo dõi hiệu quả kinh doanh. Mỗi giao dịch được lập hóa đơn rõ ràng, lưu trữ đầy đủ và dễ dàng tra cứu.

Đối với quản trị viên, hệ thống hỗ trợ giám sát toàn bộ hệ thống, quản lý người dùng và phân quyền (RBAC), kiểm duyệt đánh giá và nội dung, phân tích dữ liệu, theo dõi nhật ký hoạt động, tối ưu giao diện – hiệu năng – tính năng và hỗ trợ kỹ thuật. Vai trò này đảm bảo hệ thống vận hành ổn định, an toàn và minh bạch cho cả nhà bán lẫn khách hàng..

ABSTRACT

SportNexus is a specialized e-commerce platform for the sports industry, helping store owners comprehensively manage their business on a single platform, including multi-variant product management, inventory, orders, payments, shipping, and direct connection with customers. The interface is designed to be intuitive, with data synchronized across devices, reducing operational time and minimizing errors in daily operations.

The system serves three main user groups: customers, management staff, and administrators.

For customers, the system allows browsing and searching products by category, brand, price, and attributes; viewing product details with multiple variants (color, size) along with corresponding stock and price. Customers can add products to the cart, apply discount coupons, complete payments through multiple methods (COD, bank transfer, MoMo, VNPay, credit card), and track order status and shipment. After receiving the goods, customers can review products and retrieve invoices at any time. Customer personal information is secured and can be edited in the account section.

For management staff, the system allows managing products and variants, categories, brands, suppliers, discount coupons, and collections. Staff process orders from preparation to successful delivery, manage inventory through purchase orders and stock movements, create purchase orders from suppliers, and generate invoices. The system provides income–expense reports and statistics on revenue, orders, and inventory to monitor business performance. Every transaction is issued with a clear invoice, fully stored, and easily retrievable.

For administrators, the system supports monitoring the entire platform, managing users and permissions (RBAC), moderating reviews and content, analyzing data, tracking activity logs, optimizing interface – performance – features, and providing technical support. This role ensures the system operates stably, securely, and transparently for both sellers and customers.

ĐÁNH GIÁ KẾT QUẢ THỰC HIỆN
NIÊN LUẬN, TIỂU LUẬN, KHOÁ LUẬN
(Học kỳ: IV, Năm 2026)

TÊN ĐỀ TÀI: HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ SPORTNEXUS

GIÁO VIÊN HƯỚNG DẪN:

STT	HỌ VÀ TÊN	MSCB
1	Nguyễn Chí Cường	

SINH VIÊN THỰC HIỆN:

STT	HỌ VÀ TÊN	MSSV	THƯỞNG (Tối đa 1,0 điểm)	ĐIỂM
1	Nguyễn Chí Nguyễn	227060172		

I. HÌNH THỨC (Tối đa 1,5 điểm)	
II. NỘI DUNG (Tối đa 5 điểm)	
Ứng dụng (tối đa 3 điểm)	
Giới thiệu chương trình (0,5 điểm)	
III. CHƯƠNG TRÌNH DEMO (Tối đa 3.5 điểm)	
Giao diện thân thiện với người dùng (0.5 điểm)	
Hướng dẫn sử dụng (0.5 điểm)	
Kết quả thực hiện đúng với kết quả của phần ứng dụng Giải thuật đúng, thực thi chính xác (2.5 điểm)	

Ghi chú:

- Điểm trong khung “sinh viên thực hiện” là điểm kết quả cuối cùng của từng sinh viên trong quá trình thực hiện Niên luận.../tiểu luận/khoá luận.
- Nếu sinh viên demo chương trình và trả lời vấn đáp không đạt yêu cầu của giáo viên hướng dẫn thì sinh viên sẽ nhận điểm F cho học phần này.

Cần Thơ, ngày tháng năm 2026

GIÁO VIÊN CHẤM

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN



.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày tháng năm 2026

Giáo viên hướng dẫn

.....

CHƯƠNG I. MỞ ĐẦU

1.1 Đặt vấn đề

Hiện nay, mua sắm trực tuyến đã trở thành thói quen tiêu dùng phổ biến. Với ngành hàng đồ thể thao, nhu cầu rèn luyện sức khỏe ngày càng tăng khiến lượng cầu về giày dép, trang phục và thiết bị tập luyện liên tục tăng lên. Trong khi đó, phần lớn cửa hàng đồ thể thao nhỏ vẫn bán hàng thủ công: ghi đơn bằng sổ tay, quản lý tồn kho bằng bảng tính — dẫn đến nhiều hạn chế:

- Tồn kho phức tạp do sản phẩm có nhiều **biến thể** (*màu sắc, kích cỡ*), dễ sai lệch khi theo dõi thủ công.
- Đơn hàng rải rác giữa tin nhắn, sổ ghi chép; tra cứu và đối soát chậm, dễ nhầm lẫn.
- Thiếu số liệu thống kê về doanh thu, hàng bán chạy, tồn kho để ra quyết định kinh doanh.
- Khó tiếp cận khách hàng ngoài phạm vi địa lý của cửa hàng.

Từ thực tế đó, chúng tôi xây dựng **SportNexus** — hệ thống thương mại điện tử chuyên về đồ thể thao, gồm website bán hàng cho khách hàng và hệ thống quản trị cho chủ cửa hàng, với các giải pháp chính:

- Quản lý sản phẩm **đa biến thể**, tồn kho theo dõi riêng từng biến thể, mọi thay đổi được ghi lịch sử truy vết.
- Đặt hàng chạy trong một **transaction**: tạo đơn, trừ tồn kho, phát hành hóa đơn, tạo vận đơn đồng bộ hoặc không xảy ra gì.
- Hỗ trợ thanh toán **COD / chuyển khoản** (*PayOS, mã QR + webhook ngân hàng*).
- Hệ thống quản trị có **dashboard thống kê, phân quyền RBAC, nhật ký hoạt động**; giao diện khách hỗ trợ **đa ngôn ngữ** (*Việt/Anh*).

Trên cơ sở đó, nhóm chọn đề tài "*Xây dựng hệ thống website thương mại điện tử SportNexus chuyên về đồ thể thao*", vừa mang giá trị sử dụng thực tiễn, vừa là cơ hội vận dụng kiến thức phát triển ứng dụng web full-stack vào bài toán nghiệp vụ cụ thể.

1.2 Mục tiêu cần đạt được

Xây dựng hoàn chỉnh hệ thống thương mại điện tử SportNexus gồm **website bán hàng cho khách hàng** và **hệ thống quản trị** đồng bộ qua một bộ API chung, cụ thể:

- Xây dựng website khách hàng cho phép duyệt, tìm kiếm, lọc sản phẩm theo danh mục/thương hiệu/giá; chọn biến thể (màu, kích cỡ) dựa trên tồn kho thực; quản lý giỏ hàng và đặt hàng trực tuyến.

- Triển khai nghiệp vụ đặt hàng trong một transaction: tạo đơn, trừ tồn kho, phát hành hóa đơn, tạo vận đơn đồng bộ; hỗ trợ thanh toán COD và chuyển khoản.

- Xây dựng hệ thống quản trị đầy đủ: sản phẩm đa biến thể, tồn kho – phiếu nhập có lịch sử truy vết, đơn hàng, coupon, hóa đơn, vận đơn, đánh giá.

- Xây dựng dashboard thống kê phục vụ ra quyết định kinh doanh (*doanh thu, đơn hàng, khách hàng, tồn kho...*).

- Phân quyền người dùng theo vai trò (*RBAC*), bảo mật bằng JWT, bcrypt và kiểm tra dữ liệu đầu vào.

- Giao diện tiếng Việt/tiếng Anh, hiển thị tốt trên máy tính lẫn thiết bị di động.

=> Kết quả mong đợi là một sản phẩm chạy được end-to-end, giải quyết được các vấn đề nêu tại mục 1.1 trong vận hành bán hàng thực tế của cửa hàng đồ thể thao.

1.3 Phạm vi nghiên cứu

1.3.1 Nghiên cứu đối tượng sử dụng

Hệ thống phục vụ ba nhóm đối tượng:

- **Khách vắng lai (chưa đăng nhập):** duyệt sản phẩm, dùng giỏ hàng lưu trên trình duyệt, đặt hàng bằng thông tin liên hệ và tra cứu vận đơn theo mã.
- **Khách hàng (đã đăng nhập):** đầy đủ chức năng của khách vắng lai, cộng với quản lý hồ sơ, sổ địa chỉ, xem đơn hàng – hóa đơn, đánh giá sản phẩm, lưu coupon và lịch sử tìm kiếm.
- **Quản trị viên / nhân viên:** thao tác trên hệ thống quản trị, quyền hạn được giới hạn theo vai trò và quyền được gán (*RBAC*).

1.3.2 Chức năng chính cho các đối tượng

- **Khách hàng:** duyệt – tìm kiếm – lọc sản phẩm; chọn biến thể theo tồn kho; giỏ hàng (*đồng bộ khi đăng nhập*); áp mã giảm giá; đặt hàng trong transaction; thanh toán COD/chuyển khoản; theo dõi vận đơn; đánh giá sau khi nhận hàng; quản lý hồ sơ, địa chỉ, hóa đơn, yêu thích, coupon đã lưu; gửi yêu cầu hỗ trợ qua email.

- **Quản trị viên:** dashboard thống kê 9 khối (*doanh thu, đơn hàng, khách hàng, tồn kho...*); quản lý người dùng – phân quyền; quản lý sản phẩm/biến thể/hình ảnh/thuộc tính; danh mục, thương hiệu, nhà cung cấp; coupon; đơn hàng; phiếu nhập; tồn kho có lịch sử biến động; hóa đơn; duyệt/ấn đánh giá; nhật ký hệ thống; nhập/xuất Excel.

1.3.3 Phạm vi công nghệ

- **Frontend:** React 19 + Vite, React Router, TanStack Query (caching), i18next đa ngôn ngữ, lazy loading.
- **Backend:** Node.js + Express 5, Prisma ORM với cơ sở dữ liệu MySQL, xác thực JWT (access + refresh token), mã hóa bcrypt, kiểm tra dữ liệu đầu vào bằng Joi, gửi email Nodemailer.
- **Dịch vụ ngoài:** Supabase Storage (lưu ảnh), PayOS / webhook Casso (thanh toán chuyển khoản).

1.3.4 Phạm vi triển khai

- Sản phẩm là website chạy trên trình duyệt, thiết kế responsive cho máy tính, tablet và điện thoại; chưa phát triển ứng dụng di động riêng.
- Kiến trúc tách biệt frontend và backend, giao tiếp qua REST API; backend đóng gói nghiệp vụ thành các service độc lập.
- Hệ thống vận hành với dữ liệu thật của cửa hàng: danh mục sản phẩm, tồn kho, đơn hàng, khách hàng; thanh toán trực tuyến phụ thuộc việc cấu hình PayOS/webhook ngân hàng.

1.4 Phương pháp nghiên cứu

1.4.1 Nghiên cứu tài liệu và cơ sở lý thuyết

Quá trình nghiên cứu tài liệu và khảo sát cơ sở lý thuyết đóng vai trò là nền tảng cốt lõi, định hình toàn bộ kiến trúc hệ thống và chiến lược phát triển của ứng dụng thương mại điện tử. Nội dung nghiên cứu được chia thành các nhóm trọng tâm sau:

- **Nghiên cứu tài liệu kỹ thuật chính thức (Official Documentation):** Tiến hành khảo sát sâu rộng các tài liệu kỹ thuật chuẩn của hệ sinh thái công nghệ được lựa chọn, bao gồm thư viện giao diện ReactJS, nền tảng dịch vụ mã nguồn mở ExpressJS, công cụ quản lý cơ sở dữ liệu Prisma ORM, và cơ chế xác thực an toàn JSON Web Token (JWT). Việc nắm vững các tiêu chuẩn kỹ thuật này giúp tối ưu hóa hiệu suất thực thi, đảm bảo tính bảo mật dữ liệu ở tầng truyền tải và lưu trữ, đồng thời tận dụng triệt để các ưu điểm về khả năng mở rộng của công nghệ hiện đại.
- **Khảo sát mô hình kiến trúc ứng dụng web phân lớp (Layered Architecture):** Nghiên cứu các nguyên lý thiết kế phần mềm hướng đối tượng và hướng dịch vụ, tập trung vào mô hình phân lớp (*Presentation Layer, Business Logic Layer, Data Access Layer*). Kiến trúc này giúp tách biệt rõ ràng các thành phần chức năng, giảm thiểu sự

phụ thuộc chéo, từ đó nâng cao tính dễ bảo trì, dễ kiểm thử và khả năng nâng cấp hệ thống trong dài hạn.

- **Phân tích mô hình phân quyền và kiểm soát truy cập (RBAC - Role-Based Access Control):** Tìm hiểu cơ chế phân quyền dựa trên vai trò nhằm xây dựng hệ thống bảo mật đa tầng. Mô hình này cho phép quản trị viên phân định rõ ràng quyền hạn của từng nhóm người dùng (khách hàng, nhân viên vận hành, quản trị hệ thống), ngăn chặn hiệu quả các hành vi truy cập trái phép và bảo vệ dữ liệu nhạy cảm của doanh nghiệp cũng như khách hàng.
- **Định hình nghiệp vụ thương mại điện tử chuyên sâu:** Nghiên cứu kỹ lưỡng các quy trình vận hành thực tế của một hệ thống thương mại điện tử quy mô vừa và nhỏ, bao gồm:
 - *Quản lý tồn kho đa biến thể:* Giải pháp quản lý sản phẩm có nhiều thuộc tính khác nhau (kích thước, màu sắc, chất lượng) gắn liền với mã định danh và số lượng tồn kho theo thời gian thực.
 - *Quản lý vận đơn:* Quy trình liên kết vận chuyển, cập nhật trạng thái đơn hàng từ lúc đặt hàng, đóng gói đến khi giao hàng thành công.
 - *Thanh toán trực tuyến:* Nghiên cứu các tiêu chuẩn tích hợp cổng thanh toán an toàn, đảm bảo tính toàn vẹn của giao dịch tài chính.

Kết quả nghiên cứu: Những tài liệu và mô hình trên là cơ sở khoa học và thực tiễn vững chắc để nhóm tác giả lựa chọn bộ công nghệ tối ưu, thiết kế luồng nghiệp vụ chuẩn hóa, đáp ứng chính xác các yêu cầu vận hành thực tế của một cửa hàng thương mại điện tử vừa và nhỏ.

1.4.2 Phân tích yêu cầu và thiết kế hệ thống

Để đảm bảo phần mềm phát triển bám sát nhu cầu vận hành thực tế của một cửa hàng đồ thể thao vừa và nhỏ, quá trình khảo sát được tiến hành trực tiếp trên các quy trình nghiệp vụ hiện tại. Kết quả phân tích được chia thành các nhóm yêu cầu cụ thể như sau.

1.4.2.1. Yêu cầu chức năng theo nhóm đối tượng

- **Nhóm Khách hàng (Customer):**
 - *Tìm kiếm và lọc sản phẩm:* Cho phép tra cứu nhanh theo tên, danh mục, thương hiệu, mức giá và các thuộc tính đa biến thể (kích thước, màu sắc).
 - *Quản lý giỏ hàng và thanh toán:* Hướng dẫn thêm/xóa sản phẩm, áp dụng mã

giảm giá (coupon), tích điểm thành viên và tạo đơn hàng trực tuyến.

- *Theo dõi đơn hàng*: Xem lại lịch sử giao dịch, trạng thái vận đơn, hóa đơn điện tử và đánh giá sản phẩm sau khi mua.

- **Nhóm Quản trị viên (Admin / Operator):**

- *Quản lý danh mục và sản phẩm*: Thêm mới, chỉnh sửa, ẩn/hiện sản phẩm đa biến thể, quản lý bộ sưu tập (*collection*) theo chiến dịch marketing.
- *Quản lý kho và nhà cung cấp*: Theo dõi số lượng tồn kho theo thời gian thực, lập phiếu nhập hàng (*purchase_orders*) từ các nhà cung cấp.
- *Quản lý hệ thống khách hàng và khuyến mãi*: Thiết lập hạng thành viên (*membership_tiers*), phát hành mã giảm giá, cấu hình chương trình tích điểm và kiểm soát nhật ký hoạt động (*audit_logs*) để đảm bảo bảo mật.

1.4.2.2. Yêu cầu phi chức năng

- **Bảo mật**: Mật khẩu người dùng phải được mã hóa bằng thuật toán an toàn (như Bcrypt); xác thực phiên làm việc bằng JSON Web Token (JWT); phân quyền chặt chẽ theo mô hình RBAC; chống tấn công SQL Injection và XSS ở tầng API.
- **Hiệu năng**: Hệ thống phải đảm bảo thời gian phản hồi API trung bình dưới 500ms đối với các truy vấn thông thường; hỗ trợ cơ chế phân trang và index cơ sở dữ liệu MySQL tối ưu cho lượng lớn dữ liệu phát sinh.
- **Tính mở rộng và quốc tế hóa**: Thiết kế kiến trúc dạng tách biệt hoàn toàn giữa Frontend và Backend (Decoupled Architecture) giao tiếp qua REST API, sẵn sàng mở rộng các tính năng mới hoặc nâng cấp giao diện trong tương lai.

1.4.3. Kiểm thử và đánh giá hệ thống

1.5 Hướng giải quyết

Để giải quyết các vấn đề nêu tại mục 1.1, hệ thống được thiết kế theo hướng:

- **Kiến trúc client-server tách biệt**: frontend là ứng dụng SPA (React), backend là REST API (Express) chia thành các tầng rõ ràng — route nhận yêu cầu, controller xử lý HTTP, service chứa nghiệp vụ, Prisma thao tác dữ liệu — giúp dễ bảo trì và mở rộng.

- **Nghiệp vụ trọng yếu đảm bảo toàn vẹn**: luồng đặt hàng gom toàn bộ thao tác ghi (tạo đơn, trừ tồn kho từng biến thể, phát hành hóa đơn, tạo vận đơn) vào một transaction của cơ sở dữ liệu, loại trừ tình trạng dữ liệu lệch khi lỗi xảy ra giữa chừng.

- **Quản lý tồn kho theo biến thể và truy vết:** tồn kho tính riêng theo tổ hợp màu/kích cỡ; mọi nhập – xuất – điều chỉnh đều ghi bản ghi biến động kho kèm tham chiếu nguồn gốc.
- **Bảo mật nhiều lớp:** xác thực bằng cặp access/refresh token (JWT), mã hóa mật khẩu bcrypt, kiểm tra đầu vào bằng Joi, phân quyền RBAC theo vai trò và quyền trên từng module.
- **Tái sử dụng dịch vụ sẵn có thay vì tự xây:** lưu ảnh trên Supabase Storage, thanh toán chuyển khoản qua PayOS hoặc webhook Casso, gửi email qua Nodemailer, mô phỏng giao hàng theo mẫu GHN — giúp rút ngắn thời gian phát triển và giảm rủi ro.
- **Trải nghiệm người dùng:** giao diện SPA với lazy loading và caching (TanStack Query), đa ngôn ngữ Việt/Anh bằng i18next, hiển thị responsive trên mọi thiết bị.

1.6 Kế hoạch thực hiện

Đề tài được thực hiện theo các giai đoạn nối tiếp nhau:

Tuần	Nội dung công việc	Kết quả đạt được
1	Khảo sát nghiệp vụ bán đồ thể thao; nghiên cứu công nghệ (React, Express, Prisma)	Hiểu bài toán, chốt công nghệ sử dụng
2	Xác định yêu cầu chức năng và phi chức năng theo từng đối tượng	Danh sách use case khách hàng (UC-A/B/C) và quản trị (UC-M)
3	Thiết kế hệ thống	Sơ đồ use case, tuần tự, hoạt động; ERD MySQL; kiến trúc client–server
4	Xây dựng backend: schema Prisma, API xác thực (đăng ký/đăng nhập/JWT), quản lý người dùng	API tài khoản hoàn chỉnh, mã hóa bcrypt, Joi validation
5	Xây dựng backend: API sản phẩm, biến thể, hình ảnh, danh mục, thương hiệu, nhà cung cấp	Module dữ liệu bán hàng hoạt động
6	Xây dựng backend: giỏ hàng, đặt hàng (transaction), coupon, tồn kho – phiếu nhập, hóa đơn, vận đơn	Nghiệp vụ trọng yếu chạy đúng toàn vẹn dữ liệu
7	Xây dựng website khách hàng: trang chủ, danh sách/chi tiết sản phẩm, lọc – tìm kiếm, chọn biến	Giao diện mua sắm hiển thị đầy đủ

	thể	
8	Xây dựng website khách hàng: giỏ hàng, checkout, thanh toán COD/chuyển khoản, theo dõi đơn, hồ sơ – địa chỉ	Luồng mua hàng end-to-end cho khách
9	Xây dựng hệ thống quản trị: quản lý sản phẩm/tồn kho/đơn hàng/coupon, người dùng – phân quyền RBAC	Quản trị viên thao tác được toàn bộ dữ liệu
10	Dashboard thống kê; tích hợp dịch vụ ngoài: Supabase Storage, Nodemailer, PayOS/Casso, mô phỏng GHN	Các tính năng phụ trợ hoạt động thực tế
11	Kiểm thử thủ công theo kịch bản use case, chạy end-to-end, sửa lỗi, tối ưu	Sản phẩm ổn định trước khi bàn giao
12	Hoàn thiện tính năng, viết báo cáo, chuẩn bị demo	Báo cáo và sản phẩm hoàn chỉnh

Bảng 1. 1 Nội dung công việc

Các giai đoạn xây dựng backend và frontend (tuần 4–9) có thể đan xen nhau vì hai phần phát triển song song theo từng nhóm API; tiến độ được rà soát hằng tuần để điều chỉnh kịp thời.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ



2.1 Mô hình kinh doanh B2C

Trong thương mại điện tử, mô hình **B2C** (*Business-to-Consumer*) là mô hình trong đó doanh nghiệp/cửa hàng bán sản phẩm trực tiếp cho người tiêu dùng cuối cùng thông qua kênh trực tuyến. Khác với **C2C** (*Consumer-to-Consumer*) — nơi các cá nhân mua bán lại với nhau và nền tảng chỉ đóng vai trò trung gian hoa hồng — trong mô hình B2C, bên bán là một đầu mối duy nhất, chịu trách nhiệm toàn bộ từ khâu nhập hàng, quản lý tồn kho, định giá, giao hàng đến chăm sóc sau bán.

SportNexus được xây dựng bám sát đặc điểm của mô hình B2C cho ngành hàng đồ thể thao:

- **Một đầu bán duy nhất:** toàn bộ sản phẩm trên website do cửa hàng quản lý và niêm yết; khách hàng chỉ mua – không đăng bán hàng như sàn thương mại C2C.
- **Chủ động nguồn hàng:** cửa hàng nhập hàng từ các **nhà cung cấp** qua phiếu nhập, theo dõi tồn kho và lịch sử biến động kho để quyết định nhập thêm.
- **Kiểm soát trọn vòng lệnh bán:** từ khi khách đặt hàng, thanh toán đến khi tạo vận đơn giao đi và phát hành hóa đơn, mọi bước đều do hệ thống quản trị của cửa hàng xử lý và giám sát.
- **Quan hệ lâu dài với khách hàng:** khách hàng có tài khoản, lịch sử đơn hàng, điểm đánh giá, coupon được tặng — giúp cửa hàng chăm sóc và giữ chân khách, điều mà mô hình C2C khó thực hiện hiệu quả.

Việc lựa chọn mô hình B2C phù hợp với bài toán đặt ra tại Chương I: cửa hàng đồ thể thao cần một kênh bán hàng trực tuyến thuộc quyền sở hữu để chuyên nghiệp hóa quy trình bán và quản lý, thay vì phụ thuộc vào các nền tảng trung gian.

2.2 Mô hình Client – Server

2.2.1 Khái niệm

- Mô hình **Client – Server** là kiến trúc phân tán nền tảng và phổ biến nhất trong phát triển ứng dụng web hiện đại. Theo mô hình này, hệ thống được phân rã thành hai thành phần chính:

- **Client (Máy khách):** Chịu trách nhiệm về giao diện người dùng, định tuyến trang (Routing), quản lý trạng thái (State Management) và gọi các dịch vụ API. Trong hệ

thống **SportNexus**, tầng client được xây dựng dưới dạng một Ứng dụng trang đơn (**Single Page Application - SPA**) sử dụng thư viện **React**.

- **Server (Máy chủ):** Chịu trách nhiệm xác thực người dùng, xử lý logic nghiệp vụ và truy xuất cơ sở dữ liệu. Trong hệ thống này, tầng server là một **RESTful API** được phát triển trên nền tảng **Node.js** sử dụng framework **Express**.
- **Phương thức giao tiếp:** Giao tiếp giữa client và server thông qua giao thức **HTTP** với định dạng trao đổi dữ liệu tiêu chuẩn **JSON**, tuân thủ theo phong cách **RESTful**.

2.2.2 Lợi ích của mô hình client–server

Lợi ích cốt lõi	Mô tả chi tiết
Tách biệt trách nhiệm	Mỗi thành phần chỉ đảm nhiệm một chức năng chuyên biệt, giúp mã nguồn sáng sủa và dễ kiểm soát.
Dễ dàng mở rộng	Cho phép mở rộng quy mô (Scale) của tầng client hoặc server một cách độc lập tùy theo nhu cầu thực tế.
Đa dạng hóa máy khách	Một máy chủ trung tâm có thể phục vụ đồng thời nhiều loại client khác nhau (như Web, Mobile App, Desktop).
Bảo mật tập trung	Toàn bộ logic kiểm tra quyền hạn và bảo mật được tập trung xử lý tại server, hạn chế rủi ro lộ dữ liệu.
Tính bảo trì cao	Sự độc lập giữa hai phân hệ giúp việc thay đổi, nâng cấp giao diện hoặc logic phía backend không làm ảnh hưởng lẫn nhau.

Bảng 2. 1 Lợi ích mô hình client – server

2.2.3 Sơ đồ kiến trúc tổng thể trong SportNexus

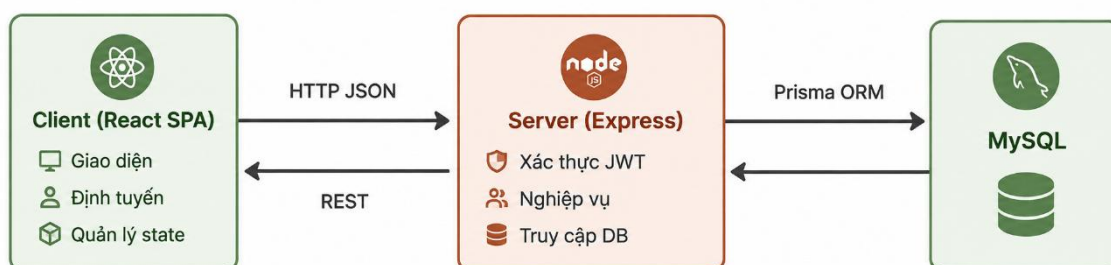
Mối quan hệ tương tác và luồng dữ liệu giữa các thành phần trong hệ thống SportNexus được minh họa qua sơ đồ tổng thể, thể hiện rõ mô hình kiến trúc phân lớp tách biệt giữa giao diện người dùng và tầng xử lý nghiệp vụ:

- **Tầng Giao diện người dùng (Client - React SPA):** Đóng vai trò là Single Page Application (*SPA*) xây dựng bằng thư viện ReactJS. Tầng này chịu trách nhiệm hoàn toàn về việc hiển thị giao diện, quản lý định tuyến phía client (*routing*) và kiểm soát trạng thái ứng dụng (*state management*). Client tương tác trực tiếp với người dùng

cuối, tiếp nhận các yêu cầu thao tác và gửi thông tin truy vấn thông qua các phương thức chuẩn.

- **Tầng Xử lý nghiệp vụ (Server - ExpressJS):** Đóng vai trò là trung tâm điều phối của hệ thống, được phát triển trên nền tảng ExpressJS. Tầng này thực hiện các nhiệm vụ cốt lõi bao gồm: xử lý xác thực bảo mật thông qua cơ chế JSON Web Token (JWT), thực thi toàn bộ logic nghiệp vụ thương mại điện tử, và quản lý giao tiếp với cơ sở dữ liệu.
- **Tầng Lưu trữ dữ liệu (Database - MySQL & Prisma ORM):** Toàn bộ dữ liệu của hệ thống từ sản phẩm, đơn hàng, khách hàng cho đến nhật ký hoạt động đều được lưu trữ an toàn trên cơ sở dữ liệu quan hệ MySQL. Việc kết nối và thao tác dữ liệu từ tầng Server xuống MySQL được thực hiện thông qua Prisma ORM, giúp tối ưu hóa các câu lệnh truy vấn, đảm bảo tính toàn vẹn và an toàn cho dữ liệu.

Luồng truyền thông tin: Giao tiếp giữa Client và Server được thực hiện thông qua giao thức HTTP/HTTPS sử dụng định dạng dữ liệu chuẩn JSON dựa trên kiến trúc RESTful API, đảm bảo sự linh hoạt, tốc độ phản hồi nhanh chóng và khả năng mở rộng dễ dàng cho hệ thống SportNexus trong tương lai.



Hình 2. 1. Tổng hợp lợi ích của mô hình Client – Server

2.2.4 Phân tích sâu việc áp dụng trong SportNexus

Khía cạnh kiến trúc	Giải pháp triển khai thực tế trong SportNexus
Tầng giao diện (Presentation)	Sử dụng React SPA chịu trách nhiệm xử lý định tuyến phía client, quản lý state ứng dụng và render component tương tác.

Tầng nghiệp vụ (Business Logic)	Sử dụng Express API đảm nhận toàn bộ việc xác thực, kiểm tra quyền hạn (RBAC) và xử lý các luồng nghiệp vụ phức tạp.
Tầng dữ liệu (Data Access)	Sử dụng cơ sở dữ liệu quan hệ MySQL , tương tác thông qua công cụ ánh xạ đối tượng Prisma ORM .
Giao thức truyền thông	Giao tiếp đồng bộ qua HTTP + JSON theo chuẩn thiết kế RESTful API .

Bảng 2. 2. Khía cạnh áp dụng mô hình Client – Server trong hệ thống SportNexus

+ **Phân tích về tính phân tách trách nhiệm:** Việc giao cho server xử lý toàn bộ logic nghiệp vụ và cơ chế bảo mật mang lại ý nghĩa vô cùng quan trọng — tầng client đóng vai trò là "tầng trình diễn" thuần túy, hoàn toàn không chứa các đoạn mã logic nhạy cảm. Nhờ đó, khi hệ thống có nhu cầu mở rộng sang các nền tảng client mới (ví dụ: Mobile App), đội ngũ phát triển chỉ cần tái sử dụng hoàn toàn hệ thống API hiện có mà không phải viết lại từ đầu logic nghiệp vụ.

+ **Phân tích về trạng thái hệ thống (Stateless):** Kiến trúc REST được áp dụng theo cơ chế phi trạng thái (*stateless*) — mỗi HTTP request gửi lên server đều chứa đầy đủ thông tin định danh (thông qua mã thông báo **JWT**), cho phép server xử lý các yêu cầu một cách độc lập. Điều này giúp hệ thống dễ dàng mở rộng theo chiều ngang (*horizontal scaling* bằng cách tăng số lượng instance server) mà không gặp trở ngại về việc phải chia sẻ bộ nhớ phiên làm việc (*session*).

2.3 Kiến trúc 3 lớp (Three-Layer Architecture)

2.3.1 Tổng quan kiến trúc

- Hệ thống Backend của SportNexus được tổ chức theo mô hình Kiến trúc 3 lớp (Three-Layer Architecture) — một trong những mẫu kiến trúc phân lớp kinh điển và phổ biến nhất trong phát triển phần mềm nhằm đảm bảo tính mạch lạc của mã nguồn:

- **Presentation Layer (Controller - Thư mục controllers/):** Đóng vai trò là tầng giao tiếp trực tiếp với bên ngoài, chịu trách nhiệm tiếp nhận các yêu cầu HTTP từ định tuyến (*Route*), thực hiện kiểm tra định dạng dữ liệu đầu vào, chuyển giao cho tầng dịch vụ xử lý nghiệp vụ, và đóng gói kết quả để trả về phản hồi (*Response*) chuẩn xác cho người dùng hoặc ứng dụng client..
- **Business Logic Layer (Service - Thư mục services/):** Đóng vai trò là tầng trung tâm của hệ thống, chứa toàn bộ logic nghiệp vụ cốt lõi (như xử lý đặt hàng, áp dụng mã

giảm giá, tính toán tồn kho đa biến thể). Tầng này điều phối các quy tắc xử lý phức tạp, kiểm tra tính hợp lệ về mặt nghiệp vụ và kết nối trực tiếp với tầng dữ liệu để thực thi các giao dịch cần thiết. **Data Access Layer:** Tầng truy xuất dữ liệu thông qua công cụ ánh xạ đối tượng Prisma ORM để tương tác an toàn với hệ quản trị cơ sở dữ liệu quan hệ MySQL.

- **Data Access Layer (Prisma ORM & MySQL):** Tầng truy xuất dữ liệu thông qua công cụ ánh xạ đối tượng hiện đại **Prisma ORM**, giúp giao tiếp an toàn, đồng bộ và hiệu quả với hệ quản trị cơ sở dữ liệu quan hệ **MySQL**, che giấu các câu lệnh SQL phức tạp thông qua các mô hình dữ liệu kiểu TypeScript-safe.

2.3.2 Ưu điểm của kiến trúc 3 lớp

Ưu điểm cốt lõi	Mô tả chi tiết
Tách biệt mối quan tâm	Tách rời các xử lý liên quan đến giao thức HTTP ra khỏi logic nghiệp vụ; tầng Controller chỉ thực hiện nhiệm vụ "tiếp nhận – chuyển gọi – phản hồi", trong khi toàn bộ quyết định logic được đóng gói tại Service.
Tính bảo trì cao	Khi có sự thay đổi về yêu cầu nghiệp vụ, nhà phát triển chỉ cần chỉnh sửa tại tầng Service mà không làm ảnh hưởng đến tầng điều khiển (Controller).
Thuận lợi cho kiểm thử	Cho phép tiến hành kiểm thử các hàm logic nghiệp vụ tại tầng Service một cách độc lập mà không cần phụ thuộc vào môi trường yêu cầu HTTP.
Khả năng tái sử dụng	Một phương thức xử lý tại tầng Service có thể được gọi linh hoạt từ nhiều Controller khác nhau hoặc từ các module chức năng độc lập khác trong hệ thống.

Bảng 2. 3. Ưu điểm của mô hình kiến trúc 3 lớp trong hệ thống Backend

2.3.3 Ví dụ minh họa luồng nghiệp vụ (Luồng tạo đơn hàng)

Quá trình xử lý một yêu cầu tạo đơn hàng từ phía khách hàng được tổ chức theo mô hình kiến trúc phân lớp, đảm bảo tính tuần tự, an toàn dữ liệu giao dịch và tính toàn vẹn của hệ thống kho vận. Luồng tương tác chi tiết qua từng bước bao gồm:

Tiếp nhận và Kiểm tra sơ bộ tại Route Layer:

- Khi client gửi yêu cầu POST /customer/order/, yêu cầu trước tiên đi qua tầng định tuyến (Route).
- Tại đây, hệ thống kích hoạt các **middleware** quan trọng:
 - `verifyTokenOptional`: Kiểm tra thông tin xác thực của người dùng (nếu có đăng nhập, nhằm liên kết đơn hàng với tài khoản cá nhân và tích điểm thành viên).
 - `validate` (middleware — Joi): Sử dụng thư viện Joi để kiểm tra cấu trúc, định dạng và tính hợp lệ của dữ liệu JSON gửi lên từ client (như thông tin người nhận, danh sách sản phẩm, số lượng, mã giảm giá).

2. Xử lý tại Controller Layer:

- Sau khi vượt qua các lớp lọc và kiểm tra định dạng, yêu cầu được chuyển giao đến hàm `orderController.create`.
- Tầng Controller đóng vai trò điều phối, chịu trách nhiệm nhận payload từ request, trích xuất dữ liệu cần thiết và gọi tầng dịch vụ tương ứng.

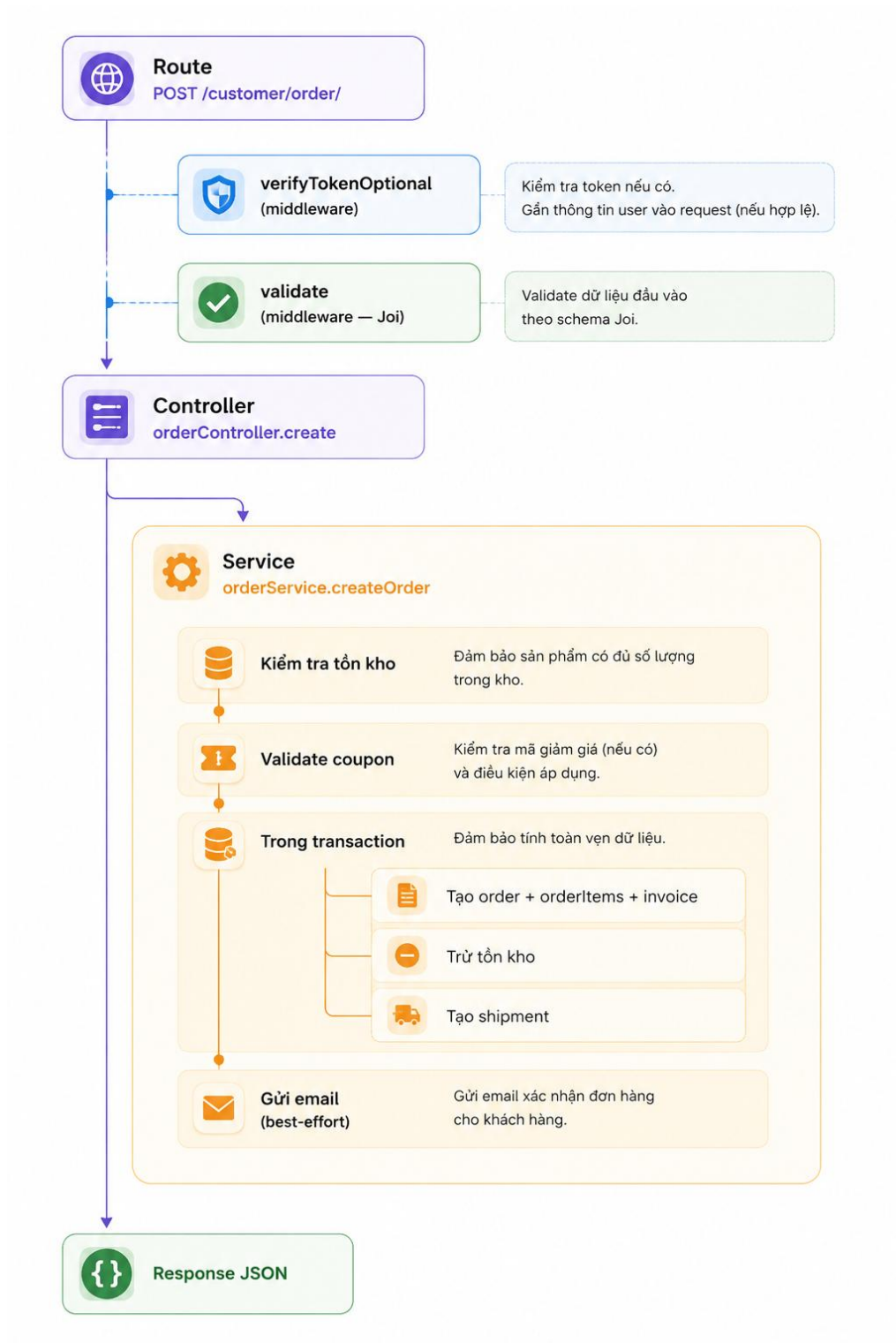
3. Thực thi nghiệp vụ tại Service Layer (`orderService.createOrder`):

- **Kiểm tra tồn kho:** Hệ thống truy vấn cơ sở dữ liệu để xác minh số lượng tồn kho thực tế của các biến thể sản phẩm (`product_variants`) trong giỏ hàng, đảm bảo đủ điều kiện cung ứng.
- **Validate coupon:** Kiểm tra tính hợp lệ của mã giảm giá (thời hạn sử dụng, giá trị đơn hàng tối thiểu, giới hạn số lần dùng của người dùng).
- **Thực thi giao dịch an toàn (Database Transaction):** Trong một khối transaction khép kín nhằm đảm bảo tính nguyên tử (*Atomicity*):
 - *Tạo đơn hàng:* Khởi tạo bản ghi trong bảng `orders`, đi kèm chi tiết sản phẩm (`order_items`) và hóa đơn điện tử (`invoices`).
 - *Trừ tồn kho:* Cập nhật giảm số lượng tương ứng của các biến thể sản phẩm trong kho.
 - *Tạo vận đơn:* Khởi tạo thông tin giao hàng (`shipment`) cho đơn hàng vừa tạo.
- **Gửi thông báo (Best-effort):** Sau khi transaction hoàn tất thành công, hệ thống tiến hành gửi email xác nhận đơn hàng cho khách hàng theo cơ chế bất

đồng bộ (best-effort), không làm ảnh hưởng đến thời gian phản hồi chính của API.

4. Trả về kết quả (Response JSON):

- Cuối cùng, kết quả xử lý thành công hoặc mã lỗi chi tiết được đóng gói thành định dạng chuẩn JSON và trả về cho client.



Hình 2. 2 Sơ đồ luồng nghiệp vụ tạo đơn hàng

Tầng Định tuyến và Kiểm duyệt (Route & Middlewares)

- **Route (POST /customer/order/):** Điểm tiếp nhận yêu cầu gửi từ phía Client (Frontend) lên server.
- **verifyTokenOptional:** Middleware kiểm tra mã thông thực thực (*JWT*). Chữ "Optional" thể hiện tính linh hoạt: hệ thống cho phép cả khách vắng lai (chưa đăng nhập) lẫn thành viên chính thức thực hiện đặt hàng.
- **validate (middleware - Joi):** Sử dụng thư viện Joi để kiểm tra cấu trúc dữ liệu đầu vào (payload validation), đảm bảo các trường bắt buộc như địa chỉ, sản phẩm, số lượng đều tuân thủ đúng định dạng trước khi chuyển vào tầng xử lý logic.

Tầng Điều khiển (Controller - orderController.create)

- Đóng vai trò trung gian tiếp nhận dữ liệu đã được làm sạch từ middleware, sau đó bóc tách dữ liệu và gọi hàm tương ứng từ tầng Service để xử lý nghiệp vụ chính.

Tầng Nghiệp vụ (Service - orderService.createOrder) Đây là nơi tập trung toàn bộ logic nghiệp vụ phức tạp của thương mại điện tử:

- **Kiểm tra tồn kho:** Truy vấn cơ sở dữ liệu để xác thực xem các biến thể sản phẩm trong giỏ hàng còn đủ số lượng đáp ứng hay không.
- **Validate coupon:** Kiểm tra tính hợp lệ của mã giảm giá (thời hạn sử dụng, giá trị tối thiểu, số lượt dùng còn lại).
- **Khởi giao dịch an toàn (Transaction):** Đảm bảo tính nguyên tử (*Atomicity*) tuyệt đối theo nguyên lý ACID. Nếu bất kỳ thao tác nào trong ba hành động dưới đây gặp sự cố, toàn bộ tiến trình sẽ được hoàn tác (rollback):
 - *Tạo order + orderItems + invoice:* Ghi nhận thông tin đơn hàng, chi tiết các sản phẩm mua và hóa đơn thanh toán.
 - *Trừ tồn kho:* Cập nhật giảm số lượng tương ứng trong bảng biến thể sản phẩm (ProductVariants).
 - *Tạo shipment:* Khởi tạo bản ghi vận đơn giao hàng.
- **Gửi email (best-effort):** Gửi thư điện tử xác nhận đơn hàng tới khách hàng theo cơ chế phi đồng bộ (nếu tiến trình gửi email có lỗi phát sinh ngoài ý muốn cũng không làm ảnh hưởng đến giao dịch đặt hàng đã thành công).

Phản hồi kết quả (Response JSON)

- Cuối cùng, hệ thống đóng gói kết quả trả về cho Client dưới dạng định dạng **JSON** kèm theo mã trạng thái HTTP chuẩn (ví dụ: 201 Created khi thành công hoặc 400/500 khi có lỗi).

2.3.4 Nhược điểm và phương án khắc phục của kiến trúc 3 lớp

- Mặc dù mang lại nhiều ưu điểm vượt trội, kiến trúc 3 lớp cũng tồn tại một số điểm hạn chế nhất định trong quá trình phát triển thực tế. Hệ thống SportNexus đã áp dụng các giải pháp linh hoạt để khắc phục:

Nhược điểm tiềm ẩn của kiến trúc	Phương án khắc phục linh hoạt trong SportNexus
Phình to số lượng tệp mã nguồn: Mỗi tính năng mới đều đòi hỏi phải khởi tạo nhiều tệp riêng biệt (Route, Controller, Service, Validator).	Tuân thủ chặt chẽ quy ước tổ chức cấu trúc thư mục chuẩn hóa, giúp việc định vị và quản lý mã nguồn trở nên trực quan, dễ dàng.
Phức tạp hóa các tính năng đơn giản: Đối với các chức năng truyền dữ liệu đơn giản, việc đi qua đầy đủ các tầng trung gian có phần dư thừa.	Cho phép tối giản hóa luồng xử lý đối với các tính năng nhỏ (ví dụ: route có thể gọi trực tiếp service nếu không cần qua bước xử lý phức tạp).
Nguy cơ lạm dụng tầng trung gian: Dễ dẫn đến tình trạng tạo ra quá nhiều lớp code bao bọc không cần thiết cho các thao tác cơ bản.	Chỉ tiến hành khởi tạo các lớp trung gian như Validator hay Service khi thực sự có nhu cầu đóng gói logic nghiệp vụ hoặc kiểm tra dữ liệu chuyên sâu.

Bảng 2. 4. Nhược điểm của kiến trúc 3 lớp và giải pháp khắc phục trong hệ thống

2.4 Công nghệ Frontend

2.4.1 React 19

- **React** là thư viện JavaScript phổ biến hàng đầu hiện nay dùng để xây dựng giao diện người dùng theo mô hình thành phần (*component*). Phiên bản **React 19** mang đến nhiều cải tiến vượt trội về hiệu năng và trải nghiệm phát triển:

- **Component-based (Kiến trúc thành phần):** Giao diện của ứng dụng được phân rã thành các thành phần độc lập, có tính mô-đun cao, dễ dàng đóng gói và tái sử dụng trên nhiều trang khác nhau, giúp giảm thiểu trùng lặp mã nguồn và tăng tốc độ xây dựng tính năng.
- **Declarative (Lập trình hướng khai báo):** Lập trình viên chỉ cần mô tả trạng thái giao diện mong muốn (*UI state*) ứng với mỗi mốc dữ liệu, React sẽ tự động quản lý quá trình cập nhật DOM ngầm định một cách thông minh và đồng bộ.

- **Virtual DOM:** Tối ưu hóa hiệu năng render bằng cách sử dụng cơ chế DOM ảo, thực hiện so sánh sự thay đổi (*diffing algorithm*) giữa các phiên bản và chỉ cập nhật tối thiểu các phần tử thực tế cần thiết trên trang, hạn chế tối đa việc thao tác trực tiếp với DOM gây tốn tài nguyên.
- **Hooks:** Cung cấp cơ chế quản lý trạng thái (*state*) và các hiệu ứng phụ (*side effects*) một cách hiện đại, trực quan mà không cần sử dụng các component dạng lớp (*class components phức tạp*).
- **Ứng dụng thực tế trong SportNexus:** Trong dự án này, React 19 không chỉ được sử dụng để xây dựng giao diện Single Page Application (*SPA*) mượt mà mà còn kết hợp chặt chẽ với kỹ thuật tải chậm (*Lazy loading*) cho từng tuyến đường (*routes*). Giải pháp này giúp tối ưu hóa dung lượng gói tin tải ban đầu (*bundle size*), tăng tốc độ hiển thị trang (*Initial Load*) và mang lại trải nghiệm tương tác liền mạch cho người dùng cuối.

2.4.2 Vite

- Vite là công cụ xây dựng (*build tool*) và máy chủ phát triển (*development server*) thế hệ mới dành cho các ứng dụng JavaScript và TypeScript, nổi bật với tốc độ vượt trội và trải nghiệm phát triển tối ưu:

Khởi động siêu tốc: (Instant Server Start): Khác với các công cụ truyền thống phải gom nhóm (*bundle*) toàn bộ mã nguồn trước khi khởi động, Vite tận dụng các mô-đun ES nguyên bản (*native ES modules*) trên trình duyệt để phục vụ mã nguồn theo yêu cầu, giúp thời gian khởi động máy chủ phát triển gần như tức thì ngay cả với các dự án lớn.

HMR (Hot Module Replacement) mượt mà: Cung cấp cơ chế cập nhật mô-đun nóng cực kỳ nhanh chóng và chính xác. Mọi thay đổi về mã nguồn (*source code*) được phản ánh tức thì trên trình duyệt mà không làm mất đi trạng thái hiện tại (*state*) của ứng dụng, giúp nâng cao đáng kể năng suất của lập trình viên.

Tối ưu hóa Production Build: Ở giai đoạn xuất bản sản phẩm, Vite tích hợp sẵn công cụ đóng gói mạnh mẽ Rollup, hỗ trợ các tính năng tiên tiến như tách mã (*code-splitting*), tối ưu tài nguyên và loại bỏ mã thừa (*tree-shaking*) cực kỳ hiệu quả, giúp giảm thiểu dung lượng tệp tin tĩnh và tăng tốc độ tải trang cho người dùng cuối.

2.4.3 React Router 7

React Router là thư viện định tuyến chuẩn mực và phổ biến nhất cho các ứng dụng một trang (*Single Page Application - SPA*) sử dụng React. Phiên bản React 7 mới nhất mang

đến nhiều cải tiến kiến trúc đột phá, cung cấp hệ thống quản lý điều hướng mạnh mẽ và linh hoạt:

- **Data Loaders (Bộ nạp dữ liệu):** Cho phép định nghĩa trước các hàm tải dữ liệu gắn liền với từng route. Dữ liệu sẽ được nạp song song với quá trình chuyển trang trước khi render route, giúp giải quyết triệt để hiện tượng chớp nháy màn hình chờ (*flash of loading*), tối ưu hóa trải nghiệm người dùng.
- **Server-driven navigation:** Cung cấp khả năng điều hướng do máy chủ kiểm soát, hỗ trợ tối ưu các mô hình kết xuất nâng cao như Kết xuất tĩnh (*Static Site Generation - SSG*) hoặc Kết xuất phía máy chủ (*Server-Side Rendering - SSR*).
- **Route Organization (Tổ chức định tuyến phân cấp):** Cây định tuyến được phân rã và tổ chức theo cấu trúc phân cấp rõ ràng, giúp lập trình viên dễ dàng quản lý quyền truy cập, các Layout chung và các trang con một cách khoa học.
- **Ứng dụng thực tế trong SportNexus:** Trong hệ thống SportNexus, cây định tuyến được phân chia rạch ròi thành ba nhóm chính bao gồm:
 - *webRoutes*: Định tuyến dành cho khu vực mua sắm của khách hàng.
 - *authRoutes*: Định tuyến xử lý xác thực (đăng nhập, đăng ký, quên mật khẩu).
 - *adminRoutes*: Định tuyến dành riêng cho khu vực quản trị hệ thống.

Đặc biệt, toàn bộ các nhóm route này đều được áp dụng kỹ thuật tải chậm (*lazy-load*) kết hợp bọc trong thành phần `<Suspense>` của React nhằm tối ưu hóa hiệu năng khởi động và tiết kiệm tài nguyên mạng.

2.4.4 TanStack Query

TanStack Query (trước đây là React Query) là thư viện chuyên biệt giải quyết bài toán quản lý trạng thái phía máy chủ (*server state*), khắc phục các phức tạp trong việc đồng bộ dữ liệu giữa client và server:

- **Cache thông minh:** Dữ liệu phản hồi được lưu trữ tạm thời (*cache*) nhằm hạn chế tối đa các lượt gọi API trùng lặp.
- **Refetch tự động:** Cơ chế tự động làm mới dữ liệu khi có sự kiện kích hoạt phù hợp.
- **Invalidation:** Hủy bỏ cache cũ và tự động kích hoạt nạp lại dữ liệu mới sau khi thực hiện các thao tác thay đổi (*mutation*).

- **Optimistic Updates:** Cập nhật giao diện ngay lập tức trước khi nhận phản hồi từ server, mang lại cảm giác mượt mà cho người dùng.
- *Ứng dụng trong SportNexus:* queryClient được cấu hình với thông số staleTime: 5 phút và refetchOnWindowFocus: false. Các loaders sử dụng queryClient.fetchQuery để nạp sẵn dữ liệu vào cache, đồng thời gọi invalidateQueries sau mỗi thao tác mutation.

2.4.5 Tailwind CSS

Tailwind CSS là framework CSS hiện đại được xây dựng dựa trên triết lý tiện ích trước (*utility-first*). Thay vì sử dụng các lớp mang ngữ cảnh trừu tượng truyền thống (như .btn-primary, .card), lập trình viên có thể sử dụng trực tiếp các lớp tiện ích có sẵn gắn gọn trong mã nguồn (như px-4 py-2 bg-blue-500), mang lại những ưu điểm vượt trội:

- **Tốc độ thiết kế cao:** Cho phép phát triển và tinh chỉnh giao diện một cách nhanh chóng ngay trên tệp JSX/TSX mà không cần chuyển đổi liên tục giữa tệp mã nguồn thành phần và tệp CSS riêng biệt, giúp tối ưu hóa đáng kể thời gian xây dựng giao diện.
- **Hỗ trợ Responsive linh hoạt:** Cung cấp hệ thống điểm gãy (*breakpoints*) trực quan thông qua các tiền tố có sẵn (như sm:, md:, lg:, xl:), giúp dễ dàng thiết kế giao diện đáp ứng hoàn hảo trên mọi kích thước thiết bị từ di động đến máy tính để bàn.
- **Tối ưu hóa dung lượng:** Tự động quét và loại bỏ toàn bộ các lớp tiện ích không được sử dụng đến trong quá trình đóng gói sản phẩm (*tree-shaking* với công cụ PurgeCSS/Tailwind CLI), đảm bảo tệp tin CSS xuất bản có dung lượng cực kỳ nhỏ gọn, tối ưu hóa tốc độ tải trang cho người dùng cuối.

2.4.6 Các thư viện hỗ trợ Frontend khác

Thư viện hỗ trợ	Mục đích sử dụng chính trong SportNexus
react-hook-form + zod	Quản lý biểu mẫu dữ liệu và kiểm tra tính hợp lệ (<i>validation</i>) đầu vào chặt chẽ.
i18next (react-i18next)	Hỗ trợ đa ngôn ngữ linh hoạt (chuyển đổi qua lại giữa Tiếng Việt và Tiếng Anh).
lucide-react	Cung cấp bộ biểu tượng (<i>icons</i>) trực quan, sắc nét.

swiper	Xây dựng các khung trình chiếu hình ảnh (<i>carousels / sliders</i>) mượt mà.
sonner	Hiển thị thông báo dạng bật lên (<i>toast notifications</i>) hiện đại.
axios	Thư viện thực hiện các yêu cầu HTTP client kết nối với backend.
dayjs	Thư viện xử lý, định dạng và tính toán thời gian, ngày tháng.
clsx + tailwind-merge	Hỗ trợ gộp nhóm các class CSS có điều kiện một cách tối ưu và an toàn.

Bảng 2. 5. Tổng hợp các thư viện hỗ trợ phát triển phía Frontend

2.5 Công nghệ Backend và Cơ sở dữ liệu

2.5.1 Javascript

2.5.1.1 Tổng quan

JavaScript là ngôn ngữ chính của cả ứng dụng di động (React Native) và máy chủ (Node.js). Ngôn ngữ hỗ trợ ES Modules (import/export), Promise, async/await, và thao tác JSON.

Trong dự án, mã nguồn phía client gọi API bằng Axios, xử lý dữ liệu JSON, và hiển thị lên UI. Phía server dùng Node.js để nhận request, xử lý nghiệp vụ và trả về JSON.

- Bất đồng bộ: dùng Promise và async/await để gọi API, truy vấn DB, và đọc/ghi file mà không chặn luồng.
- Mô-đun hóa: tách mã thành nhiều mô-đun nhỏ, dễ tái sử dụng và kiểm thử.
- TypeScript (nếu dùng): thêm kiểu tĩnh cho biến, hàm, và phản hồi API nhằm giảm lỗi runtime.

2.5.1.2 NodeJS và Express 5

+ **Node.js:** Môi trường thực thi mã JavaScript phía máy chủ dựa trên V8 Engine của Google Chrome, nổi bật với mô hình bất đồng bộ và xử lý nhập/xuất không khóa (*event-driven, non-blocking I/O*). Điều này giúp hệ thống phản hồi tối ưu với các ứng dụng web chịu tải lớn và thực hiện nhiều thao tác I/O đồng thời (như truy xuất cơ sở dữ liệu hay gọi API bên thứ ba).

+ **Express 5:** Framework web tối giản, linh hoạt dành cho Node.js, chịu trách nhiệm quản lý:

- **Routing:** Định nghĩa và phân phối các điểm cuối HTTP.

- **Middleware:** Xử lý trung gian trước và sau khi hoàn tất yêu cầu (xác thực, ghi nhật ký, phân tích dữ liệu, kiểm soát CORS).
- **Error Handling:** Cơ chế tập trung bắt và xử lý ngoại lệ. – *Ưu điểm phiên bản 5:* Cải tiến vượt trội cơ chế xử lý lỗi hứa hẹn (*promise rejection*) và chuẩn hóa hành vi định tuyến so với thế hệ trước.

2.5.2 Prisma ORM

- **Prisma** là công cụ ánh xạ quan hệ đối tượng (*Object-Relational Mapping - ORM*) thế hệ mới dành cho hệ sinh thái Node.js/TypeScript, sở hữu nhiều điểm khác biệt cốt lõi:

- **Schema-first:** Mô hình dữ liệu được khai báo trực tiếp tại tệp `schema.prisma`, đóng vai trò là nguồn chân lý duy nhất (*single source of truth*) cho toàn bộ cấu trúc cơ sở dữ liệu.
- **Type Safety:** Tự động sinh mã Prisma Client với kiểu dữ liệu TypeScript đầy đủ, giúp giảm thiểu tối đa các lỗi phát sinh khi thực thi (*runtime errors*).
- **Migration Management:** Hỗ trợ sinh và kiểm soát các bước dịch chuyển cơ sở dữ liệu (*migrations*) trực quan.
- **Transaction Support:** Cung cấp cơ chế giao dịch nguyên tử (*transactions*) an toàn cho các chuỗi thao tác phức tạp.
- **Query API linh hoạt:** Hỗ trợ các truy vấn lồng nhau (*nested includes*), bộ lọc và phân trang mạnh mẽ.
- *Ứng dụng trong SportNexus:* Toàn bộ **22 bảng dữ liệu** được khai báo tại `server/prisma/schema.prisma` và mọi thao tác truy xuất dữ liệu trong tầng service đều sử dụng Prisma Client.

2.5.3 MySQL

- **MySQL** là hệ quản trị cơ sở dữ liệu quan hệ (*RDBMS*) mã nguồn mở phổ biến hàng đầu thế giới, được tin dùng rộng rãi nhờ vào tính ổn định, hiệu năng cao và khả năng vận hành bền bỉ trong các hệ thống doanh nghiệp: **Độ tin cậy cao:** Đã được kiểm chứng thực tế qua nhiều hệ thống quy mô lớn.

- **Hỗ trợ chuẩn SQL và tuân thủ ACID:** Đảm bảo tính toàn vẹn dữ liệu tuyệt đối theo tiêu chuẩn ACID (*Atomicity, Consistency, Isolation, Durability*), hỗ trợ mạnh mẽ các ràng buộc khóa ngoại, giao dịch phức tạp và các câu lệnh truy vấn kết hợp (*JOIN*) hiệu suất cao.

- **Độ tin cậy cao:** Đã được kiểm chứng thực tế qua nhiều hệ thống quy mô lớn, đảm bảo khả năng hoạt động liên tục, an toàn và chịu tải tốt trong các mô hình ứng dụng thương mại điện tử.
- **Hỗ trợ kiểu dữ liệu JSON:** Cung cấp khả năng lưu trữ và truy vấn linh hoạt các dữ liệu dạng bán cấu trúc (*semi-structured data*), rất thuận lợi cho việc lưu trữ lịch sử hoạt động, cấu hình động hoặc thông tin chi tiết nhật ký hệ thống.
- **Cộng đồng lớn và sinh thái phong phú:** Sở hữu nguồn tài liệu phong phú, các công cụ quản trị trực quan đa dạng và cộng đồng hỗ trợ kỹ thuật rộng lớn, giúp dễ dàng khắc phục sự cố và tối ưu hóa hệ thống.
- *Ứng dụng trong SportNexus:* Trong hệ thống SportNexus, MySQL chịu trách nhiệm lưu trữ toàn bộ dữ liệu nghiệp vụ cốt lõi (*từ thông tin người dùng, sản phẩm đa biến thể, đơn hàng, hóa đơn cho đến hệ thống phân quyền*), toàn bộ quá trình tương tác và truy vấn được thực hiện thông qua công cụ Prisma ORM một cách an toàn và tối ưu.

2.5.4 JWT (JSON Web Token)

Trong các kiến trúc ứng dụng web hiện đại phân rã thành **Client (React SPA)** và **Server (Express REST API)**, việc sử dụng cơ chế xác thực phi trạng thái (*Stateless Authentication*) là yêu cầu tất yếu để đảm bảo khả năng mở rộng hệ thống. **JSON Web Token (JWT)** theo tiêu chuẩn quốc tế **RFC 7519** đã được lựa chọn làm giải pháp cốt lõi cho hệ thống SportNexus.

- **JWT** là tiêu chuẩn mở quốc tế (RFC 7519) dùng để trao đổi thông tin một cách an toàn và nhỏ gọn giữa các bên dưới dạng các gói token mã hóa. Cấu trúc của một JWT bao gồm ba phần chính được phân tách bằng dấu chấm:**Header:** Khai báo thuật toán mã hóa chữ ký (ví dụ: HS256).

- **Header:** Khai báo loại token và thuật toán mã hóa chữ ký được sử dụng (ví dụ: HS256 hoặc RS256).
- **Payload:** Chứa thông tin tải trọng dữ liệu định danh người dùng (ví dụ: định danh id, vai trò role, và thư điện tử email).
- **Signature:** Chữ ký số được tạo bằng cách kết hợp Header, Payload và một khóa bí mật (*secret key*), đảm bảo tính toàn vẹn dữ liệu và xác thực chính xác nguồn gốc token.
- Chiến lược xác thực hai mã thông báo (*Two-Token Strategy*) trong SportNexus Để tối ưu hóa giữa tính bảo mật tài nguyên và trải nghiệm mượt mà cho người dùng,

hệ thống SportNexus áp dụng chiến lược xác thực kép (*Two-Token Strategy*)
phân rã thành hai loại token với thời hạn và mục đích sử dụng khác nhau:

Loại Token	Thời hạn hiệu lực	Mục đích sử dụng chính
Access Token	15 phút	Xác thực định danh người dùng trong mỗi lượt gọi API, đảm bảo tính bảo mật ngắn hạn.
Refresh Token	7 ngày	Cấp lại Access Token mới khi mã cũ hết hạn mà không bắt buộc người dùng phải đăng nhập lại từ đầu.

Bảng 2. 6. Phân loại và thời hạn của chiến lược Token trong SportNexus

Để giải quyết bài toán cân bằng giữa **tính bảo mật tối đa** (*giảm thiểu rủi ro khi token bị đánh cắp*) và **trải nghiệm người dùng mượt mà** (*không phải đăng nhập liên tục*), hệ thống SportNexus áp dụng mô hình xác thực kép (*Two-Token Strategy*), chia tách thành hai loại token với vòng đời và nhiệm vụ riêng biệt.

Encoded

PASTE A TOKEN HERE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

HMACSHA256(
 base64UrlEncode(header) + "." +
 base64UrlEncode(payload),
 your-256-bit-secret
) ☐ secret base64 encoded

Signature Verified

Hình 2. 3 Kết quả của giải mã một JWT ví dụ

- *Đánh giá chiến lược*: Mô hình này tạo ra sự cân bằng hoàn hảo giữa tính bảo mật (*Access Token ngắn hạn giúp giảm thiểu rủi ro khi bị lộ*) và trải nghiệm người dùng (*khởi phải thao tác đăng nhập liên tục*).

2.5.5 Các thư viện hỗ trợ Backend khác

Thư viện hỗ trợ	Mục đích sử dụng chính trong hệ thống Backend
joi	Định nghĩa lược đồ và kiểm tra tính hợp lệ dữ liệu đầu vào của yêu cầu.
bcrypt	Mã hóa mật khẩu người dùng theo cơ chế băm bảo mật cao.
jsonwebtoken	Tạo lập và xác thực chữ ký JWT.
multer + sharp	Tiếp nhận tải lên tệp tin và xử lý, nén, chỉnh sửa kích thước ảnh tự động.
@supabase/supabase-js	Tích hợp dịch vụ lưu trữ tệp tin đám mây.
nodemailer + ejs	Tạo mẫu và gửi email tự động (xác thực tài khoản, thông báo đơn hàng).
exceljs	Hỗ trợ tính năng xuất và nhập dữ liệu dưới định dạng bảng tính Excel.
@payos/node	Tích hợp cổng thanh toán điện tử PayOS.
google-auth-library	Xây dựng tính năng xác thực đăng nhập một chạm qua tài khoản Google.
slugify	Chuyển đổi chuỗi tiếng Việt có dấu thành chuỗi định danh dạng slug thân thiện với URL.
adm-zip	Xử lý các tác vụ nén và giải nén tệp tin.

Bảng 2. 7. Tổng hợp các thư viện hỗ trợ phía Backend

2.5.6 Các phần mềm hỗ trợ

Quá trình phát triển dự án sử dụng kết hợp nhiều công cụ hỗ trợ thuộc các nhóm khác nhau:

Postman là ứng dụng dùng để kiểm thử API REST. Với giao diện trực quan, developer có thể gửi request đến các endpoint của server (*GET, POST, PUT, DELETE*), kiểm tra phản hồi (*status code, JSON body*), lưu lại các bộ request dưới dạng collection để

chạy lại khi cần. Trong dự án này, Postman được dùng chủ yếu để kiểm tra từng API của backend trước khi tích hợp với frontend, giúp phát hiện lỗi nghiệp vụ hoặc sai cấu trúc dữ liệu ở giai đoạn sớm mà không cần chờ giao diện hoàn chỉnh.

Laragon là môi trường phát triển cục bộ (local development environment) trên Windows, tích hợp sẵn máy chủ web (*Apache hoặc Nginx*) và hệ quản trị cơ sở dữ liệu MySQL. Laragon giúp khởi chạy MySQL nhanh chóng qua vài cú click, đồng thời hỗ trợ quản lý nhiều project riêng biệt qua cơ chế Virtual Host. Trong dự án, Laragon đóng vai trò cung cấp MySQL cục bộ để Prisma có thể chạy migration và truy xuất dữ liệu trong quá trình phát triển, thay vì phải cài đặt từng phần mềm riêng lẻ.

Visual Studio Code (VSCode) là trình soạn thảo mã nguồn được sử dụng chính trong dự án. VSCode hỗ trợ nhiều tính năng phục vụ phát triển JavaScript/Node.js và React: gợi ý code thông minh (*IntelliSense*), tích hợp terminal, hỗ trợ extension như Prettier (*tự động định dạng code*), ESLint (kiểm tra lỗi code theo quy tắc), và Prisma (*đọc file schema với màu sắc cú pháp*). Giao diện phân chia panel đa cửa sổ giúp developer vừa xem code vừa chạy lệnh terminal mà không cần chuyển ứng dụng.

draw.io (*hiện có tên là diagrams.net*) là công cụ vẽ sơ đồ miễn phí hoạt động trực tiếp trên trình duyệt, hỗ trợ nhiều loại sơ đồ: use case, ERD, sequence, flowchart, architecture... Sơ đồ được lưu dưới dạng tệp SVG hoặc PNG có thể nhúng trực tiếp vào tài liệu Word/Markdown. Trong đề tài, draw.io được dùng để vẽ toàn bộ sơ đồ trong báo cáo (*sơ đồ use case, sơ đồ ERD, lưu đồ luồng xử lý...*) với giao diện kéo thả dễ chỉnh sửa.

Looping là phần mềm miễn phí do Đại học Toulouse (*Pháp*) phát triển, dùng chuyên biệt cho mô hình hóa dữ liệu theo phương pháp Merise. Looping hỗ trợ vẽ sơ đồ **MCD** (*Modèle Conceptuel de Données*) với entité và association kèm hệ số lớp, sau đó tự chuyển đổi sang **MLD** (*Modèle Logique de Données*) rồi **MPD** (*Modèle Physique Code SQL*) — rút ngắn đáng kể quy trình thiết kế cơ sở dữ liệu so với khi làm thủ công. Ngoài ra, Looping còn vẽ được sơ đồ UML class diagram và sơ đồ tổ chức luồng (*MOF*), xuất ra hình PNG để nhúng vào tài liệu. Trong dự án này, Looping được dùng để vẽ MCD khái niệm cho toàn bộ 31 thực thể trước khi triển khai trên Prisma, giúp hình dung rõ quan hệ một–nhiều, nhiều–nhiều và ràng buộc duy nhất trước khi code.

Nodemon là tiện ích chạy song song với Node.js server, tự động khởi động lại server mỗi khi phát hiện thay đổi trong tệp mã nguồn. Cơ chế hoạt động là "quét liên tục" (*looping*) thư mục dự án, khi nhận thấy file `.js` được lưu thì ngay lập tức dừng tiến trình cũ và chạy lại `node index.js` — giúp developer không cần tắt – mở server thủ công mỗi lần sửa code.

Trong dự án, nodemon được cấu hình trong file `package.json` với script `npm run dev` ở cả phía server lẫn client.

Trình duyệt web (*Chrome, Firefox...*) là công cụ cuối cùng để chạy và kiểm tra giao diện người dùng. Trình duyệt cung cấp bộ Developer Tools tích hợp sẵn (F12) với nhiều tính năng: Console debug, Network tab theo dõi request/response, LocalStorage/SessionStorage xem dữ liệu client, và Lighthouse audit đánh giá hiệu năng. Trong quá trình phát triển, trình duyệt đóng vai trò "người dùng cuối" — mọi giao diện từ trang chủ, giỏ hàng đến trang quản trị đều được kiểm tra trực tiếp trên trình duyệt để đảm bảo hiển thị đúng trên cả máy tính lẫn điện thoại.

Git & GitHub: Hệ thống quản lý phiên bản mã nguồn phân tán (*Version Control System*) và nền tảng lưu trữ kho lưu trữ trực tuyến hàng đầu. Git giúp ghi lại lịch sử thay đổi của toàn bộ mã nguồn, hỗ trợ cơ chế phân nhánh (*branching*) linh hoạt để phát triển các tính năng mới hoặc sửa lỗi mà không làm ảnh hưởng đến nhánh vận hành chính (main). GitHub đóng vai trò là trung tâm làm việc nhóm, hỗ trợ quản lý dự án, kiểm tra mã nguồn (*Pull Request / Code Review*) và phối hợp đồng bộ giữa các thành viên. Trong dự án SportNexus, Git và GitHub quản lý toàn bộ mã nguồn của cả tầng Frontend và Backend, đồng thời là nền tảng để tích hợp các công cụ tự động hóa (*CI/CD*)

Ngoài ra, dự án còn sử dụng **npm** (*Node Package Manager*) để quản lý thư viện phía client và server riêng biệt, **Concurrently** để chạy đồng thời cả hai tiến trình dev trong một terminal duy nhất, và ***Tailwind CS** kèm plugin của nó để xây dựng giao diện-responsive bằng utility class mà không cần viết CSS thủ công.

2.5.7 Lợi ích thực tiễn

- Việc tích hợp sẵn tính năng đa ngôn ngữ mang lại nhiều giá trị chiến lược cho dự án:

- **Mở rộng thị trường:** Giúp hệ thống không chỉ phục vụ tốt khách hàng nội địa mà còn sẵn sàng tiếp cận với phân khúc khách hàng quốc tế.
- **Tính nhất quán cao:** Duy trì một ngôn ngữ gốc chuẩn mực (Tiếng Việt) và quản lý tập trung việc biên dịch sang tiếng Anh thông qua hệ thống tệp JSON.
- **Trải nghiệm người dùng tối ưu:** Mang lại sự thoải mái và gần gũi khi người dùng được thao tác hoàn toàn bằng ngôn ngữ quen thuộc của họ.

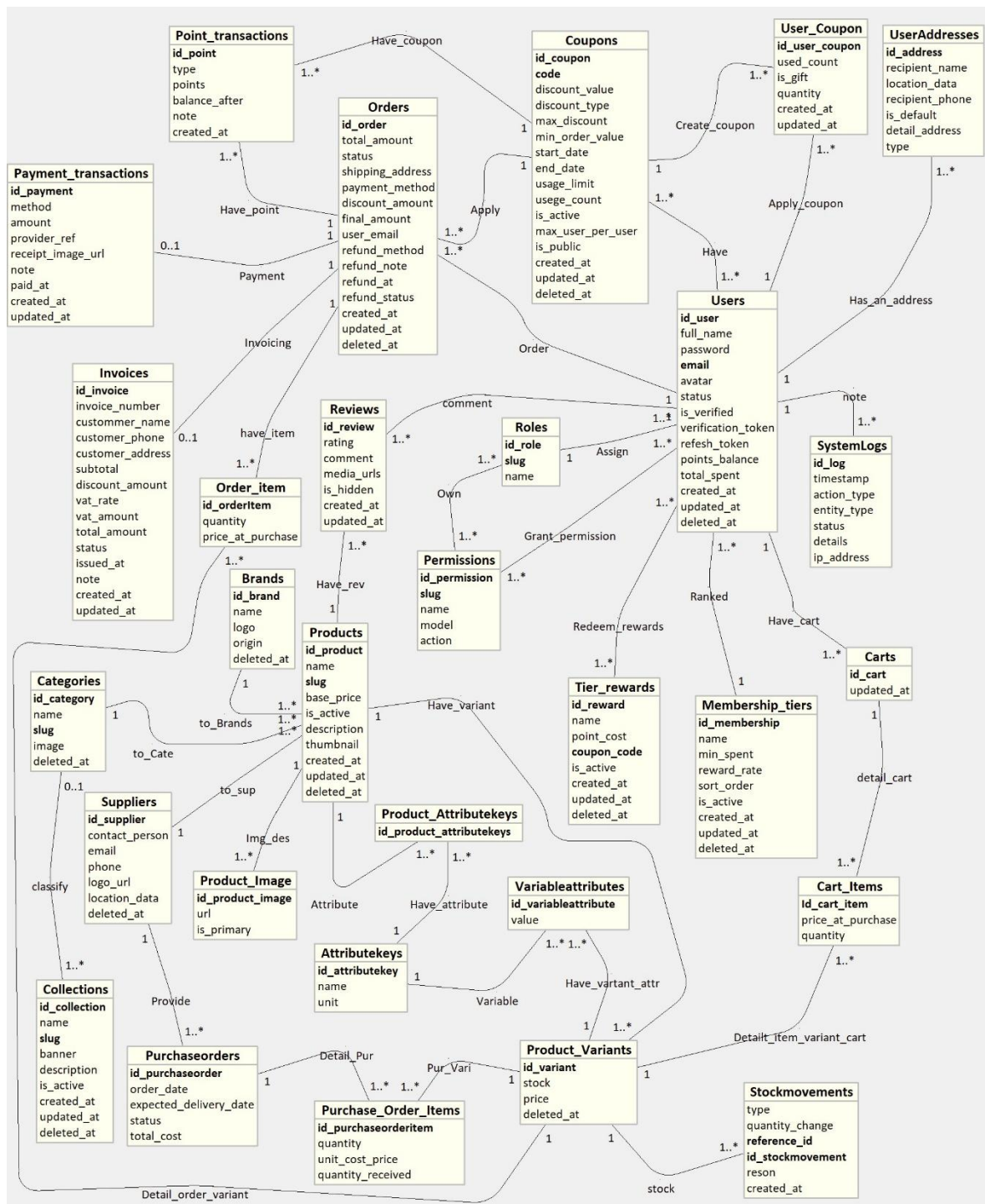
CHƯƠNG III. KẾT QUẢ THỰC HIỆN



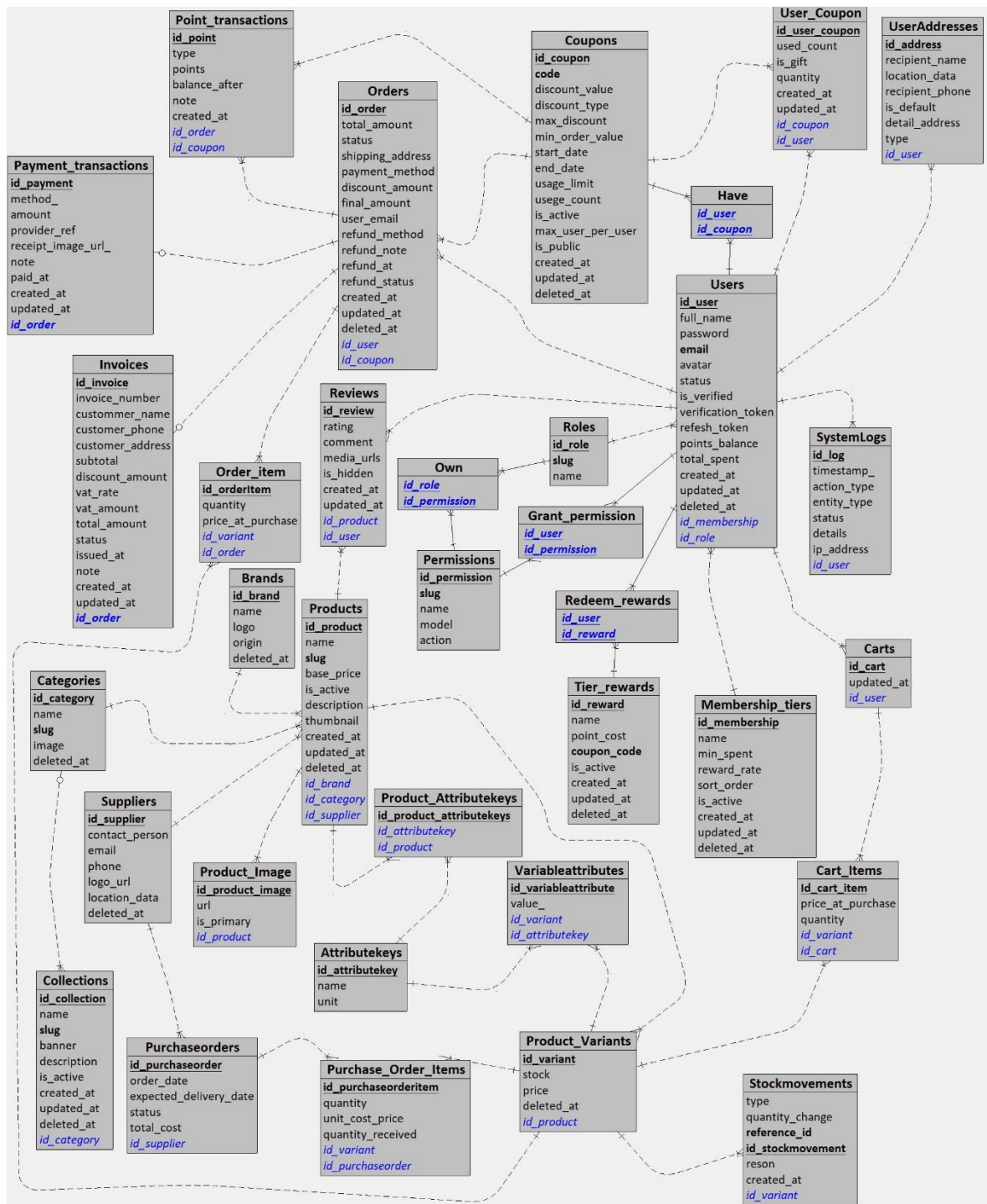
3.1 Cơ sở dữ liệu



Hình 3. 1 Mô hình MCD của hệ thống



Hình 3. 2 Mô hình MLD của hệ thống



Hình 3.3 Mô hình MPD của hệ thống

Mối quan hệ giữa sản phẩm và các danh mục phân loại trong hệ thống được thiết kế theo kiểu "một-nhiều". Cụ thể, mỗi sản phẩm chỉ thuộc về một danh mục (Categories), một thương hiệu (Brands) và một nhà cung cấp (Suppliers) duy nhất, được xác định thông qua các khóa ngoại category_id, brand_id và supplier_id trong thực thể Products. Ngược lại, mỗi danh mục, thương hiệu hay nhà cung cấp có thể sở hữu nhiều sản phẩm khác nhau theo quan hệ "1,n". Điểm đặc biệt của hệ thống nằm ở thực thể ProductVariants: một sản phẩm cha có thể có nhiều biến thể, trong đó mỗi biến thể mang riêng số lượng tồn kho (stock) và giá bán (price) — đây chính là cơ chế cho phép quản lý chi tiết từng tổ hợp màu sắc, kích cỡ của

cùng một mẫu giày hay trang phục thể thao, điều mà mô hình "một sản phẩm một giá" thông thường không đáp ứng được.

Để mô tả biến thể một cách linh hoạt, hệ thống xây dựng bộ ba thực thể AttributeKeys – VariableAttributes – ProductAttributeKeys. AttributeKeys định nghĩa các loại thuộc tính (ví dụ "Màu sắc", "Kích cỡ") còn VariableAttributes gán giá trị cụ thể cho từng biến thể, kèm ràng buộc duy nhất trên cặp (variable_id, attribute_key_id) để một biến thể không bị khai báo trùng hai lần cùng một thuộc tính. Nhờ cấu trúc này, phía website có thể tính toán tập giá trị khả dụng dựa trên tồn kho thực tế và tự động khớp biến thể khi khách chọn đầy đủ các thuộc tính, đồng thời khóa chính id của từng biến thể trở thành mốc tham chiếu chung cho toàn bộ nghiệp vụ mua bán.

Quy trình mua sắm được phản ánh qua chuỗi thực thể Carts – CartItems – Orders – OrderItems. Mỗi người dùng đã đăng nhập sở hữu một giỏ hàng (Carts) chứa nhiều dòng hàng (CartItems); ràng buộc duy nhất trên cặp (cart_id, product_variant_id) đảm bảo cùng một biến thể không xuất hiện thành hai dòng riêng biệt. Khi khách tiến hành đặt hàng, hệ thống tạo bản ghi Orders kèm các dòng OrderItems tham chiếu đến biến thể tương ứng với giá chốt tại thời điểm mua (price_at_purchase). Đơn hàng không bắt buộc phải có tài khoản (usersId cho phép rỗng) nhằm hỗ trợ khách vắng lai, nhưng vẫn ghi lại user_email để liên lạc. Mỗi đơn luôn đi kèm một hóa đơn Invoices theo quan hệ "1-1" (order_id là duy nhất) chứa đầy đủ thông tin thuế VAT và tổng tiền, cùng bảng PaymentTransactions ghi nhận mọi lần thanh toán — bao gồm cả những lần thất bại — kèm mã giao dịch từ cổng PayOS hay ảnh chứng minh thư chuyển khoản.

Tính toán vận của nghiệp vụ kho được đảm bảo bằng thực thể StockMovements: mỗi lần đặt hàng thành công, hệ thống vừa giảm stock của các biến thể liên quan, vừa ghi một bản ghi biến động với type = "OUT" và quantity_change âm; ngược lại khi nhận hàng nhập từ nhà cung cấp qua chuỗi PurchaseOrders – PurchaseOrderItems (có theo dõi quantity_received để chấp nhận nhận từng phần), kho được tăng lên bằng bản ghi type = "IN". Bên cạnh đó, chức năng đánh giá Reviews được gắn đồng thời với người dùng, đơn hàng và sản phẩm, kết hợp trạng thái của đơn hàng để chỉ cho phép đánh giá sau khi hàng đã giao thành công — tạo nên độ tin cậy cho phản hồi trên website. Các mã giảm giá Coupons được chia sẻ cho nhiều người dùng thông qua bảng trung gian UserCoupons (lưu số lượt đã dùng và cờ is_gift cho mã được tặng), trong khi chương trình khách hàng thân thiết gồm MembershipTiers xếp hạng theo tổng chi tiêu và PointTransactions ghi nhận ký cộng/trừ điểm kèm số dư sau mỗi giao dịch. Cuối cùng, SystemLogs ghi nhận toàn bộ thao tác quan trọng (action_type, entity_type, status, ip_address) giúp quản trị viên truy vết lịch

sử hệ thống minh bạch và đáng tin cậy.

Dưới đây là chi tiết một số thực thể trọng yếu gắn liền với các chức năng chính của hệ thống.

3.2 Nhóm tài khoản và địa chỉ

3.2.1 users (Người dùng)

Thực thể trung tâm của hệ thống, lưu trữ thông tin tài khoản của cả khách hàng lẫn quản trị viên, phục vụ các nghiệp vụ đăng ký – đăng nhập, phân quyền và chương trình khách hàng thân thiết.

Thuộc tính	Kiểu dữ liệu	Null	Dữ liệu mặc định	Khóa chính
id	int	none	auto_increment	true
full_name	varchar(191)	none	-	-
email	varchar(191)	none	unique theo deleted_at	-
password	varchar(191)	none	bcrypt	-
phone_number	varchar(191)	none	unique theo deleted_at	-
avatar	text	true	-	-
status	tinyint(1)	none	true	-
is_verified	tinyint(1)	none	false	-
verification_token	varchar(191)	true	-	-
refresh_token	varchar(191)	true	-	-
role_id	int	none	FK → roles.id	-
points_balance	int	none	0	-
total_spent	decimal(10,2)	none	0	-
tier_id	int	true	FK → membership_tiers.id	-
created_at	datetime(3)	none	now()	-

Thuộc tính	Kiểu dữ liệu	Null	Dữ liệu mặc định	Khóa chính
updated_at	datetime(3)	none	tự cập nhật	-
deleted_at	datetime(3)	none	'1000-01-01'	-

Bảng 3. 1 Chi tiết thực thể users

Trong đó có:

- id: Mã định danh duy nhất cho mỗi người dùng.
- full_name: Họ tên đầy đủ của người dùng.
- email: Địa chỉ email dùng đăng nhập và liên hệ.
- password: Mật khẩu đã được mã hóa bằng bcrypt.
- phone_number: Số điện thoại liên lạc.
- avatar: Đường dẫn ảnh đại diện lưu trên Supabase Storage.
- status: Trạng thái hoạt động (false nghĩa là tài khoản bị khóa, không được đăng nhập).
- is_verified: Cờ đã xác minh email hay chưa.
- verification_token: Token dùng cho link xác minh email / đặt lại mật khẩu.
- refresh_token: Refresh token hiện hành của phiên đăng nhập.
- role_id: Khóa ngoại tới vai trò, quyết định quyền hạn trong hệ thống.
- points_balance, total_spent, tier_id: Điểm thưởng, tổng chi tiêu và hạng thành viên phục vụ chương trình khách hàng thân thiết.

3.2.2 useraddresses (Sổ địa chỉ)

Lưu nhiều địa chỉ giao hàng của một người dùng, cho phép chọn nhanh khi checkout thay vì nhập lại mỗi lần.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
recipient_name	varchar(191)	none	-	-
recipient_phone	varchar(191)	none	-	-
location_data	json	none	-	-
detail_address	varchar(191)	none	-	-

is_default	tinyint(1)	none	false	-
type	varchar(191)	none	home	-
user_id	int	none	FK → users.id	-

Bảng 3. 2 Chi tiết thực thể useraddresses

Trong đó:

- recipient_name / recipient_phone: Thông tin người nhận tại địa chỉ này, có thể khác chủ tài khoản (*đặt hàng tặng*).
- location_data: Đối tượng JSON chứa mã tỉnh, huyện, xã — phục vụ tính phí vận chuyển theo bảng giá theo vùng.
- detail_address: Địa chỉ chi tiết (*số nhà, đường*), bổ sung cho location_data để hình thành địa chỉ hoàn chỉnh.
- is_default: Đánh dấu địa chỉ được tự động chọn khi mở trang thanh toán, tránh thao tác chọn lại mỗi lần.
- type: Phân loại mục đích sử dụng (*home, office, company*), giúp người dùng nhận biết nhanh khi chọn.

3.3 Nhóm phân quyền

3.3.1 permissions (Quyền hạn)

Mỗi bản ghi đại diện cho đúng một quyền cụ thể (ví dụ *"products:read"*, *"orders:update"*), được tổ chức theo mô-đun và hành động, phục vụ mô hình phân quyền RBAC ở cấp độ fine-grained.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	KHOÁ CHÍNH
id	int	none	auto_increment	true
slug	varchar(191)	none	unique	-
name	varchar(191)	none	-	-
module	varchar(191)	none	-	-
action	varchar(191)	none	-	-

Bảng 3. 3 Chi tiết thực thể permissions

Trong đó:

- id: Mã định danh duy nhất, được dùng làm khóa ngoại từ bảng liên kết người dùng – vai trò.
- name: Tên hiển thị thân thiện trên giao diện quản trị (ví dụ "Thêm sản phẩm"), giúp quản trị viên hiểu quyền mà không cần nhớ slug.
- module: Nhóm chức năng mà quyền thuộc về (Products, Orders, Users...), dùng để hiển thị ma trận quyền theo cột trên giao diện.
- action: Hành động cụ thể (read, create, update, delete), cho phép tách biệt quyền xem và quyền sửa trong cùng một module.

3.3.2 roles (Vai trò)

Gom nhiều quyền thành một nhóm có ý nghĩa nghiệp vụ (ví dụ "*Quản trị viên*", "*Nhân viên kho*"), giúp gán quyền hàng loạt thay vì chọn từng cái cho mỗi người dùng.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	KHOÁ CHÍNH
id	int	none	auto_increment	true
slug	varchar(191)	none	unique	-
name	varchar(191)	none	-	-

Bảng 3. 4 Chi tiết thực thể roles

Trong đó:

- id: Mã vai trò, được tham chiếu bởi `users.role\_id`.
- name: Tên hiển thị trên giao diện phân quyền.

3.4 Nhóm phân loại sản phẩm

3.4.1 categories (Danh mục)

Nhóm hàng hóa theo loại lớn (*giày, quần áo, phụ kiện...*), là bộ lọc cơ bản nhất cho khách hàng trên website và là khóa ngoại bắt buộc khi tạo sản phẩm.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	KHOÁ CHÍNH
id	int	none	auto_increment	true
name	varchar(191)	none	-	-
slug	varchar(191)	none	unique theo deleted_at	-

image	text	true	-	-
is_active	tinyint(1)	none	true	-
deleted_at	datetime(3)	none	'1000-01-01'	-

Bảng 3. 5 Chi tiết thực thể categories

Trong đó:

- slug: Mã hóa đường dẫn URL cho trang danh mục (ví dụ `'/danh-muc/giay'`), hỗ trợ SEO và bookmark.
- image: Ảnh minh họa danh mục dùng làm banner danh mục trên trang chủ.
- is_active: Cho phép ẩn một danh mục mà không cần xóa, useful khi ngừng phân loại tạm thời.
- deleted_at: Xóa mềm — sản phẩm vẫn giữ `'category_id'` tham chiếu mà không phá vỡ ràng buộc.

3.4.2 collections (Bộ sưu tập)

Nhóm sản phẩm theo chủ đề marketing (ví dụ *"Bộ sưu tập mùa hè 2025"*), có banner riêng và gắn với một danh mục cha, khác với categories ở chỗ集合 chủ động và thời gian, phục vụ chiến dịch truyền thông.

THUỘC TÍNH	Kiểu dữ liệu	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
name	varchar(191)	none	-	-
slug	varchar(191)	none	unique theo deleted_at	-
banner	text	true	-	-
description	text	true	-	-
is_active	tinyint(1)	none	true	-
deleted_at	datetime(3)	none	'1000-01-01'	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-

category_id	int	none	FK → categories.id	-
-------------	-----	------	--------------------	---

Bảng 3. 6 Chi tiết thực thể collections

Trong đó:

- banner: Ảnh bìa bộ sưu tập, hiển thị riêng trên trang bộ sưu tập khách hàng.
- description: Mô tả chủ đề, dùng render phần giới thiệu đầu trang bộ sưu tập.
- category_id: Bộ sưu tập chỉ kéo dài trong một danh mục — giúp hệ thống tự động lọc sản phẩm thuộc cả danh mục cha và bộ sưu tập con.

3.4.3 brands (Thương hiệu)

Lưu thông tin thương hiệu thể thao (*Nike, Adidas...*), tham chiếu bởi `products.brand\_id`, phục vụ lọc sản phẩm và hiển thị logo trên trang danh sách.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	KHOÁ CHÍNH
id	int	none	auto_increment	true
name	varchar(191)	none	-	-
logo	varchar(191)	true	-	-
origin	text	true	-	-
deleted_at	datetime(3)	none	'1000-01-01'	-

Bảng 3. 7 Chi tiết thực thể brands

Trong đó:

- logo: Đường dẫn ảnh logo thương hiệu, hiển thị trên web và trang quản trị.
- origin: Xuất xứ thương hiệu, có thể dùng làm thông tin bổ sung cho khách quan tâm nguồn gốc.

3.4.4 suppliers (Nhà cung cấp)

Lưu thông tin đối tác nhập hàng, bắt buộc khi tạo sản phẩm, phục vụ both đơn nhập hàng và báo cáo nguồn hàng.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	KHOÁ CHÍNH
id	int	none	auto_increment	true
name	varchar(191)	none	-	-

logo	varchar(191)	true	-	-
origin	text	true	-	-
deleted_at	datetime(3)	none	'1000-01-01'	-

Bảng 3. 8 Chi tiết thực thể suppliers

Trong đó:

- contact_person: Tên người liên hệ phụ trách đơn hàng tại nhà cung cấp.
- location_data: JSON chứa thông tin tỉnh/huyện, dùng khi cần hiển thị hoặc lọc nhà cung cấp theo vùng.
- logo: Ảnh logo dùng trên danh sách quản trị.

3.5 Nhóm hình ảnh và thuộc tính sản phẩm

3.5.1 productimages (Hình ảnh sản phẩm)

Mỗi sản phẩm có thể có nhiều ảnh; ảnh chính (*is_primary*) dùng làm thumbnail khi hiển thị danh sách, các ảnh phụ hiển thị phần gallery trong trang chi tiết.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
url	text	none	-	-
is_primary	tinyint(1)	none	false	-
product_id	int	none	FK → products.id	-

Bảng 3. 9 Chi tiết thực thể productimages

Trong đó:

- url: Đường dẫn ảnh trên Supabase Storage, được render trực tiếp trong thẻ `<img>`.
- is_primary: Cờ đánh dấu ảnh chính, chỉ được một ảnh mỗi sản phẩm; dùng khi hệ thống cần trả về một ảnh duy nhất (trang chủ, danh sách, hóa đơn).
- product_id: Khóa ngoại gắn ảnh với sản phẩm cha.

3.5.2 attributekeys (Khóa thuộc tính)

Định nghĩa tên loại thuộc tính dùng chung cho toàn hệ thống (kích cỡ, màu sắc, chất liệu...), đóng vai trò làm "metadata" cho hệ thống lọc phía khách hàng.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
name	varchar(191)	none	unique	-
unit	varchar(50)	true	-	-

Bảng 3. 10 Chi tiết thực thể attributekeys

Trong đó:

- name: Tên duy nhất hiển thị trên giao diện lọc (ví dụ "Kích cỡ", "Màu sắc") và trên form thêm biến thể.
- unit: Đơn vị đo (cm, mm...) dùng cho thuộc tính số — hiện tại hệ thống chủ yếu dùng chuỗi, nhưng giữ trường này để mở rộng sau.

3.5.3 variableattributes (Giá trị thuộc tính biến thể)

Gán giá trị cụ thể cho từng biến thể theo từng khóa thuộc tính, ràng buộc `UNIQUE(variable_id, attribute_key_id)` đảm bảo một biến thể không bị khai báo trùng hai lần cùng loại thuộc tính.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
variable_id	int	none	FK → productvariants.id	-
attribute_key_id	int	none	FK → attributekeys.id	-
value	varchar(191)	none	-	-

Bảng 3. 11 Chi tiết thực thể variableattributes

Trong đó:

- variable_id / attribute_key_id: Cặp khóa ngoại xác định biến thể nào mang thuộc tính nào — chính cặp này bị ràng buộc UNIQUENESS.
- value: Giá trị cụ thể (ví dụ "42" cho kích cỡ, "Đen" cho màu sắc), được hiển thị trên bộ lọc và trong phần chọn biến thể.

3.5.4 productattributekeys (Thuộc tính hiển thị của sản phẩm)

Liệt kê những loại thuộc tính mà một sản phẩm sử dụng (chỉ danh mục, không chứa giá trị), phục vụ giao diện quản trị hiển thị đúng form nhập biến thể cho từng loại sản phẩm.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
product_id	int	none	FK → products.id	-
attribute_key_id	int	none	FK → attributekeys.id	-

Bảng 3. 12 Chi tiết thực thể productattributekeys

Trong đó:

- product_id / attribute_key_id: Ràng buộc UNIQUENESS — một sản phẩm chỉ liệt kê mỗi khóa thuộc tính một lần, tránh nhập trùng khi tạo biến thể.
- Bảng này không chứa dữ liệu giá trị mà chỉ đóng vai "danh sách cho phép": biến thể của sản phẩm chỉ được phép gán thuộc tính có trong danh sách này.

3.6 Nhóm mã giảm giá

3.6.1 coupons (Mã giảm giá)

Lưu thông tin mã giảm giá và toàn bộ điều kiện áp dụng, được cả khách hàng lẫn hệ thống kiểm tra khi đặt hàng.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
code	varchar(191)	none	unique	-
discount_value	int	none	-	-
discount_type	enum(CASH, PERCENTAGE)	none	CASH	-
max_discount	int	none	-	-
min_order_value	int	none	-	-

start_date	datetime(3)	none	-	-
end_date	datetime(3)	none	-	-
usage_limit	int	none	-	-
usage_count	int	none	0	-
is_active	tinyint(1)	none	true	-
is_public	tinyint(1)	none	true	-
max_uses_per_user	int	none	1	-
deleted_at	datetime(3)	none	'1000-01-01'	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-

Bảng 3. 13 Chi tiết thực thể coupons

Trong đó:

- code: Mã khách nhập khi thanh toán (ví dụ "SALE10"), unique toàn cục, được kiểm tra chính xác chữ hoa/thường.
- discount_value: Giá trị giảm (số tiền hoặc phần trăm tùy `discount\_type`).
- discount_type: Phân biệt giảm cố định (CASH – ví dụ giảm 50.000đ) hay giảm theo tỷ lệ (PERCENTAGE – giảm 10%).
- max_discount: Trần giảm tối đa khi dùng mã phần trăm (ví dụ giảm 10% nhưng tối đa 200.000đ).
- min_order_value: Giá trị đơn hàng tối thiểu để mã được chấp nhận.
- start_date / end_date: Khung thời gian mã có hiệu lực — mã hết hạn bị hệ thống tự động loại khỏi danh sách hiển thị.
- usage_limit: Tổng số lần mã có thể được sử dụng trên toàn hệ thống.
- usage_count: Số lần đã dùng — khi `usage\_count ≥ usage\_limit`, mã bị chặn.
- is_public: Mã hiển thị trang coupon công khai hay chỉ dùng nội bộ (tặng khách VIP).
- max_uses_per_user: Số lần tối đa một người dùng được dùng mã này, được kiểm tra qua bảng `user\_coupons`.

3.6.2 user_coupons (Mã giảm giá đã lưu):

Bảng trung gian giữa người dùng và mã giảm giá, ghi nhận mã nào đã được lưu và đã dùng bao nhiêu lần — cần thiết để áp dụng giới hạn `max\_uses\_per\_user`.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
user_id	int	none	FK → users.id	-
coupon_id	int	none	FK → coupons.id	-
used_count	int	none	0	-
is_gift	tinyint(1)	none	false	-
quantity	int	none	1	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-

Bảng 3. 14 Chi tiết thực thể user_coupons

Trong đó:

- used_count: Số lần người dùng này đã dùng mã — khi bằng `max\_uses\_per\_user` trong bảng coupons thì bị chặn dùng thêm.
- is_gift: Phân biệt mã tự lưu (false) và mã được hệ thống tặng (true), phục vụ hiển thị và quản lý ở phía khách hàng.
- quantity: Số lượng phiếu mã được cấp (hỗ trợ scenario tặng nhiều phiếu cùng lúc).

3.7 Nhóm đơn hàng và thanh toán

3.7.1 invoices (Hóa đơn)

Tạo tự động theo quan hệ 1–1 với đơn hàng, chứa đầy đủ thông tin thuế VAT phục vụ xuất hóa đơn cho khách hàng; trạng thái luôn đồng bộ với đơn hàng gốc.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true

invoice_number	varchar(191)	none	unique	-
order_id	int	none	unique, FK → orders.id	-
customer_name	varchar(191)	none	-	-
customer_email	varchar(191)	true	-	-
customer_phone	varchar(191)	true	-	-
shipping_address	varchar(191)	none	-	-
subtotal	decimal(10,2)	none	-	-
discount_amount	decimal(10,2)	none	-	-
vat_rate	decimal(5,2)	none	0.08	-
vat_amount	decimal(10,2)	none	-	-
total_amount	decimal(10,2)	none	-	-
status	enum(Pending, Completed, Cancelled)	none	Pending	-
issued_at	datetime(3)	none	now()	-
note	text	true	-	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-

Bảng 3. 15 Chi tiết thực thể invoices

Trong đó:

- invoice_number: Số hóa đơn duy nhất, hiển thị khi khách in hoặc xem chi tiết hóa đơn.
- order_id: Ràng buộc UNIQUENESS — mỗi đơn chỉ có đúng một hóa đơn, tránh tạo trùng.

- subtotal / discount_amount / vat_rate / vat_amount / total_amount: Phân tách rõ các thành phần tiền: hàng chưa giảm giá, giảm giá, tỷ lệ VAT 8%, số tiền VAT và tổng cộng — phục vụ khách in hóa đơn đúng chuẩn tài chính.
- status: Đồng bộ với orders — Completed khi đơn giao thành công và đã thanh toán; Cancelled khi đơn bị hủy.

3.7.2 payment_transactions (Lịch sử thanh toán)

Ghi nhận mọi lần giao dịch thanh toán liên quan đến một đơn — bao gồm cả thành công và thất bại — giúp truy vết nguồn tiền và đối chiếu khi có tranh chấp.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
order_id	int	none	FK → orders.id	-
method	enum(COD,BANK_TRANSFER,MOMO,VNPAY,CREDIT_CARD)	none	-	-
amount	decimal(10,2)	none	-	-
status	enum(Pending,Paid,Failed,Refunded)	none	Pending	-
provider_ref	varchar(191)	true	-	-
transaction_code	varchar(191)	true	-	-
receipt_image_url	text	true	-	-
note	text	true	-	-
paid_at	datetime(3)	true	-	-
created_at	datetime(3)	none	now()	-

Bảng 3. 16 Chi tiết thực thể payment_transactions

Trong đó:

- order_id: Gắn giao dịch với đơn hàng gốc.
- method: Phương thức thanh toán — mỗi lần trả tiền có thể khác nhau (lần 1 COD, lần 2 PayOS khi đổi mind).

- provider_ref/ transaction_code: Mã tham chiếu từ cổng thanh toán (PayOS) hoặc mã giao dịch Casso, dùng đối chiếu khi cần tra cứu.
- receipt_image_url: Ảnh chứng minh chuyển khoản thủ công (khách tải lên khi chọn BANK_TRANSFER).
- paid_at: Thời điểm xác nhận tiền thành công, dùng tính toán hạn thanh toán và báo cáo doanh thu theo ngày.

3.7.3 orders (Đơn hàng)

Thực thể ghi nhận kết quả của nghiệp vụ đặt hàng — luồng xử lý trọng yếu chạy trong transaction của hệ thống.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
total_amount	decimal(10,2)	none	-	-
status	enum(Processing, Shipping, Delivered, Cancelled, Refunded)	none	Processing	-
shipping_address	varchar(191)	none	-	-
payment_method	enum(COD, BANK_TRANSFER, MOMO, VNPAY, CREDIT_CARD)	none	COD	-
payment_status	enum(Pending, Paid, Failed, Refunded)	none	Pending	-
discount_amount	decimal(10,2)	none	-	-
final_amount	decimal(10,2)	none	-	-
coupon_code	varchar(191)	true	FK → coupons.code	-
user_email	varchar(191)	true	-	-
userId	int	true	FK → users.id	-

refund_status	varchar(191)	true	'pending'	-
refunded_at	datetime(3)	true	-	-
created_at	datetime(3)	none	now()	-

Bảng 3. 17 Chi tiết thực thể orders

Trong đó có:

- id: Mã định danh duy nhất của đơn hàng.
- total_amount: Tổng tiền hàng trước giảm giá.
- status: Trạng thái xử lý đơn từ lúc tiếp nhận đến giao thành công/hủy/hoàn.
- shipping_address: Địa chỉ giao hàng chốt tại thời điểm đặt.
- payment_method / payment_status: Phương thức và tiến độ thanh toán.
- discount_amount / final_amount: Số tiền giảm và số tiền khách phải trả.
- coupon_code: Mã giảm giá áp dụng (nếu có).
- user_email / userId: Thông tin khách; userId trống tương ứng khách vắng lai.
- refund_status / refunded_at: Theo dõi hoàn tiền khi đơn bị hủy sau thanh toán.

3.7.4 orderitems (Dòng chi tiết đơn hàng)

Lưu lại chính xác biến thể nào, số lượng bao nhiêu và giá bao nhiêu tại thời điểm mua — giá snapshot(*price_at_purchase*) không thay đổi khi biến thể cập nhật giá sau này.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	KHOÁ CHÍNH
id	int	none	auto_increment	true
quantity	int	none	-	-
price_at_purchase	decimal(10,2)	none	-	-
order_id	int	none	FK → orders.id	-
product_variant_id	int	none	FK → productvariants.id	-

Bảng 3. 18 Chi tiết thực thể orderitems

Trong đó:

- price_at_purchase: Giá bán snapshot tại thời điểm khách đặt — bảo đảm lịch sử đơn hàng không bị sai lệch khi 管理员 sau đó sửa giá biến thể.

- quantity: Số lượng biến thể này trong đơn, dùng cả khi tính tổng tiền lần khi hoàn tồn kho khi hủy đơn.
- order_id / product_variant_id: Khóa ngoại hai chiều — phục vụ cả khi tra cứu đơn lần khi tổng hợp báo cáo doanh số theo biến thể.

3.8 Nhóm đánh giá

3.8.1 reviews (Đánh giá sản phẩm)

Lưu đánh giá kèm ảnh minh chứng từ khách mua thực tế, chỉ cho phép tạo khi đơn hàng đã giao thành công, gắn đồng thời với người dùng, sản phẩm và đơn hàng.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
rating	int	none	-	-
comment	text	true	-	-
media_urls	json	none	-	-
reply_comment	text	true	-	-
is_hidden	tinyint(1)	none	true	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-
user_id	int	none	FK → users.id	-
order_id	int	none	FK → orders.id	-
product_id	int	none	FK → products.id	-

Bảng 3. 19 Chi tiết thực thể reviews

Trong đó:

- rating: Số điểm (1–5), được sử dụng để tính điểm trung bình hiển thị trên thẻ sản phẩm.
- media_urls: Mảng JSON chứa đường dẫn ảnh/video minh chứng từ khách, hiển thị trong phần review chi tiết.

- `reply_comment`: Phản hồi của cửa hàng dưới đánh giá — trường văn bản riêng, không ghi đề lên `comment` của khách.
- `is_hidden`: Đánh giá mặc định bị ẩn (`true`) cho đến khi quản trị viên duyệt hiện lên website — cơ chế kiểm duyệt.
- `order_id`: Tham chiếu đơn hàng, phục vụ kiểm tra "đánh giá này có gắn với đơn đã giao thành công không" khi hiển thị.

3.9 Nhóm kho và nhập hàng

3.9.1 purchaseorders (Đơn nhập hàng)

Ghi nhận quyết định nhập hàng từ nhà cung cấp, theo dõi trạng thái từ lúc đặt đến khi nhận đủ hoặc hủy.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	Khoá chính
id	int	none	auto_increment	true
supplier_id	int	none	FK suppliers.id →	-
order_date	datetime(3)	none	now()	-
expected_delivery_date	datetime(3)	none	-	-
status	enum(PENDING, RECEIVED, PARTIALLY_RECEIVED, CANCELLED)	none	PENDING	-
total_cost	decimal(10,2)	none	-	-

Bảng 3. 20 Chi tiết thực thể purchaseorders

Trong đó:

- `supplier_id`: Nhà cung cấp chịu trách nhiệm giao hàng cho đơn này.
- `expected_delivery_date`: Ngày dự kiến nhận hàng, phục vụ cảnh báo trễ hạn và lập kế hoạch kinh doanh.
- `total_cost`: Tổng chi phí nhập hàng theo đơn, dùng trong báo cáo tài chính và thống kê dashboard.

- status: Theo dõi tiến độ — PENDING (chờ giao), RECEIVED (đã nhận đủ), PARTIALLY_RECEIVED (nhận một phần), CANCELLED (hủy).

3.9.2 purchaseorderitems (Chi tiết đơn nhập)

Lưu từng dòng hàng nhập: biến thể nào, số lượng bao nhiêu, giá vốn bao nhiêu và đã nhận thực tế bao nhiêu.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
purchase_order_id	int	none	FK → purchaseorders.id	-
product_variant_id	int	none	FK → productvariants.id	-
quantity	int	none	-	-
unit_cost_price	decimal(10,2)	none	-	-
quantity_received	int	none	0	-

Bảng 3. 21 Chi tiết thực thể purchaseorderitems

Trong đó:

- unit_cost_price: Giá vốn nhập (giá mua từ NCC), khác với giá bán `productvariants.price` — hai giá này tạo nên biên lợi nhuận.
- quantity_received: Số lượng đã nhận thực tế — cho phép NCC giao hàng nhiều lần (PARTIALLY_RECEIVED), quản trị viên cập nhật theo từng lô.

3.9.3 stockmovements (Biến động tồn kho):

Thực thể phục vụ tính năng truy vết kho — mọi thay đổi số lượng đều được ghi lại một dòng lịch sử.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
variant_id	int	none	FK → productvariants.id	-

type	enum(IN, OUT, ADJUSTMENT)	none	-	-
quantity_change	int	none	-	-
reference_id	int	true	-	-
reason	varchar(255)	true	-	-
created_at	datetime(3)	none	now()	-

Bảng 3. 22 Chi tiết thực thể stockmovements

Trong đó có:

- id: Mã định danh duy nhất của bản ghi biến động.
- variant_id: Biến thể bị tác động.
- type: Loại biến động — IN khi nhập hàng, OUT khi bán, ADJUSTMENT khi kiểm kê điều chỉnh.
- quantity_change: Lượng thay đổi (âm khi xuất, dương khi nhập).
- reference_id: Tham chiếu nguồn gốc phát sinh (ví dụ mã đơn hàng, mã phiếu nhập).
- reason: Ghi chú lý do điều chỉnh.

3.10 Nhóm chương trình thành viên và tích điểm

3.10.1 membership_tiers (Hạng thành viên)

Định nghĩa các bậc khách hàng thân thiết theo tổng chi tiêu, seed cố định nhưng ngưỡng `min\_spent` có thể điều chỉnh.

THUỘC TÍNH	Kiểu dữ liệu	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
name	varchar(191)	none	-	-
min_spent	decimal(10,2)	none	-	-
reward_rate	decimal(5,2)	none	0	-
discount_percent	int	none	0	-
sort_order	int	none	0	-

is_active	tinyint(1)	none	true	-
deleted_at	datetime(3)	none	'1000-01-01'	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-

Bảng 3. 23 Chi tiết thực thể membership_tiers

Trong đó:

- min_spent: Tổng chi tiêu tối thiểu để đạt hạng — khi `users.total_spent ≥ min_spent`, hệ thống nâng hạng tự động.
- reward_rate: Tỷ lệ hoàn điểm (ví dụ 0.05 = cứ 1000đ nhận 0.05 điểm), áp dụng cho giao dịch mua hàng.
- discount_percent: Phần trăm giảm giá độc quyền cho hạng này, được áp dụng tự động khi thanh toán.
- sort_order: Thứ tự hiển thị trên giao diện quản trị và trang chương trình thành viên.

3.10.2 tier_rewards (Phần thưởng đổi điểm)

Mỗi hạng có bộ phần thưởng riêng (voucher, quà tặng...), khách dùng điểm để đổi lấy.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
tier_id	int	none	FK → membership_tiers.id	-
name	varchar(191)	none	-	-
point_cost	int	none	-	-
coupon_code	varchar(191)	true	-	-
is_active	tinyint(1)	none	true	-
deleted_at	datetime(3)	none	'1000-01-01'	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-

Bảng 3. 24 Chi tiết thực thể tier_rewards

Trong đó:

- tier_id: Phần thưởng chỉ dành cho đúng hạng thành viên — khách hạng thấp không thấy phần thưởng hạng cao.
- point_cost: Số điểm cần bỏ ra để đổi lấy phần thưởng này.
- coupon_code: Nếu phần thưởng là mã giảm giá thì lưu mã tương ứng, khi đổi thành công tự động ghi vào 'user_coupons'.

3.10.3 point_transactions (Nhật ký tích điểm)

Ghi lại cộng/trừ điểm, cung cấp số dư sau mỗi giao dịch để phục vụ truy vết và hiển thị lịch sử điểm.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
user_id	int	none	FK → users.id	-
type	varchar(191)	none	-	-
points	int	none	-	-
balance_after	int	none	-	-
order_id	int	true	FK → orders.id	-
coupon_id	int	true	FK → coupons.id	-
note	text	true	-	-
created_at	datetime(3)	none	now()	-

Bảng 3. 25 Chi tiết thực thể point_transactions

Trong đó:

- type: Phân loại giao dịch điểm (ví dụ "PURCHASE", "REDEEM", "REFUND", "ADJUSTMENT"), giúp lọc lịch sử.
- points: Số điểm thay đổi (dương khi cộng, âm khi trừ).
- balance_after: Số dư điểm sau giao dịch này — giúp hiển thị lịch sử mà không cần cộng dồn lại từ đầu.

- `order_id` / `coupon_id`: Tham chiếu nguồn phát sinh (đơn hàng mua, mã giảm giá dùng điểm, hoặc admin điều chỉnh thủ công).

3.11 Nhóm sản phẩm và giỏ hàng

3.11.1 productvariants (Biến thể sản phẩm)

Thực thể cốt lõi tạo nên điểm khác biệt của hệ thống — mỗi biến thể tương ứng một tổ hợp thuộc tính (màu × kích cỡ) với tồn kho và giá bán độc lập.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
stock	int	none	-	-
price	decimal(10,2)	none	-	-
product_id	int	none	FK → products.id	-
deleted_at	datetime(3)	none	'1000-01-01'	-

Bảng 3. 26 Chi tiết thực thể productvariants

Trong đó có:

- `id`: Mã định danh duy nhất cho mỗi biến thể, được tham chiếu bởi giỏ hàng, đơn hàng và kho.
- `stock`: Số lượng tồn kho hiện có của đúng tổ hợp này.
- `price`: Giá bán riêng của biến thể (có thể khác giá gốc của sản phẩm).
- `product_id`: Khóa ngoại tới sản phẩm cha.
- `deleted_at`: Xóa mềm biến thể khi ngừng kinh doanh size/màu.

3.11.2 products (Sản phẩm)

Thực thể lưu thông tin gốc của mỗi mặt hàng đồ thể thao, làm nền cho toàn bộ trang bán hàng và trang quản trị sản phẩm.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
name	varchar(191)	none	-	-

slug	varchar(191)	none	unique theo deleted_at	-
base_price	decimal(10,2)	none	-	-
description	text	none	-	-
thumbnail	text	true	-	-
is_active	tinyint(1)	none	true	-
category_id	int	none	FK → categories.id	-
brand_id	int	none	FK → brands.id	-
supplier_id	int	none	FK → suppliers.id	-
created_at	datetime(3)	none	now()	-
updated_at	datetime(3)	none	tự cập nhật	-
deleted_at	datetime(3)	none	'1000-01-01'	-

Bảng 3. 27 Chi tiết thực thể products

Trong đó có:

- id: Mã định danh duy nhất cho mỗi sản phẩm.
- name: Tên hiển thị của sản phẩm.
- slug: Chuỗi định danh trên URL, hỗ trợ SEO. Mã định danh dạng chữ viết liền (ví dụ `products:create`), được route middleware đối chiếu khi kiểm tra quyền truy cập endpoint.
- base_price: Giá cơ bản khi biến thể chưa khai báo giá riêng.
- description: Mô tả chi tiết sản phẩm.
- thumbnail: Ảnh đại diện của sản phẩm.
- is_active: Cờ cho phép sản phẩm hiển thị lên website.
- category_id / brand_id / supplier_id: Phân loại sản phẩm theo danh mục, thương hiệu và nguồn nhập hàng.
- deleted_at: Cột xóa mềm, giữ lịch sử mà không phá vỡ ràng buộc slug.

3.11.3 carts (Giỏ hàng)

Mỗi người dùng đã đăng nhập có một giỏ hàng duy nhất (1–1), lưu giây phút cuối cùng giỏ được cập nhật.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
updated_at	datetime(3)	none	tự cập nhật	-
user_id	int	none	FK → users.id	-

Bảng 3. 28 Chi tiết thực thể carts

Trong đó: updated_at: Ghi nhận lần cuối giỏ hàng thay đổi, phục vụ cleanup các giỏ hàng quá lâu không cập nhật (nếu cần).

3.11.4 cartitems (Dòng giỏ hàng)

Mỗi dòng ghi một biến thể và số lượng khách muốn mua; ràng buộc `UNIQUE(cart\_id, product\_variant\_id)` đảm bảo cùng một biến thể không xuất hiện hai dòng — thay vào đó hệ thống sẽ cộng dồn số lượng.

THUỘC TÍNH	KIỂU DỮ LIỆU	NULL	DỮ LIỆU MẶC ĐỊNH	KHOÁ CHÍNH
id	int	none	auto_increment	true
quantity	int	none	-	-
product_variant_id	int	none	FK → productvariants.id	-
cart_id	int	none	FK → carts.id	-

Bảng 3. 29 Chi tiết thực thể cartitems

Trong đó:

- quantity: Số lượng biến thể khách muốn mua, bị chặn không cho vượt tồn kho thực tế của biến thể.
- cart_id / product_variant_id: Cặp khóa ngoài tạo thành ràng buộc UNIQUENESS ở tầng database.

3.12 Nhóm nhật ký hệ thống

3.12.1 systemlogs (Nhật ký hoạt động)

Ghi lại mọi thao tác quan trọng của người dùng nội bộ (*tạo, sửa, xóa, điều chỉnh kho*), phục vụ truy vết bảo mật và giải quyết tranh chấp nội bộ.

THUỘC TÍNH	Kiểu dữ liệu	NULL	Dữ liệu mặc định	Khoá chính
id	int	none	auto_increment	true
timestamp	datetime(3)	none	now()	-
user_id	int	true	FK → users.id	-
action_type	varchar(50)	none	-	-
entity_type	varchar(50)	none	-	-
entity_id	int	true	-	-
status	enum(SUCCESS,FAILED)	true	-	-
ip_address	varchar(45)	true	-	-
details	json	true	-	-

Bảng 3. 30 Chi tiết thực thể systemlogs

Trong đó:

- action_type: Loại thao tác (CREATE, UPDATE, DELETE, STOCK_ADJUSTMENT...), cho phép lọc nhanh theo loại hành động.
- entity_type: Loại đối tượng bị tác động (Products, Orders, StockMovements...), phục vụ lọc theo phạm vi.
- entity_id: ID bản ghi bị ảnh hưởng — khi phát hiện bất thường, nhảy thẳng đến bản ghi đó để kiểm tra.
- status: SUCCESS hoặc FAILED, cho phép admin xem nhanh thao tác nào đã thất bại cần kiểm tra lại.
- ip_address: Địa chỉ IP của client thực thi thao tác, phục vụ xác minh nguồn gốc khi có nghi ngờ truy cập trái phép.
- details: JSON chứa dữ liệu thay đổi (ví dụ `{oldPrice: 100, newPrice: 120}`), cung cấp thông tin chi tiết để so sánh trước – sau.

3.13 Mối quan hệ thực thể chi tiết

Hệ thống SportNexus gồm 31 thực thể được chia thành 10 nhóm chức năng. Toàn bộ quan hệ giữa các thực thể được thiết kế theo ba kiểu cơ bản: **một-một (1-1)**, **một-nhiều (1-**

N) và **nhieu-nhiều (M-N)**. Mỗi kiểu quan hệ phản ánh đúng nghiệp vụ kinh doanh thực tế và được thực thi bằng khóa ngoại (foreign key) trên MySQL thông qua Prisma ORM.

ST T	Thực thể cha (Parent)	Thực thể con (Child)	Kiểu quan hệ	Khóa ngoại (Foreign Key)	Hành vi xóa (onDelete)
I Nhóm Người dùng & Phân quyền					
1	Roles	Users	1 - N	role_id	Cascade
2	Users	UserAddresses	1 - N	user_id	Cascade
3	Users	Orders	1 - N	userId	Restrict / -
4	Users	Reviews	1 - N	user_id	Cascade
5	Users	UserCoupons	1 - N	user_id	Cascade
6	Users	SystemLogs	1 - N	user_id	Restrict / -
7	Users	PointTransactions	1 - N	user_id	Cascade
8	Users	Carts	1 - 1	user_id	Cascade
9	Users	Permissions	M - N	(Bảng trung gian)	-
10	Users	Coupons	M - N	(Qua UserCoupons)	-
II Nhóm Sản phẩm & Kho hàng					
11	Categories	Products	1 - N	category_id	Cascade

12	Categories	Collections	1 - N	category_id	Cascade
13	Brands	Products	1 - N	brand_id	Cascade
14	Suppliers	Products	1 - N	supplier_id	Cascade
15	Suppliers	PurchaseOrders	1 - N	supplier_id	Cascade
16	Products	ProductImages	1 - N	product_id	Cascade
17	Products	ProductVariants	1 - N	product_id	Cascade
18	Products	ProductAttributeKeys	1 - N	product_id	Cascade
19	Products	Reviews	1 - N	product_id	Cascade
20	ProductVariants	VariableAttributes	1 - N	variable_id	Cascade
21	ProductVariants	OrderItems	1 - N	product_variant_id	Cascade
22	ProductVariants	CartItems	1 - N	product_variant_id	Cascade
23	ProductVariants	StockMovements	1 - N	variant_id	Cascade
24	ProductVariants	PurchaseOrderItems	1 - N	product_variant_id	Cascade
25	AttributeKeys	VariableAttributes	1 - N	attribute_key_id	Cascade
26	AttributeKeys	ProductAttributeKeys	1 - N	attribute_key_id	Cascade
27	PurchaseOrders	PurchaseOrderItems	1 - N	purchase_order_id	Cascade
III Nhóm Đơn hàng, Giỏ hàng & Thanh toán					

28	Orders	OrderItems	1 - N	order_id	Cascade
29	Orders	PaymentTransactions	1 - N	order_id	Cascade
30	Orders	Reviews	1 - N	order_id	Cascade
31	Orders	Invoices	1 - 1	order_id (UNIQUE)	Cascade
32	Orders	PointTransactions	1 - N	order_id	Restrict / -
33	Carts	CartItems	1 - N	cart_id	Cascade
IV Nhóm Khuyến mãi & Thành viên					
34	Coupons	Orders	1 - N	coupon_code	Cascade
35	Coupons	UserCoupons	1 - N	coupon_id	Cascade
36	Coupons	PointTransactions	1 - N	coupon_id	Restrict / -
37	MembershipTiers	Users	1 - N	tier_id	Restrict / -
38	MembershipTiers	TierRewards	1 - N	tier_id	Cascade

Bảng 3. 31 liệt kê đầy đủ các cặp quan hệ chính trong hệ thống

Bảng trên cho thấy toàn bộ 37 cặp quan hệ trong hệ thống, trong đó có 35 quan hệ 1-N và 2 quan hệ M-N (*Users–Permissions* và *Users–Coupons*). Không có quan hệ 1-1 nào được khai báo trực tiếp bằng Prisma — thay vào đó, cả hai trường hợp 1-1 (*Users–Carts* và *Orders–Invoices*) đều được đảm bảo bằng ràng buộc '@unique' trên khóa ngoại phía con.

3.14 Nguyên tắc toàn vẹn dữ liệu và khóa

Toàn bộ quan hệ trong hệ thống đều được thực thi bằng **khóa ngoại (foreign key)** trên MySQL, đảm bảo nguyên tắc toàn vẹn tham chiếu — không thể tồn tại bản ghi con tham chiếu đến bản ghi cha đã bị xóa.

Soft delete nhất quán: Hệ thống sử dụng cơ chế soft delete bằng trường `deleted_at` trên hầu hết các thực thể chính (*Users, Products, Categories, Brands, Suppliers, Coupons, MembershipTiers...*). Giá trị mặc định `'1000-01-01 00:00:00'` biểu thị bản ghi đang hoạt động; khi xóa, trường này được cập nhật thành thời gian hiện tại. Mọi truy vấn danh sách đều kèm điều kiện `deleted_at = '1000-01-01 00:00:00'` để chỉ hiển thị bản ghi đang hoạt động.

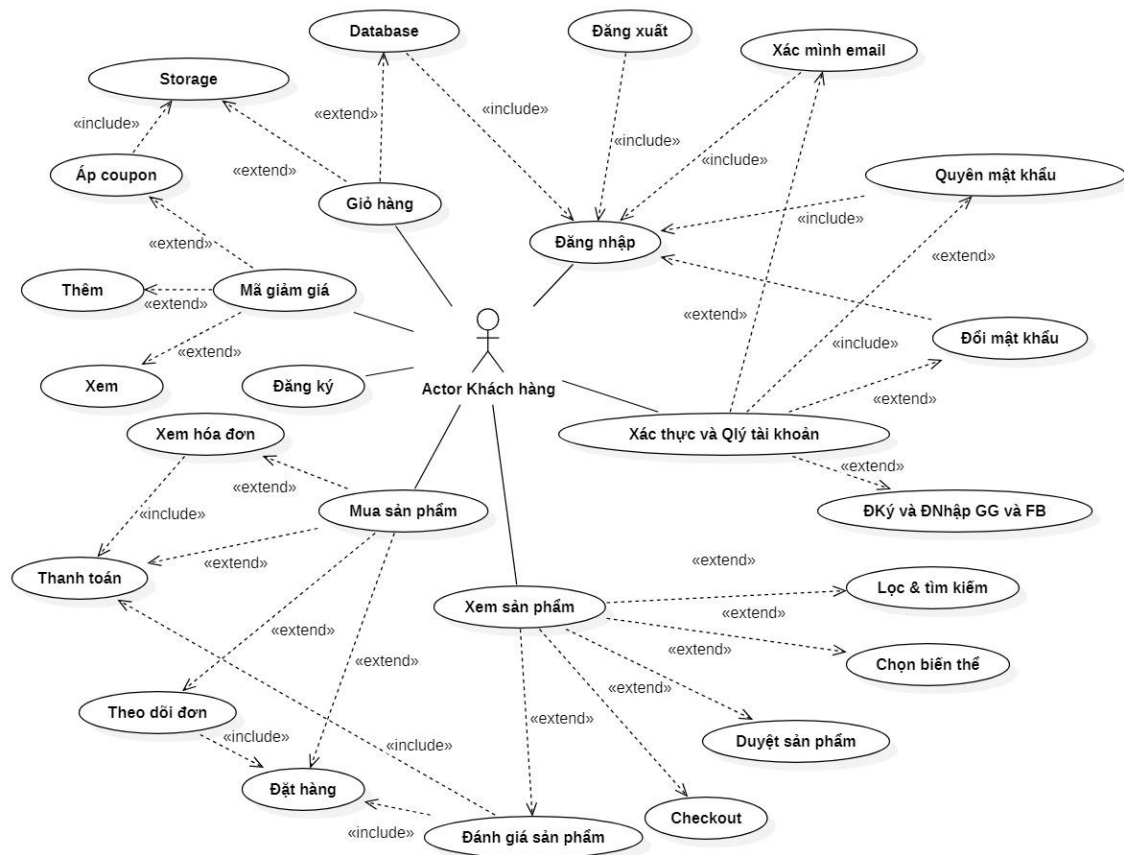
Unique constraint kết hợp soft delete: Nhiều bảng sử dụng `'@@unique([field, deleted_at])'` (ví dụ `'@@unique([email, deleted_at])'` trên *Users*, `'@@unique([slug, deleted_at])'` trên *Products*) để cho phép cùng một giá trị tồn tại sau khi bản ghi cũ bị soft delete — tức là sau khi xóa một danh mục "Giày", vẫn có thể tạo lại danh mục "Giày" mới mà không vi phạm ràng buộc duy nhất.

Ràng buộc nghiệp vụ qua code: Một số quy tắc toàn vẹn không thể thể hiện bằng schema mà được kiểm soát ở tầng service:

- Một biến thể không thể có hai giá trị cùng loại thuộc tính (*đảm bảo bởi* `'@@unique([variable_id, attribute_key_id])'` trên *VariableAttributes*).
- Cùng một người dùng không thể lưu cùng một mã giảm giá hai lần (*đảm bảo bởi* `'@@unique([user_id, coupon_id])'` trên *UserCoupons*).
- Số dư điểm của người dùng không bao giờ bị âm — toàn bộ giao dịch cộng/trừ điểm chạy trong `'$transaction'`, kiểm tra số dư trước khi thực thi.
- Mã giảm giá chỉ có thể sử dụng khi còn trong thời hạn (`'start_date <= now() <= end_date'`) và lượt dùng còn lại (`'usage_count < usage_limit'`).
- Đánh giá chỉ có thể tạo khi đơn hàng ở trạng thái `'Delivered'` — kiểm soát ở tầng controller trước khi gọi Prisma create.

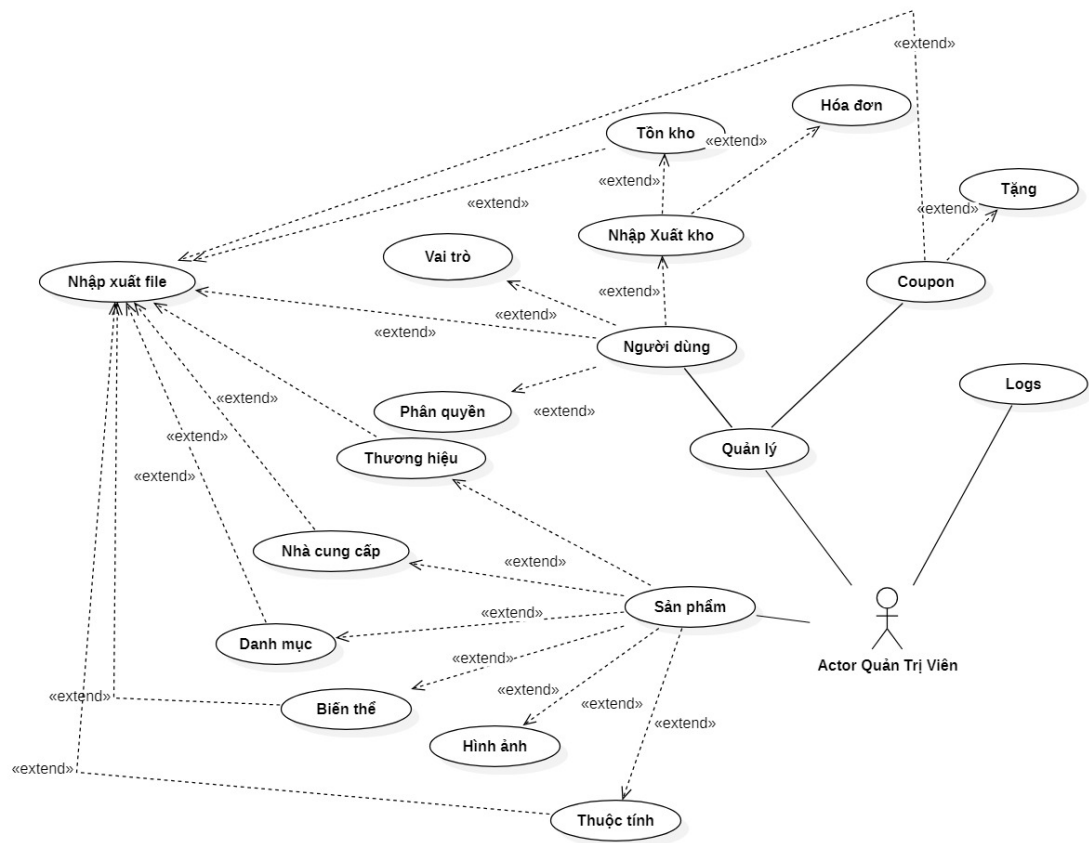
3.15 Biểu đồ use case tổng quát

3.15.1 Use case — Khách hàng



Hình 3. 4. Sơ đồ use case của actor khách hàng

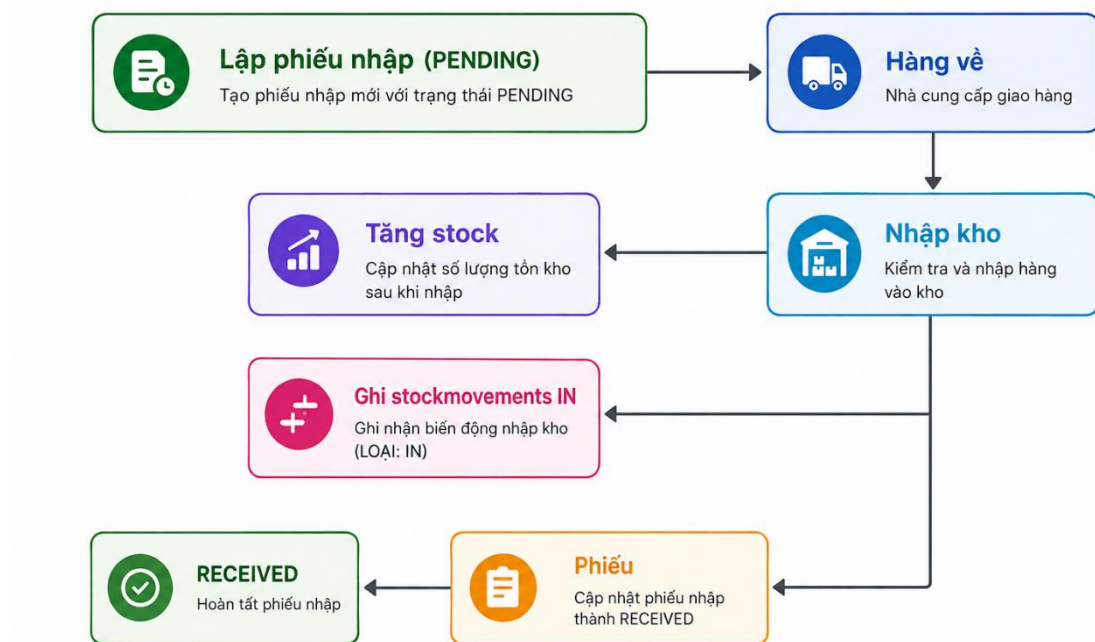
3.15.2 Use case — Quản trị viên



Hình 3. 5. Sơ đồ use case của actor quản trị viên (Admin)

3.16 Phân tích các quy trình nghiệp vụ chính

3.16.1 Quy trình nhập kho



Hình 3. 6. Lưu đồ chi tiết quy trình nghiệp vụ Nhập kho

Lập phiếu nhập (PENDING): Ý nghĩa: Khi cửa hàng đặt mua hàng từ nhà cung cấp, nhân viên quản lý sẽ tạo một phiếu nhập kho mới trên hệ thống. Ban đầu, phiếu này sẽ mang trạng thái **PENDING** (*Đang chờ xử lý / Chờ hàng về*).

Hàng về & Nhập kho:

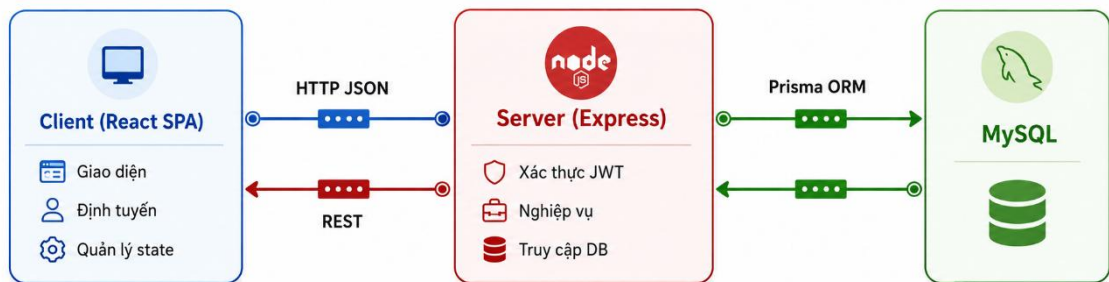
- **Hàng về:** Lô hàng thực tế được vận chuyển đến kho của cửa hàng.
- **Nhập kho:** Thủ kho hoặc người quản lý tiến hành kiểm đếm thực tế và xác nhận thao tác nhập kho trên phần mềm. Hành động này kích hoạt hệ thống thực hiện 3 nhiệm vụ cốt lõi đồng thời (trong một giao dịch cơ sở dữ liệu - *Transaction*):

Ba tác vụ xử lý tự động khi nhập kho:

- **Tăng stock:** Hệ thống tự động cộng dồn số lượng hàng thực tế vừa nhập vào trường tồn kho (stock) của các biến thể sản phẩm tương ứng trong bảng ProductVariants.
- **Ghi stockmovements IN:** Hệ thống ghi nhận một dòng nhật ký biến động kho (loại IN - nhập kho) vào bảng StockMovements. Việc này giúp lưu vết lịch sử

(Ai nhập? Vào lúc nào? Số lượng bao nhiêu?) để phục vụ việc kiểm toán và đối soát sau này.

- **Cập nhật phiếu thành RECEIVED:** Trạng thái của phiếu nhập ban đầu được chuyển từ PENDING sang **RECEIVED** (Đã nhận hàng), đánh dấu hoàn tất chu trình nhập kho và khóa phiếu lại để tránh chỉnh sửa ngoài ý nghĩa.



Hình 3. 7. Lưu đồ luồng data của quy trình nhập kho

Chi tiết các bước trong sơ đồ:

1. **Bắt đầu:** Khởi tạo tiến trình nhập hàng khi cửa hàng lên kế hoạch thu mua từ nhà cung cấp.
2. **Tạo Purchase Order + items + giá vốn:**
 - Nhân viên hoặc quản lý tiến hành lập phiếu đặt hàng mua (Purchase Orders).
 - Thêm các mặt hàng chi tiết vào đơn (Purchase Order Items) kèm theo đơn giá nhập kho (giá vốn) của từng sản phẩm biến thể.
3. **Kiểm + nhận hàng từ nhà cung cấp:**
 - Khi lô hàng vật lý được vận chuyển tới kho, nhân viên tiến hành khai thùng, kiểm đếm thực tế số lượng và chất lượng sản phẩm so với phiếu đặt hàng ban đầu.
4. **Cập nhật tồn kho và trạng thái đơn hàng (Xử lý hệ thống):**
 - **Tăng stock variant:** Hệ thống tự động cộng dồn số lượng thực tế vừa nhập vào trường tồn kho (stock) của từng biến thể sản phẩm (product_variants).

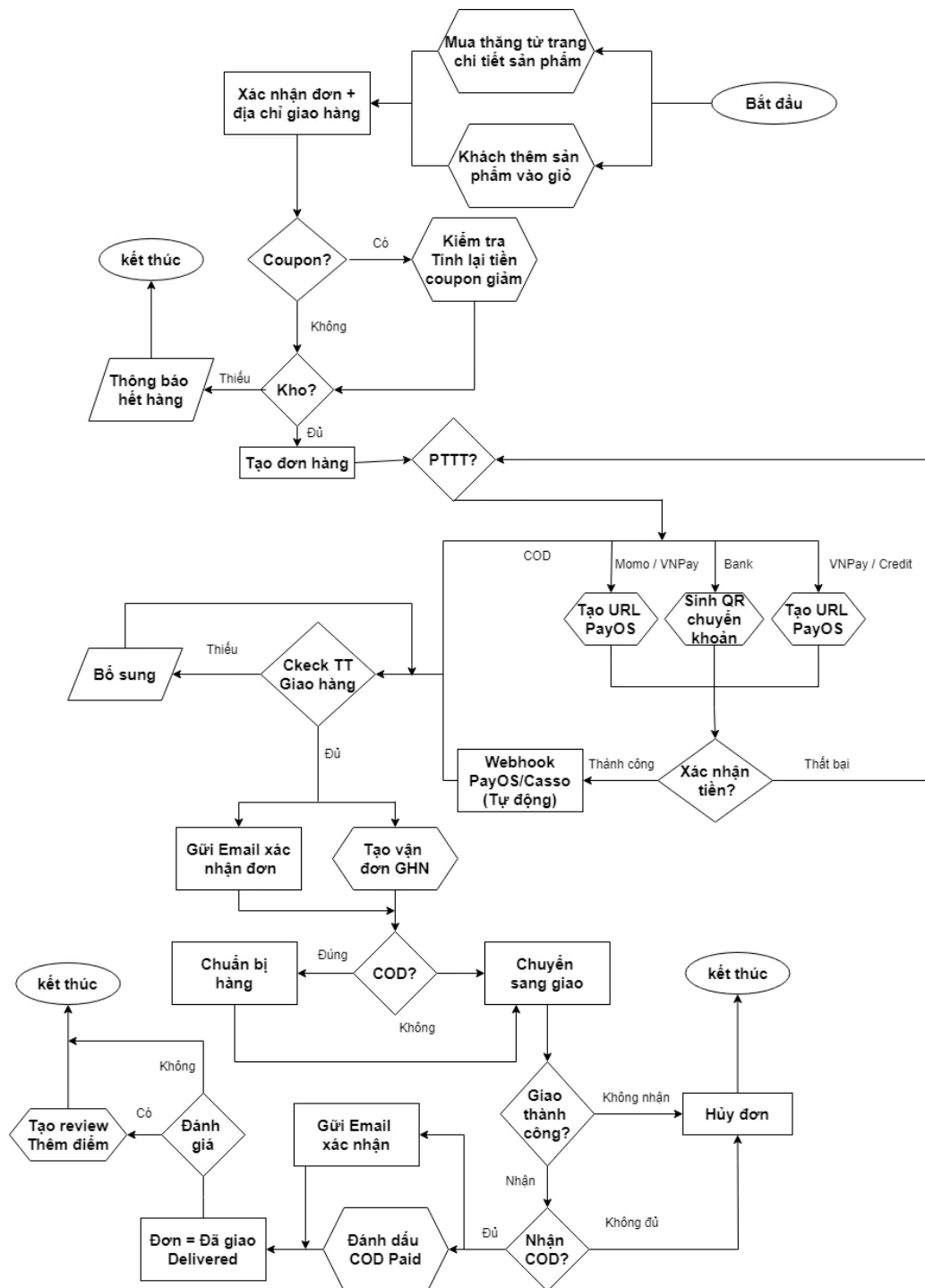
- **Cập nhật trạng thái PO:** Nếu đơn hàng có liên kết mã (order_id), hệ thống sẽ tự động chuyển trạng thái của phiếu nhập mua (PO) sang **RECEIVED** (Đã nhận hàng đầy đủ), ghi nhận hoàn tất giao dịch nhập kho.

5. **Kết thúc:** Hoàn tất chu trình nhập hàng, dữ liệu tồn kho và lịch sử dòng tiền/giá vốn đã được ghi nhận đồng bộ vào cơ sở dữ liệu.

3.16.2 Quy trình bán hàng

Các điểm kiểm soát chính:

- Bắt buộc kiểm tra tồn kho trước khi xác nhận đơn hàng.
- Cập nhật trạng thái tuần tự theo chuỗi quy định rõ ràng.
- Trạng thái giao thành công (Delivered) sẽ kích hoạt ghi nhận thanh toán đối với các đơn hàng COD.



Hình 3. 8. Lưu đồ luồng data của quy trình bán hàng

Chi tiết các bước trong sơ đồ:

1. Khởi tạo và Kiểm tra điều kiện đơn hàng

- **Bắt đầu:** Khách hàng có thể chọn **Mua thẳng từ trang chi tiết sản phẩm** hoặc **Thêm sản phẩm vào giỏ hàng** để tiến hành đặt mua.
- **Xác nhận thông tin:** Hệ thống yêu cầu người dùng xác nhận đơn hàng và địa chỉ giao hàng.

- **Xử lý Mã giảm giá (Coupon):**
 - Nếu có dùng mã, hệ thống kiểm tra và tính toán lại số tiền được giảm giá.
 - Nếu không, bỏ qua bước này.
- **Kiểm tra tồn kho (Kho?):**
 - Hệ thống kiểm tra số lượng tồn kho thực tế của các biến thể sản phẩm.
 - Nếu **Thiếu**, hiển thị thông báo hết hàng và kết thúc tiến trình.
 - Nếu **Đủ**, tiến hành **Tạo đơn hàng** trong cơ sở dữ liệu và chuyển sang bước chọn phương thức thanh toán (PTTT).

2. Phân nhánh Phương thức thanh toán (PTTT)

Tùy thuộc vào lựa chọn của khách hàng, hệ thống chia thành các luồng xử lý riêng biệt:

- **Thanh toán khi nhận hàng (COD):**
 - Hệ thống kiểm tra thông tin giao hàng. Nếu thiếu, yêu cầu bổ sung.
 - Nếu đủ, đồng thời thực hiện hai tác vụ: **Gửi Email xác nhận đơn** và **Tạo vận đơn GHN** (Giao Hàng Nhanh).
- **Thanh toán trực tuyến (Momo, VNPAY, Credit Card, Chuyển khoản ngân hàng):**
 - Đối với ví điện tử/thẻ: Hệ thống gọi API **tạo URL thanh toán PayOS**.
 - Đối với chuyển khoản ngân hàng: Hệ thống **sinh mã QR chuyển khoản**.
 - Hệ thống chờ xác nhận tiền: Nếu thất bại, quay lại bước chọn phương thức thanh toán; nếu thành công, **Webhook tự động từ PayOS/Casso** ghi nhận, đồng thời kích hoạt việc **Gửi Email xác nhận đơn** và **Tạo vận đơn GHN**.

3. Vận chuyển và Giao nhận hàng hóa

- **Chuẩn bị hàng:** Đối với đơn COD, nhân viên kho tiến hành đóng gói hàng hóa (Chuẩn bị hàng) trước khi chuyển cho đơn vị vận chuyển.
- **Giao hàng (Chuyển sang giao):** Đơn vị vận chuyển tiến hành giao hàng cho khách.

- **Giao thành công (Nhận):**
 - Nếu là đơn COD, hệ thống kiểm tra số tiền thu hộ (Nhận COD?). Nếu đủ, tiến hành **đánh dấu COD Paid**, gửi email xác nhận và cập nhật trạng thái **Đơn = Đã giao (Delivered)**.
 - Nếu là đơn đã thanh toán trước (Prepaid), chuyển thẳng sang trạng thái **Đã giao (Delivered)**.
- **Không thành công (Không nhận):** Hệ thống tiến hành **Hủy đơn** và kết thúc.

4. Hậu mãi và Đánh giá sản phẩm

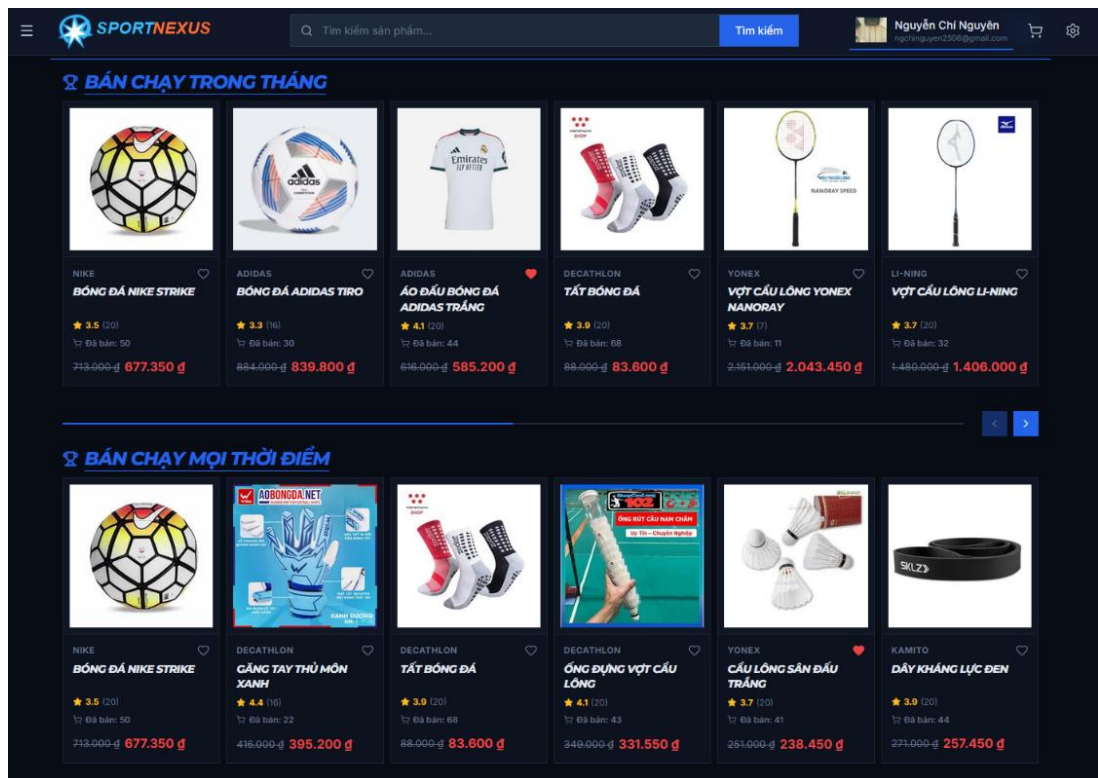
- Sau khi đơn hàng hoàn tất ở trạng thái *Đã giao*, khách hàng có thể thực hiện **Đánh giá sản phẩm**:
 - Nếu khách hàng đánh giá, hệ thống sẽ **Tạo review và cộng điểm thưởng** vào tài khoản thành viên (points_balance), sau đó kết thúc quy trình.
 - Nếu không đánh giá, quy trình kết thúc tại đây.

3.17 Giao diện và chức năng (Client)

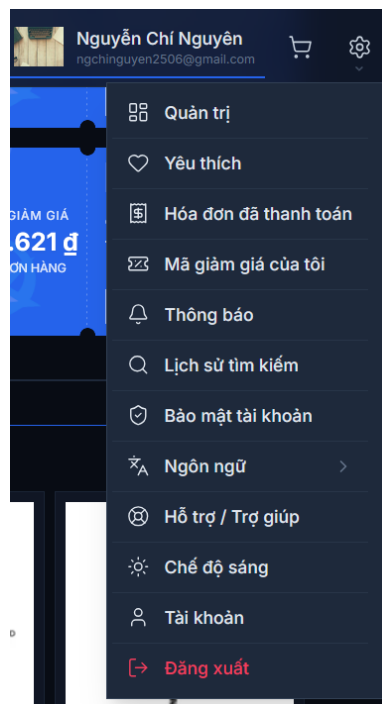
3.17.1 Trang chủ

Trang chủ là giao diện đầu tiên người dùng nhìn thấy khi truy cập website SportNexus. Giao diện được thiết kế responsive, thích ứng cả máy tính lẫn điện thoại, chia thành các khu vực rõ ràng theo bố cục dọc. Phần đầu trang là banner chính (*hero banner*) quảng bá chương trình khuyến mãi nổi bật, tiếp nối là dãy banner danh mục giúp khách nhanh chóng đi tới nhóm hàng quan tâm (*giày, quần áo, phụ kiện...*).

Ngay bên dưới là khu vực hiển thị các mã giảm giá đang chạy (coupons section) cho phép khách xem điều kiện và sao chép mã để dùng khi thanh toán. Phần giữa trang gồm các khối sản phẩm: **hàng mới về** (*sắp xếp theo thời gian tạo gần nhất*), **khuyến mãi đặc biệt** (*sản phẩm đang giảm giá*) và các section sản phẩm theo nhóm — mỗi thẻ sản phẩm hiển thị ảnh đại diện, tên, giá (*kể cả giá gốc bị gạch ngang khi có giảm giá*) để khách nhấn vào xem chi tiết. Toàn bộ nội dung hỗ trợ song ngữ Việt/Anh thông qua bộ i18next.



Hình 3. 9 Giao diện trang chủ



Hình 3. 10 Giao diện menu ở header

Quản trị: Chức năng đặc quyền dành cho Admin/Nhân viên, dùng để điều hướng vào khu vực trang quản trị (*Dashboard*) để quản lý sản phẩm, đơn hàng, người dùng,...

Yêu thích: Nơi lưu trữ danh sách các sản phẩm mà người dùng đã thả tim hoặc đánh dấu muốn mua trong tương lai (*Wishlist*).

Hóa đơn đã thanh toán: Quản lý lịch sử mua hàng, cho phép người dùng tra cứu lại các đơn hàng đã hoàn tất thanh toán hoặc theo dõi tình trạng đơn.

Mã giảm giá của tôi: Khu vực quản lý các voucher, coupon, mã khuyến mãi hoặc điểm thưởng mà tài khoản này đang sở hữu.

Lịch sử tìm kiếm: Lưu lại các từ khóa hoặc sản phẩm mà người dùng đã tìm kiếm trước đó, giúp tăng cường trải nghiệm mua sắm cá nhân hóa.

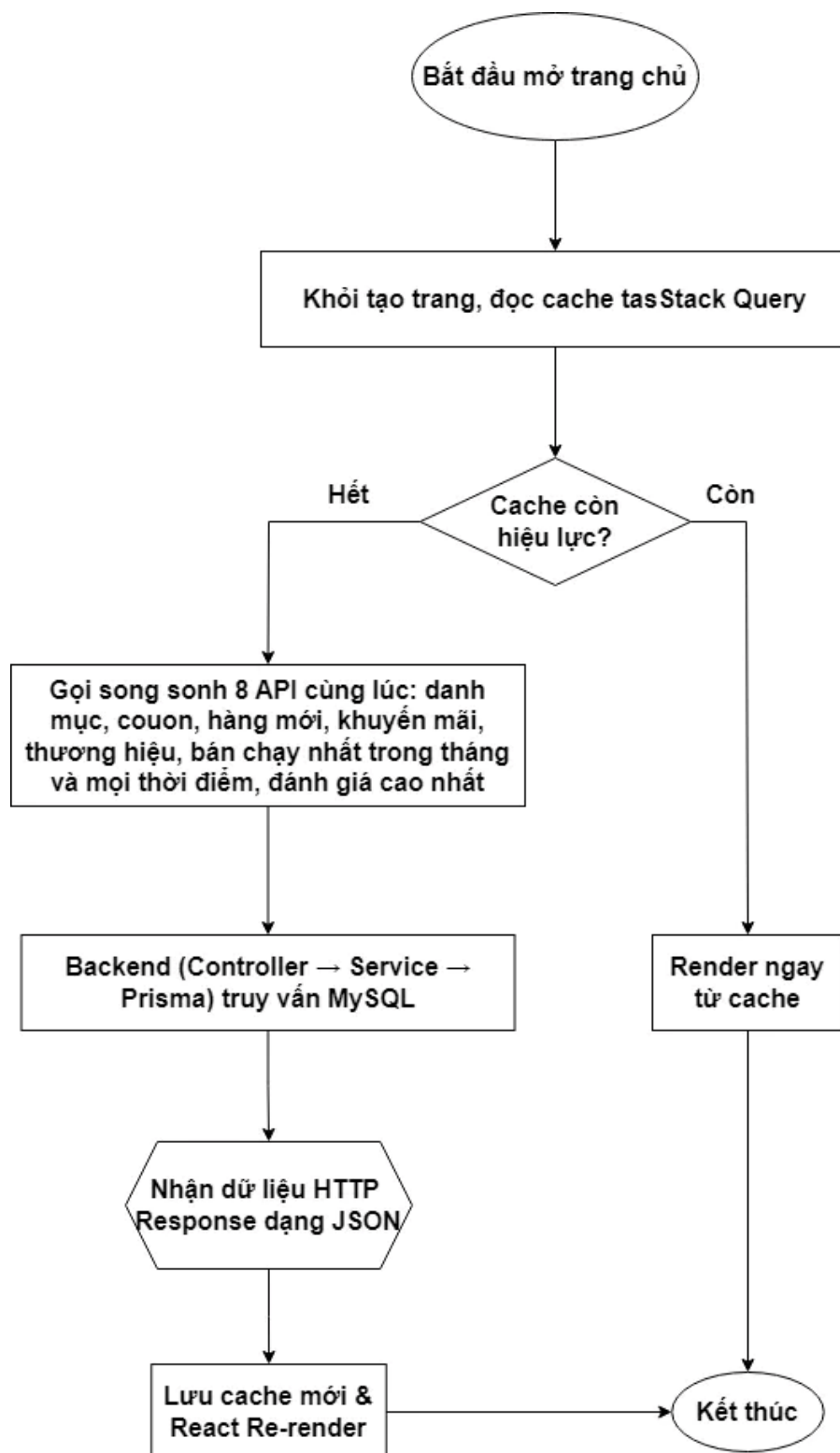
Ngôn ngữ (>): Cho phép thay đổi ngôn ngữ hiển thị của toàn bộ trang web (*có mũi tên chỉ báo sẽ mở ra một menu con*) hiện tại có 2 ngôn ngữ tiếng anh và tiếng việt.

Hỗ trợ / Trợ giúp: Điều hướng đến trang chăm sóc khách hàng, trung tâm trợ giúp hoặc các câu hỏi thường gặp (*FAQ*).

Chế độ sáng: Nút chuyển đổi nhanh giao diện (Theme Toggle). Hiện tại hệ thống đang ở chế độ Tối (Dark Mode), bấm vào đây sẽ chuyển sang giao diện Sáng.

Tài khoản: Truy cập vào trang cập nhật hồ sơ cá nhân (*thay đổi thông tin cá nhân, cập nhật số địa chỉ giao hàng*).

Đăng xuất: Nút lệnh màu đỏ nổi bật giúp người dùng kết thúc và hủy phiên làm việc hiện tại (*xóa Token/Session bảo mật*), đưa hệ thống về lại trạng thái khách vắng lai.



Hình 3. 11 – Lưu đồ lấy dữ liệu trang chủ

Trang chủ chỉ phát sinh **một request duy nhất** (`GET /home`) do React Router loader gửi trước khi component render — người dùng không thấy màn hình trắng thiếu dữ liệu. Phía backend, `homeService.getHomePageData()` dùng `Promise.all` để chạy **song song 8 truy vấn**: sản phẩm mới (*lọc `is_active`, còn tồn kho biến thể*), danh mục, thương hiệu, hai bảng xếp hạng bán chạy (*theo tháng và toàn thời gian, tổng hợp từ*

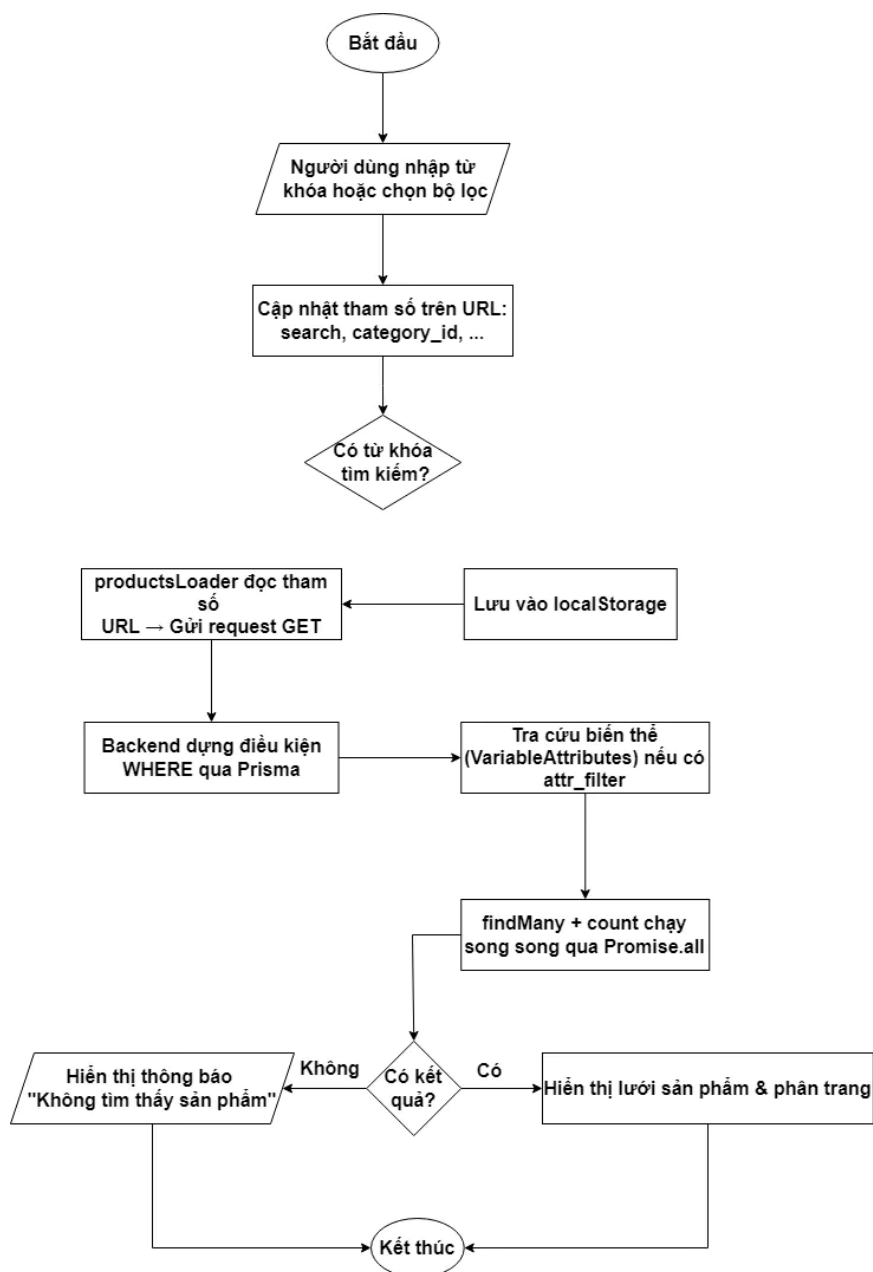
3.17.2 Danh sách sản phẩm – tìm kiếm và lọc

Bộ lọc (filter sidebar) hỗ trợ các tiêu chí chính:

- Kết quả được hiển thị dạng lưới thẻ sản phẩm kèm **phân trang** phía backend để giảm lượng dữ liệu tải về. Khi người dùng bấm vào một sản phẩm bất kỳ, hệ thống chuyển sang trang chi tiết tương ứng.



Hình 3. 12 Giao diện trang danh sách sản phẩm – tìm kiếm và lọc



Hình 3. 13 – Lưu đồ lọc và tìm kiếm sản phẩm

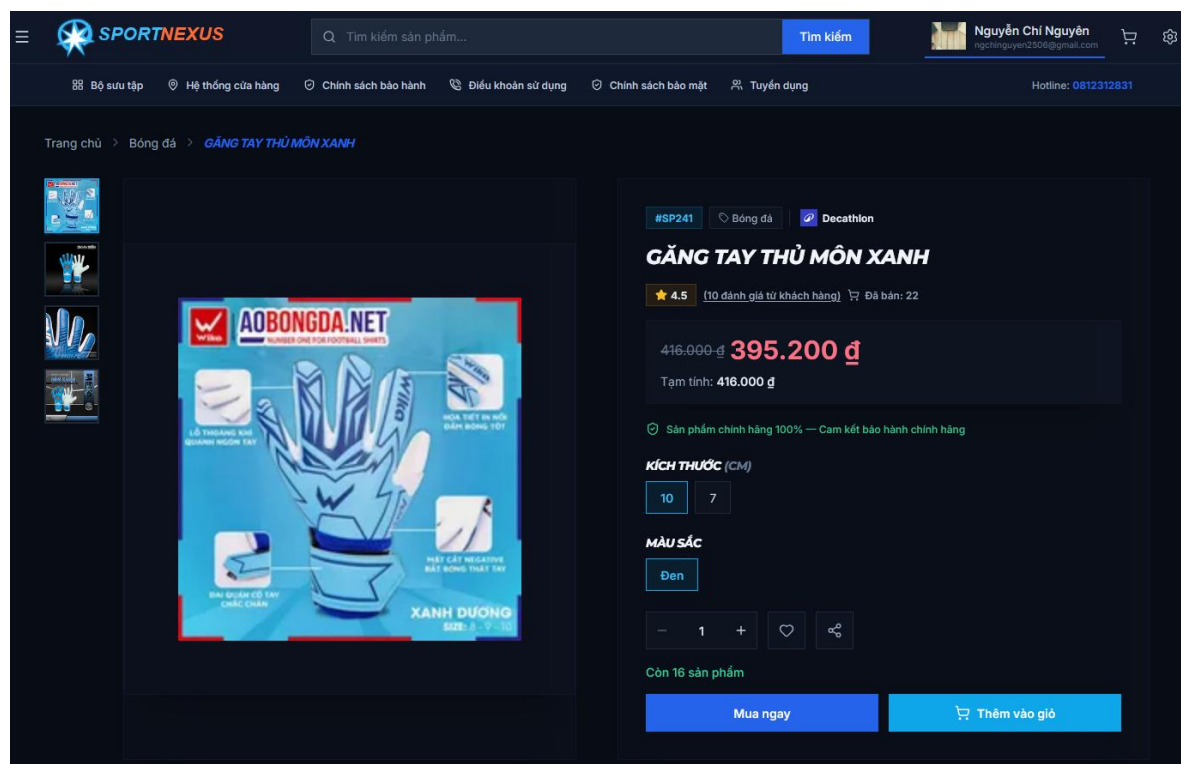
Toàn bộ tiêu chí lọc được ghi lên **URL**, nhờ vậy người dùng có thể chia sẻ hoặc quay lại bằng nút Back mà trạng thái lọc vẫn giữ nguyên. React Router loader đọc tham số từ URL rồi gửi đúng một request đến `/home/product/products`. Backend lần lượt ghép các điều kiện vào mệnh đề WHERE của Prisma (chỉ nhận sản phẩm đang hoạt động, chưa xóa mềm); riêng bộ lọc theo thuộc tính (*attr_filter=Kích cỡ:42,...*) hệ thống truy vấn trước vào bảng biến thể để rút ra danh sách sản phẩm tương ứng, sau đó mới lọc tiếp trên bảng sản phẩm. Truy vấn lấy trang dữ liệu và truy vấn đếm tổng số bản ghi chạy song song để tính phân trang; cuối cùng hệ thống bổ sung số lượng đã bán cho từng sản phẩm trước khi trả về. Từ khóa

tìm gần đây được lưu vào lịch sử tìm kiếm ngay trên trình duyệt (*localStorage*) để hiển thị gợi ý khi người dùng gõ tiếp.

3.17.3 Chi tiết sản phẩm và chọn biến thể

Đây là màn hình trọng tâm của trải nghiệm mua sắm. Trang chi tiết hiển thị thư viện ảnh, tên, giá, mô tả, thông tin thương hiệu và phần đánh giá của khách hàng. Điểm đặc biệt là cơ chế **chọn biến thể**: hệ thống gom toàn bộ thuộc tính của sản phẩm (*màu sắc, kích cỡ...*) rồi tính toán tập giá trị khả dụng dựa trên tồn kho thực tế — những tổ hợp hết hàng sẽ bị vô hiệu hóa thay vì cho phép chọn xong mới báo lỗi.

Khi người dùng chọn đủ các thuộc tính, frontend tự động khớp về đúng biến thể tương ứng và cập nhật giá theo biến thể đó. Từ đây khách có thể thêm vào giỏ hàng hoặc mua ngay; nếu sản phẩm chưa được đánh giá thì hiển thị trạng thái chưa có đánh giá thay vì để trống.

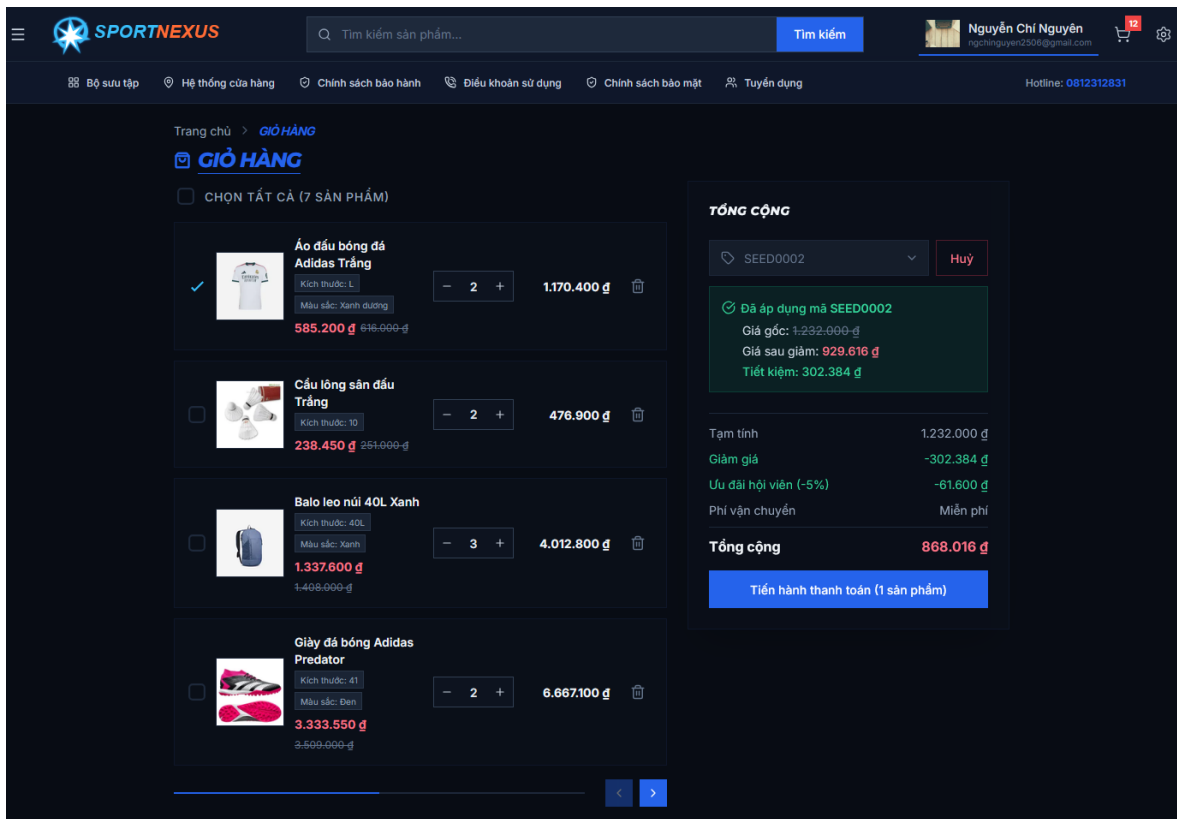


Hình 3. 14 Giao diện trang chi tiết sản phẩm

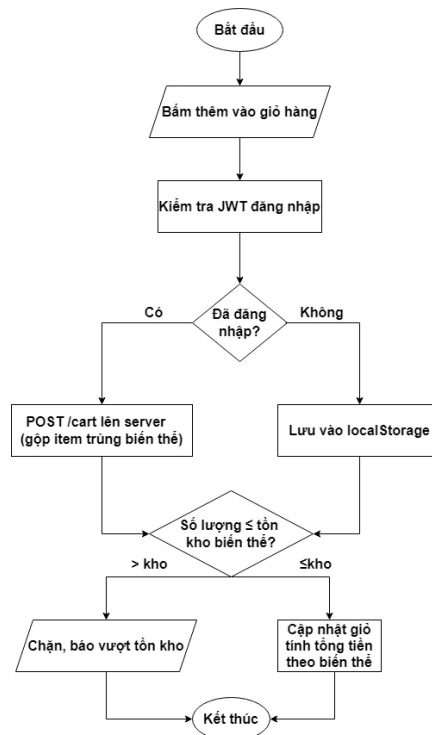
3.17.4 Giỏ hàng

Giỏ hàng hỗ trợ đồng thời hai loại khách: khách vắng lai lưu dữ liệu tại *localStorage* của trình duyệt, và khách đã đăng nhập có giỏ lưu trên máy chủ. Ngay sau khi đăng nhập, hệ thống tự động đồng bộ giỏ: các mặt hàng trong local và trên server được gộp lại, item trùng biến thể được cộng dồn số lượng.

Tại trang giỏ hàng, người dùng có thể tăng/giảm số lượng, xóa từng món hoặc làm trống giỏ; số lượng không được vượt quá tồn kho hiện có của biến thể. Tổng tiền được tính lại tức thời theo giá của từng biến thể.



Hình 3. 15 Giao diện trang giỏ hàng



Hình 3. 16 Lưu đồ thêm vào giỏ hàng

Tùy trạng thái đăng nhập (*JWT*), item được đẩy lên server hoặc lưu *localStorage*; số lượng luôn bị chặn nếu vượt tồn kho biến thể, hợp lệ thì cập nhật giỏ và tính lại tổng tiền theo giá từng biến thể.

3.17.5 Đặt hàng và thanh toán (Checkout)

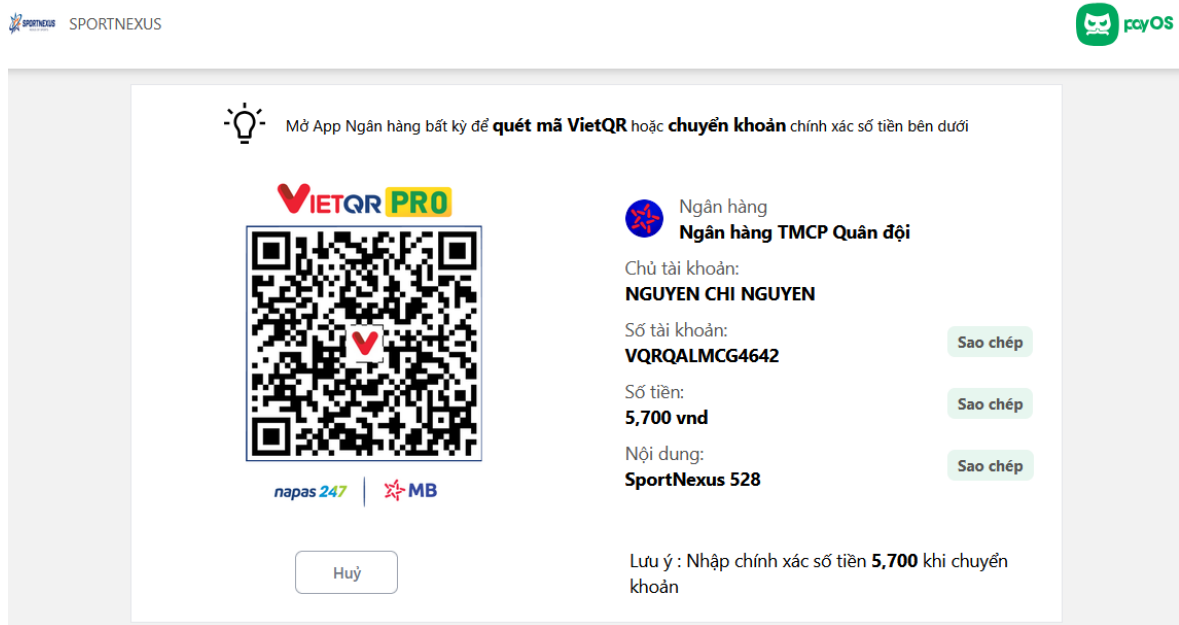
Trang checkout tập trung toàn bộ thao tác chốt đơn gồm các khu vực:

- **Thông tin liên hệ:** email, họ tên, số điện thoại nhận đơn.
- **Địa chỉ giao hàng:** chọn nhanh từ sổ địa chỉ đã lưu (*tỉnh/thành phố* → *quận/huyện* → *phường/xã theo dữ liệu hành chính*) hoặc nhập địa chỉ mới; hệ thống **tính phí vận chuyển động** dựa trên tỉnh thành và cân nặng của đơn.
- **Phương thức thanh toán:** COD (*tiền mặt khi nhận hàng*) hoặc chuyển khoản qua cổng **PayOS**; khi chưa cấu hình cổng, hệ thống hiển thị **mã QR** và đối soát tự động qua webhook ngân hàng Casso.
- **Điểm thưởng:** khách hạng thành viên có thể nhập số điểm muốn sử dụng để trừ vào tổng tiền.
- **Tóm tắt đơn:** liệt kê sản phẩm, phí ship, giảm giá từ coupon và tổng cộng.

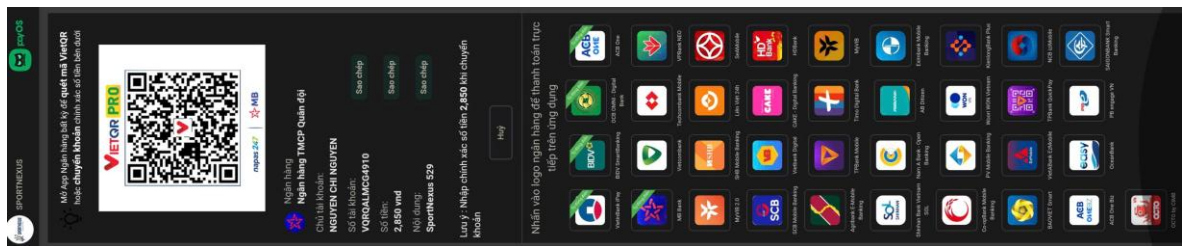
Khi khách nhân đặt hàng, backend thực hiện toàn bộ nghiệp vụ trong **một transaction** — tạo đơn, trừ tồn kho từng biến thể, phát hành hóa đơn, tạo vận đơn — thành công thì chuyển hướng: COD về trang **OrderSuccess** chờ xác nhận, thanh toán online chuyển tới cổng PayOS và quay về sau khi hoàn tất. Mã coupon áp dụng cũng được kiểm tra hợp lệ (*thời hạn, giá trị tối thiểu, lượt dùng*) trước khi chốt đơn.

The screenshot shows the checkout interface of the SPORTNEXUS website. The page layout includes a header with the logo, search bar, and user account information. The main content area is titled 'THANH TOÁN' (Checkout) and contains several sections for user input and order summary. The 'THÔNG TIN LIÊN HỆ' section includes fields for email, name, and phone number. The 'ĐỊA CHỈ GIAO HÀNG' section has dropdown menus for selecting the province and district, followed by a text field for the specific address. The 'PHƯƠNG THỨC THANH TOÁN' section offers options for cash on delivery or bank transfer. The 'ĐƠN HÀNG' section lists the items being purchased. There is also a section for using loyalty points to discount the order. The final summary shows the subtotal, discount, and total amount, with a prominent 'Đặt hàng' button to complete the purchase.

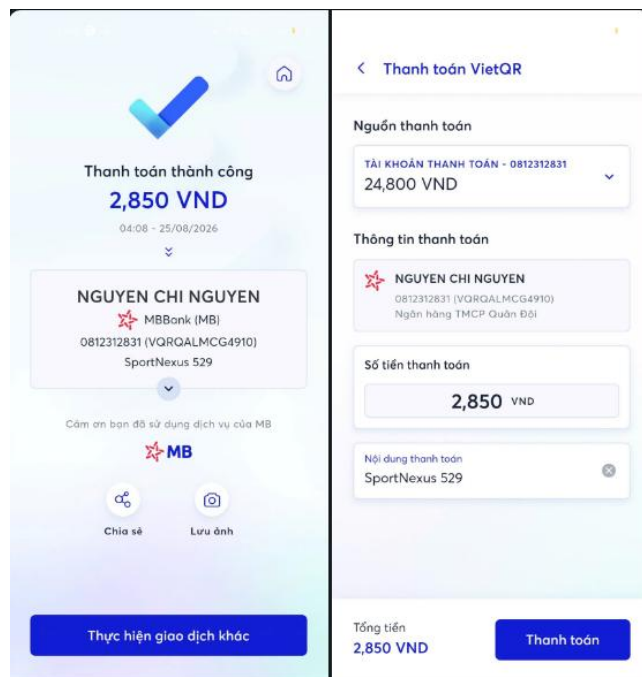
Hình 3. 17 Giao diện trang thanh toán



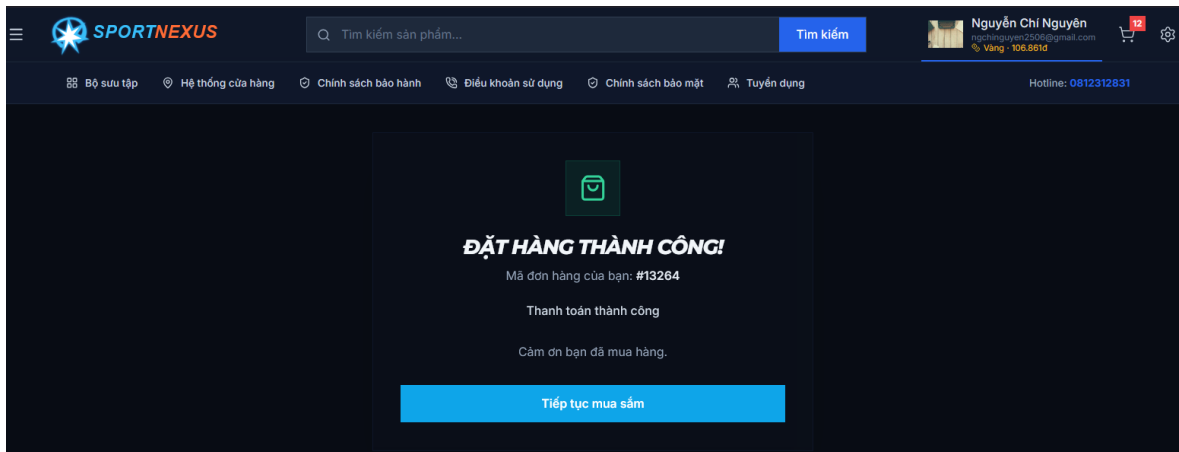
Hình 3. 18 Giao diện QR thanh toán của payOS khi chọn thanh toán qua app ngân hàng



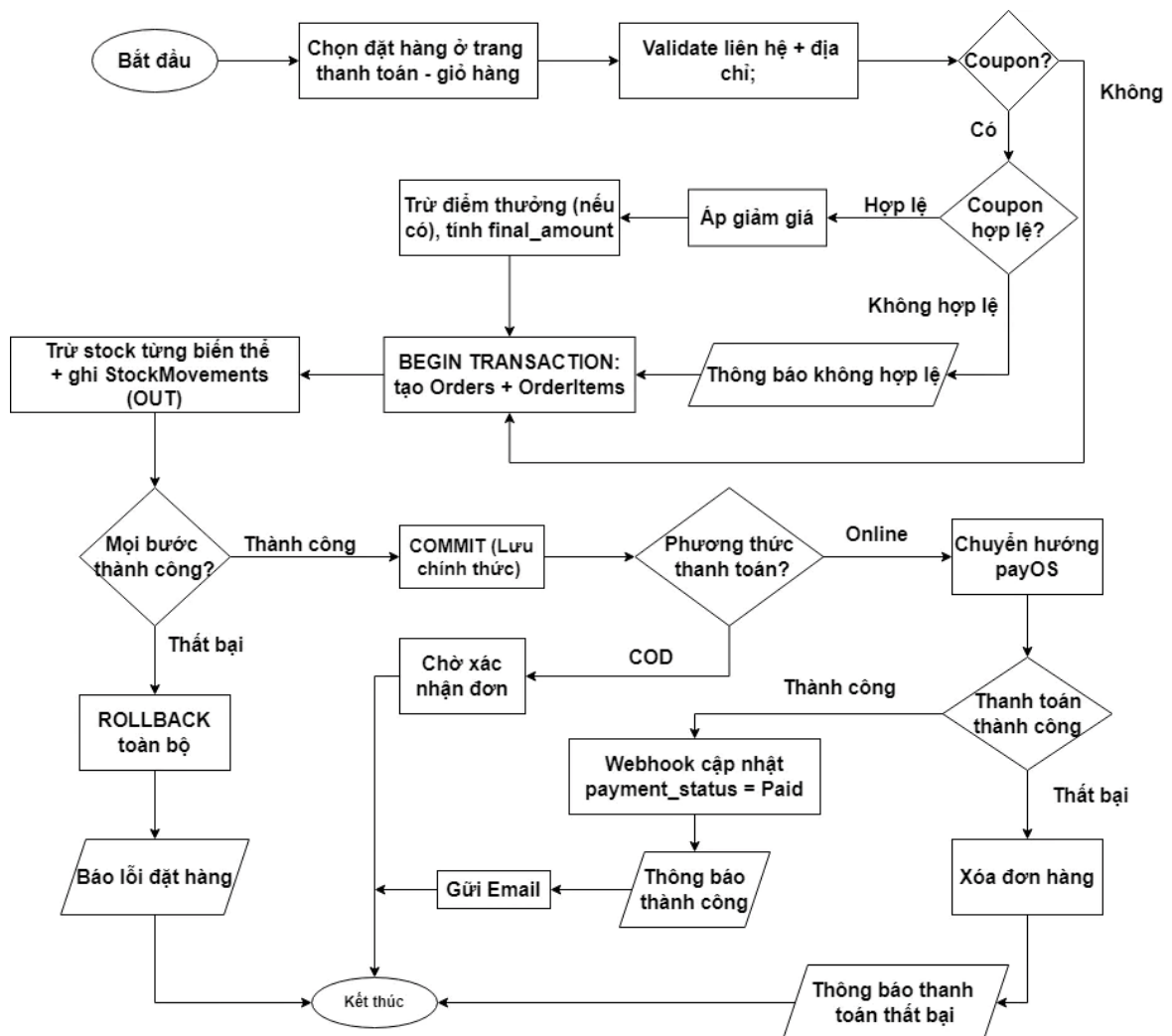
Hình 3. 19 Giao diện QR thanh toán của payOS khi chọn thanh toán qua app ngân hàng (mobile)



Hình 3. 20 Thông tin đơn hàng sẽ được tự động điện sẵn



Hình 3. 21 Giao diện khi thanh toán thành công



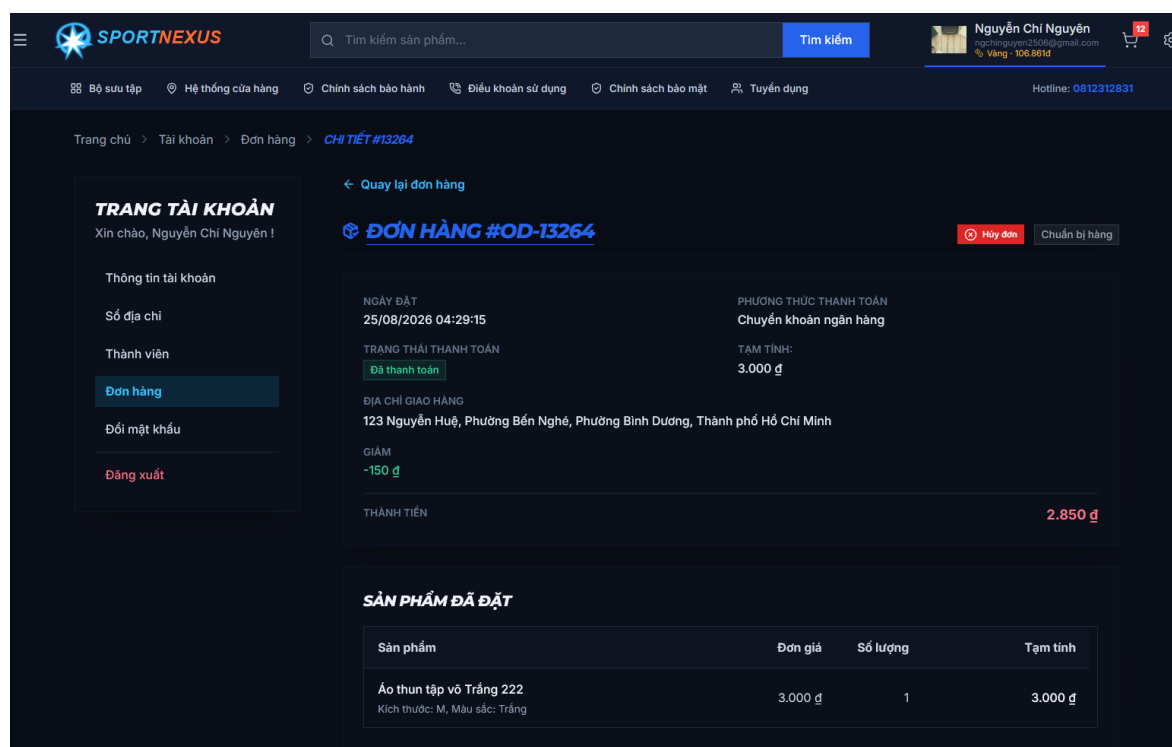
Hình 3. 22 – Lưu đồ đặt hàng và thanh toán

Sau khi validate thông tin và tính phí ship động, hệ thống áp coupon/điểm thưởng rồi mở transaction: tạo đơn + chi tiết, trừ tồn kho kèm ghi biến động OUT, phát hành hóa đơn và tạo vận đơn. Bất kỳ bước nào lỗi là ROLLBACK toàn bộ; thành công thì COMMIT và rẽ

nhánh theo phương thức — COD về OrderSuccess, online chuyển hướng cổng PayOS và chờ webhook cập nhật payment_status.

3.17.6 Theo dõi đơn hàng

Khách đã đăng nhập xem toàn bộ đơn hàng của mình tại mục tài khoản, mỗi đơn hiển thị trạng thái (Processing → Shipping → Delivered, hoặc Cancelled/Refunded) và chi tiết các dòng hàng, số tiền, phương thức thanh toán.



Hình 3. 23 Giao diện trang theo dõi đơn hàng

3.17.7 Đăng ký và đăng nhập

Hệ thống cung cấp hai chức năng xác thực cơ bản, xây dựng trên nền tảng JWT với cặp access/refresh token như trình bày ở Chương II.

Đăng ký yêu cầu người dùng khai báo họ tên, email, mật khẩu (có thể bổ sung số điện thoại). Trong quá trình xử lý, hệ thống kiểm tra tính duy nhất của email và số điện thoại — nếu trùng với tài khoản đã tồn tại sẽ báo lỗi và yêu cầu nhập lại. Sau khi tạo tài khoản, hệ thống gửi email chứa **link xác minh** (sinh token ngẫu nhiên lưu vào cơ sở dữ liệu); việc xác minh email là bắt buộc để hạn chế tài khoản ảo. Mật khẩu được mã hóa bằng **bcrypt** trước khi lưu. **Đăng nhập** chấp nhận email hoặc số điện thoại làm tên tài khoản cùng mật khẩu; tài khoản bị khóa (*status = false*) sẽ bị từ chối. Xác thực thành công, server trả về cặp token: access token đính kèm header 'Authorization' của mọi request cần xác thực, refresh token dùng để gia hạn phiên mà không bắt người dùng đăng nhập lại; đăng xuất sẽ hủy refresh token phía máy chủ. Ngoài ra, hệ thống tích hợp **đăng nhập xã hội** bằng

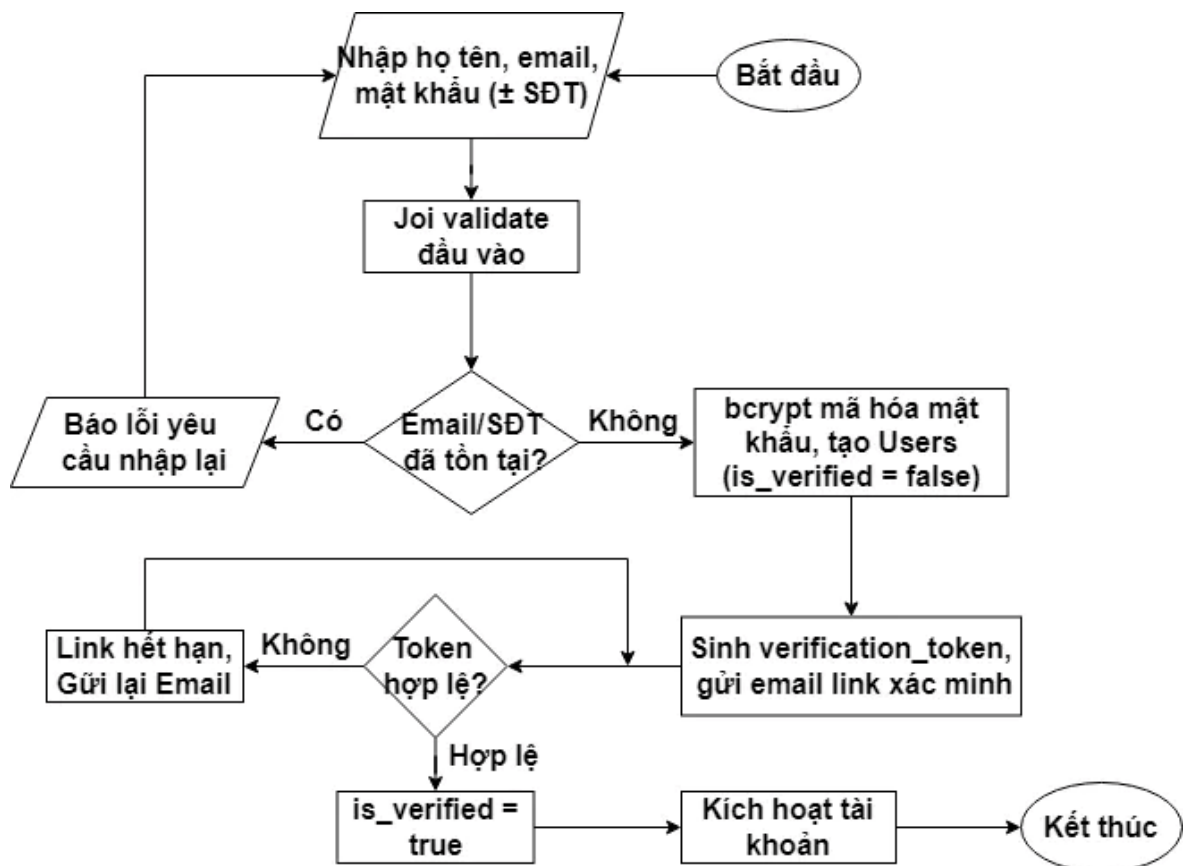
Google/Facebook — tài khoản mới qua OAuth được tự động khởi tạo với vai trò khách hàng — và chức năng **quên mật khẩu** gửi email chứa link đặt lại.

The login page features a dark blue background with a logo at the top. Below the logo is the title "ĐĂNG NHẬP". There are two input fields: "EMAIL / SỐ ĐIỆN THOẠI" and "MẬT KHẨU". A link "Quên mật khẩu?" is located below the password field. A large blue button "ĐĂNG NHẬP NGAY" is centered below the fields. Below this button is the text "HOẶC". At the bottom, there are two buttons for social media login: "FACEBOOK" and "GOOGLE". At the very bottom, it says "Chưa có tài khoản? Đăng ký ngay".

Hình 3. 24 Giao diện trang đăng nhập

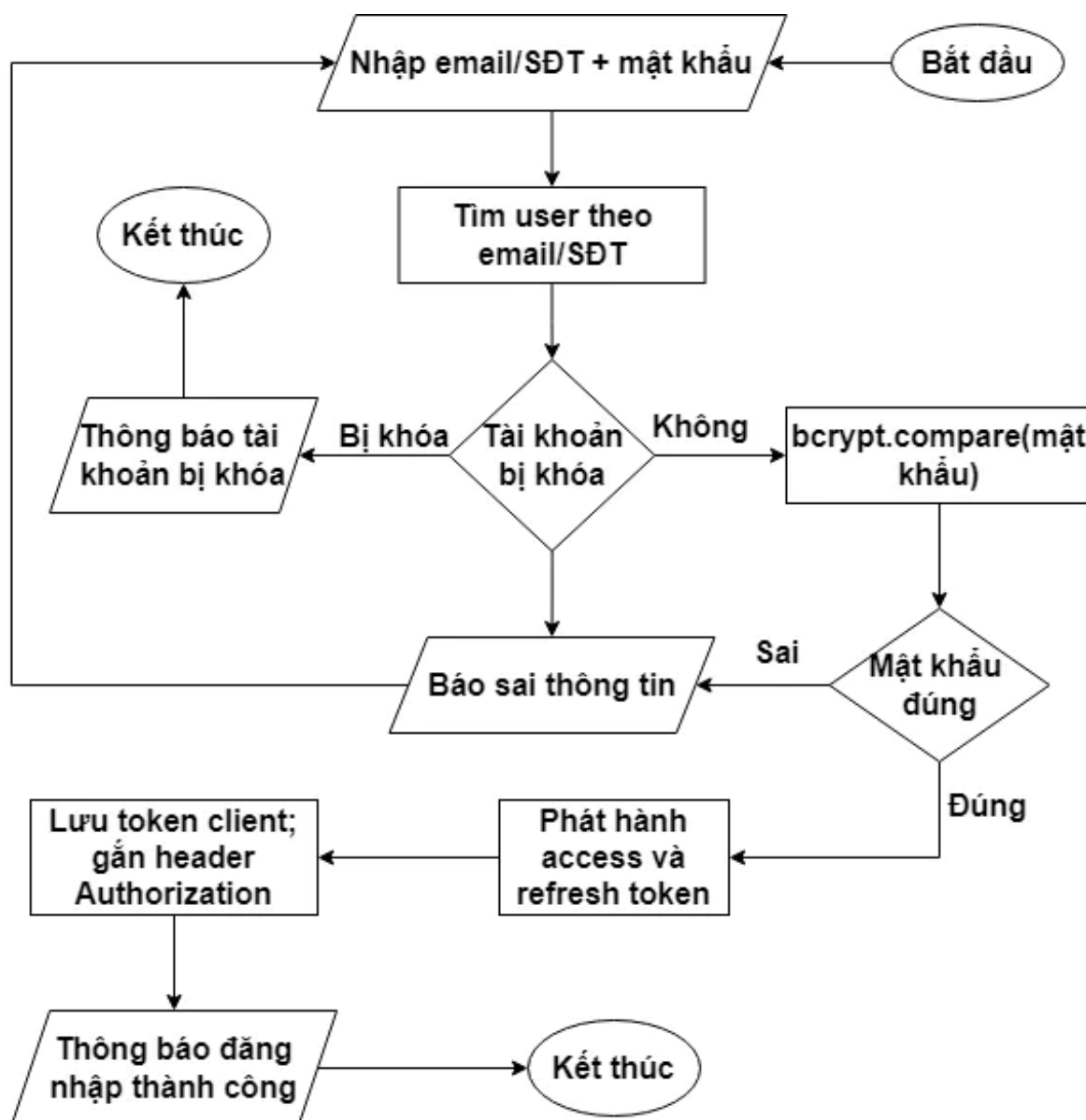
The registration page has a dark blue background with the same logo. The title is "ĐĂNG KÝ". There are four input fields: "HỌ VÀ TÊN", "ĐỊA CHỈ EMAIL", "MẬT KHẨU", and "SỐ ĐIỆN THOẠI". A large blue button "ĐĂNG KÝ NGAY" is centered below the fields. At the bottom, it says "Đã có tài khoản? Đăng nhập".

Hình 3. 25 Giao diện trang đăng ký



Hình 3. 26 – Lưu đồ đăng ký tài khoản

Đăng ký yêu cầu email/SĐT duy nhất; mật khẩu mã hóa bcrypt; tài khoản chỉ được kích hoạt sau khi bấm link xác minh hợp lệ từ email.



Hình 3. 27 – Lưu đồ đăng nhập

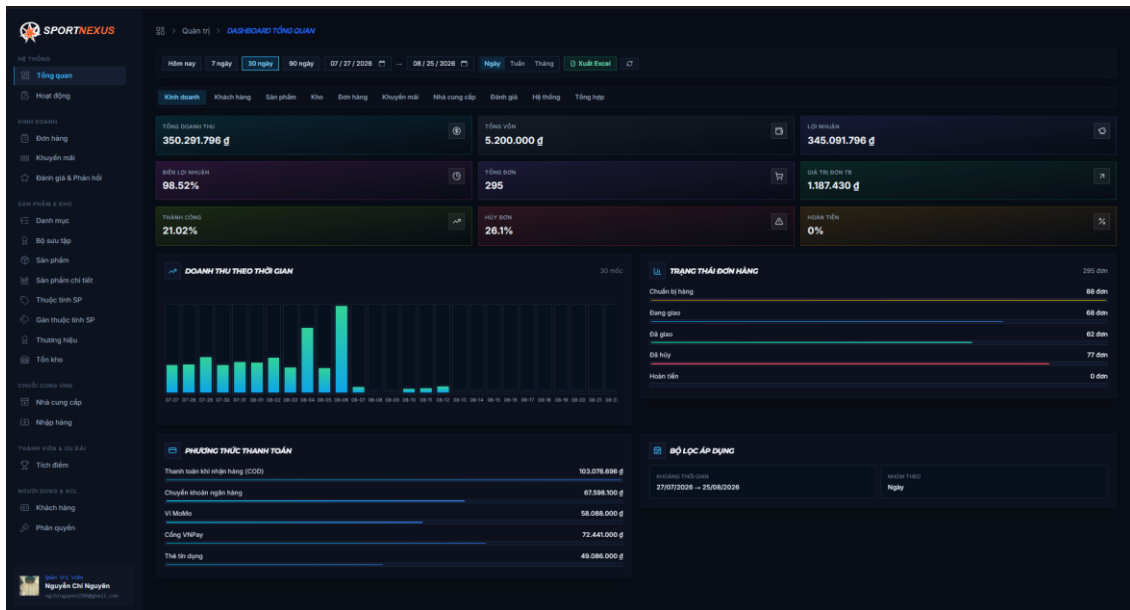
Đăng nhập chấp nhận email hoặc số điện thoại; từ chối tài khoản bị khóa; so khớp mật khẩu bằng bcrypt, thành công thì phát hành cặp access/refresh token gắn vào các request tiếp theo.

3.18 Giao diện và chức năng (Quản trị viên)

Phản quản trị được đặt tại đường dẫn `/management`, bảo vệ bởi lớp kiểm tra đăng nhập và phân quyền trước khi vào bất kỳ trang nào. Giao diện dùng chung khung layout riêng (sidebar danh mục chức năng, thanh trên hiển thị thông tin người dùng), mỗi nhóm nghiệp vụ là một trang độc lập được tải lười (lazy loading) kèm loader nạp dữ liệu trước khi render.

3.18.1 Dashboard tổng kê

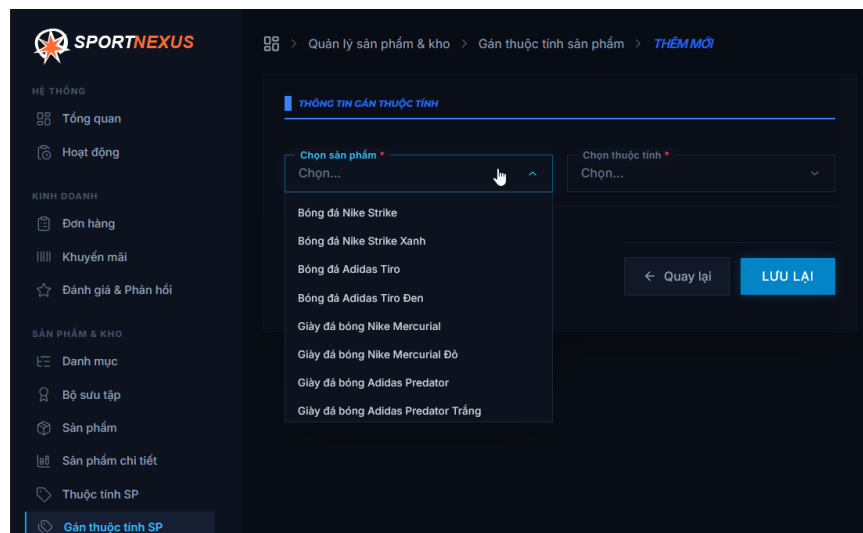
Trang đầu tiên sau khi đăng nhập, tổng hợp số liệu kinh doanh theo thời gian thực: doanh thu, số đơn hàng theo trạng thái (*đang xử lý, đang giao, đã giao, đã hủy*), tình trạng thanh toán, tổng giá trị tồn kho và xu hướng biến động kho nhập/xuất/điều chỉnh theo tháng; kèm biểu đồ trực quan và bộ chỉ số nhanh phục vụ ra quyết định.



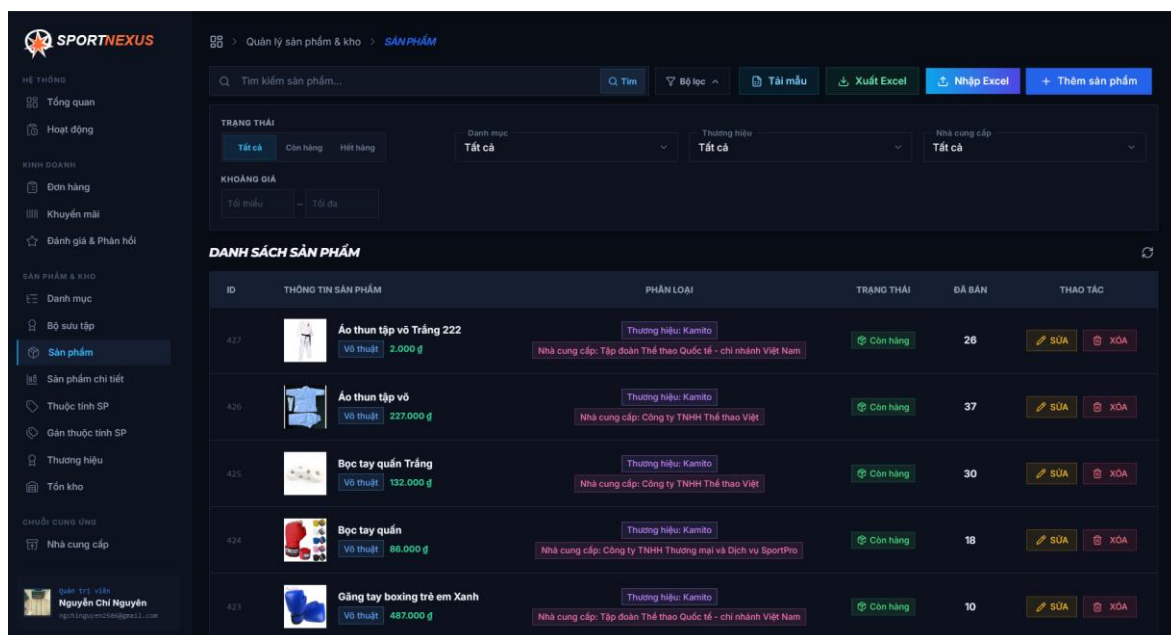
Hình 3. 28 Giao diện trang dashboard tổng kê kinh doanh

3.18.2 Quản lý sản phẩm, biến thể và thuộc tính

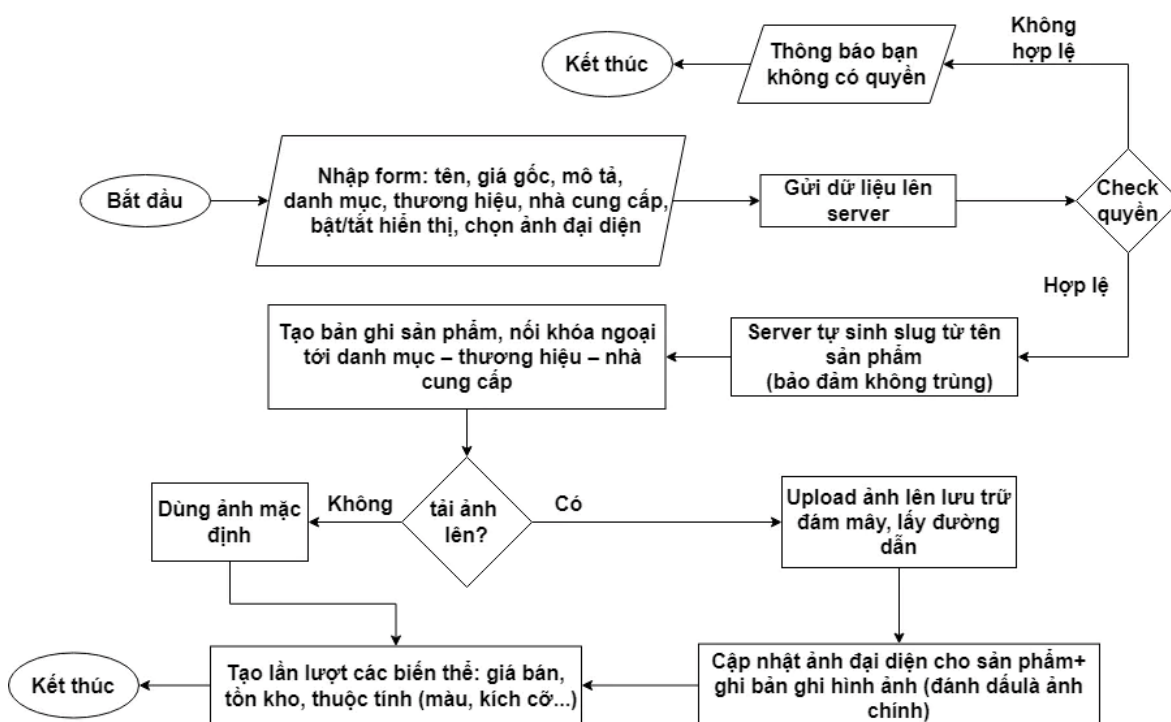
Cho phép thêm mới, sửa, xóa mềm sản phẩm; mỗi sản phẩm khai báo nhiều biến thể (*màu, kích cỡ*) với giá và tồn kho riêng, gắn thuộc tính theo cặp khóa – giá trị và tải hình ảnh lên lưu trữ đám mây. Danh sách hỗ trợ tìm kiếm, lọc theo danh mục/thương hiệu/nhà cung cấp/khoảng giá và xuất Excel. Trang thuộc tính cho phép định nghĩa các khóa thuộc tính (*kích cỡ, màu sắc...*) dùng chung cho toàn hệ thống lọc phía khách hàng.



Hình 3. 29 form thêm biến thể cho sản phẩm



Hình 3. 30 Giao diện trang quản lý sản phẩm

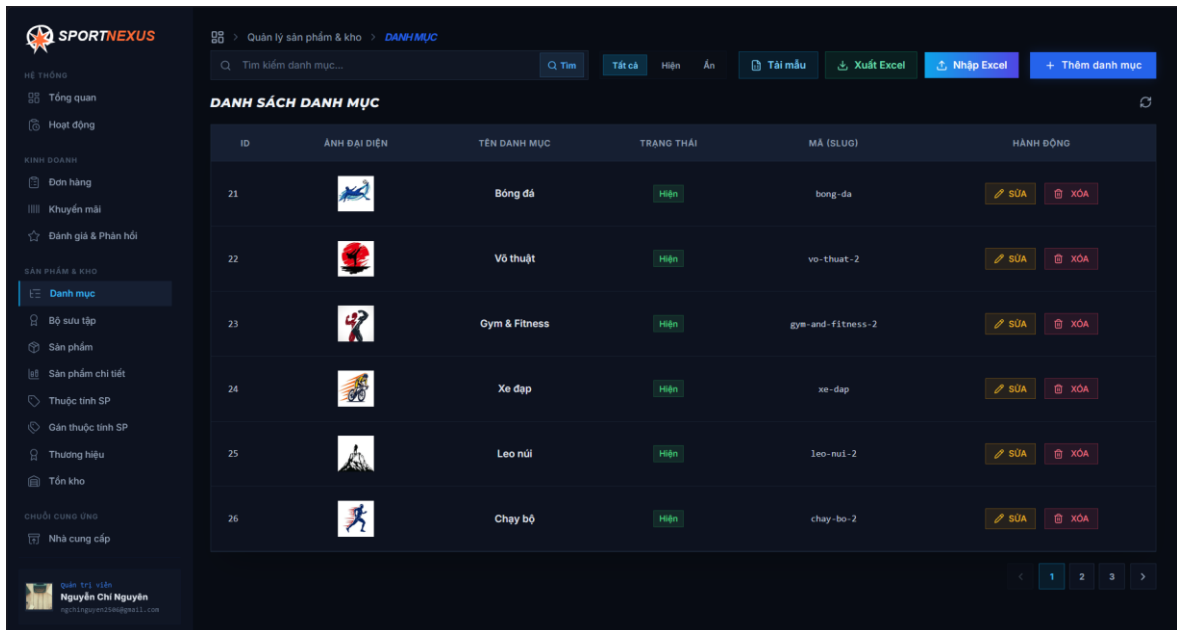


Hình 3. 31 – Lưu đồ thêm sản phẩm mới

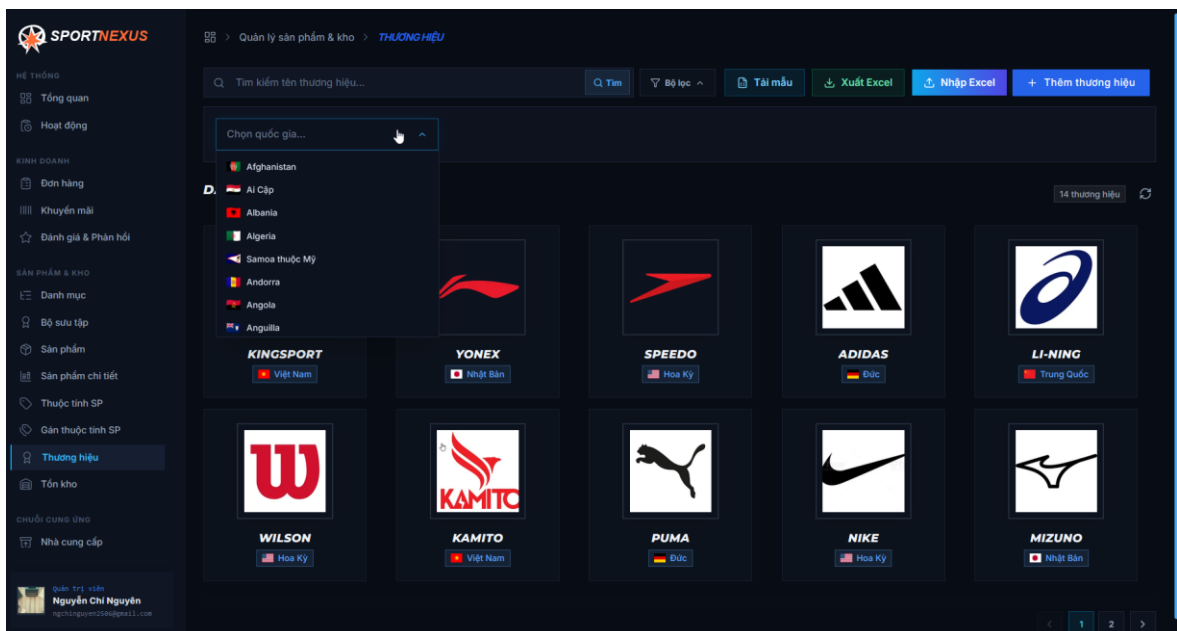
Mã định danh đường dẫn (*slug*) do server tự phát sinh từ tên sản phẩm và bảo đảm duy nhất, quản trị viên không cần tự nhập. Nếu có tệp ảnh đính kèm, hệ thống tải ảnh lên bộ lưu trữ đám mây rồi mới cập nhật đường dẫn và đánh dấu đó là ảnh chính. Sản phẩm tạo xong chưa có hàng bán được ngay — phải tiếp tục tạo ít nhất một biến thể kèm giá và tồn kho; số lượng biến thể không giới hạn, mỗi biến thể mang tổ hợp thuộc tính riêng để phía khách hàng chọn mua.

3.18.3 Quản lý danh mục, thương hiệu, bộ sưu tập và nhà cung cấp

Bốn danh mục dữ liệu nền có quy trình thống nhất: danh sách phân trang, tạo mới, chỉnh sửa, xóa mềm. Bộ sưu tập (*collection*) cho phép nhóm sản phẩm theo chủ đề để hiển thị banner riêng trên website khách hàng; nhà cung cấp lưu thông tin liên hệ và địa chỉ phục vụ đơn nhập hàng.



Hình 3. 32 Giao diện quản lý danh mục



Hình 3. 33 Giao diện quản lý thương hiệu

3.18.4 Đơn nhập hàng và quản lý kho

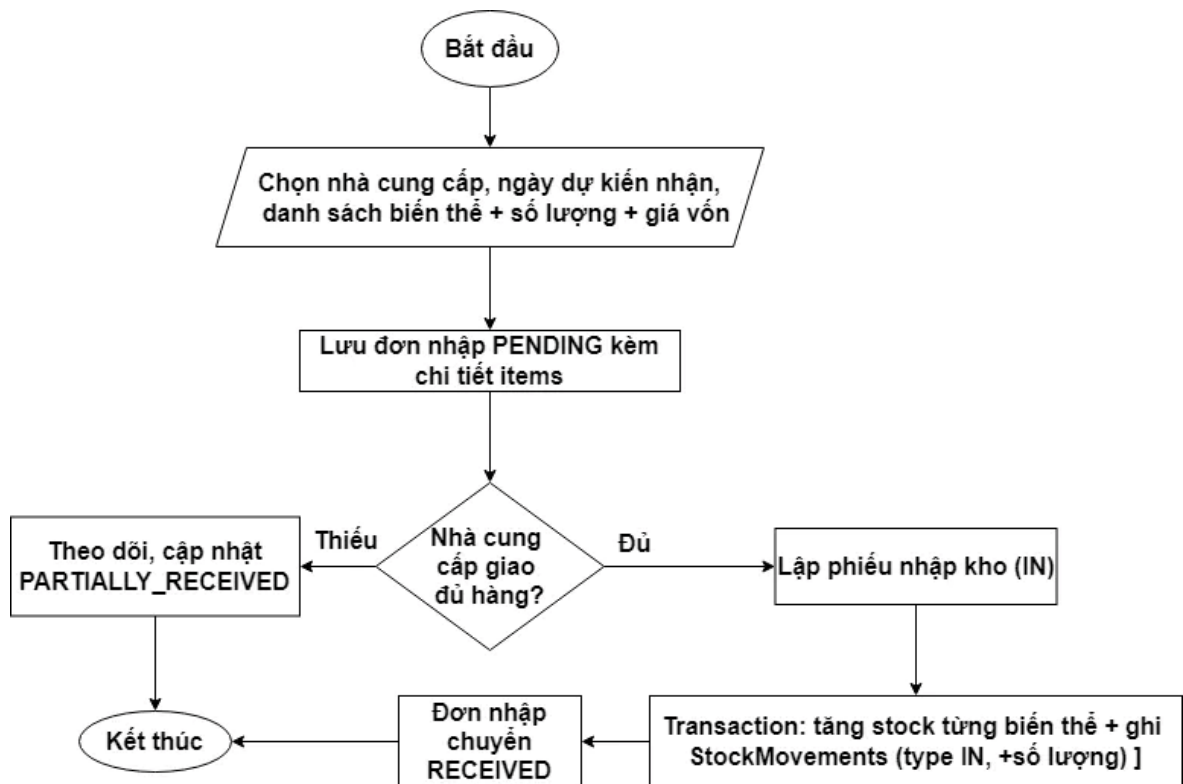
Đơn nhập hàng (*purchase order*) ghi nhận việc mua hàng từ nhà cung cấp: chọn nhà cung cấp, ngày dự kiến nhận, danh sách biến thể – số lượng – giá vốn; trạng thái lần lượt là

PENDING → RECEIVED (*hoặc CANCELLED*). Khi nhận hàng, quản trị viên lập phiếu nhập kho: hệ thống tăng tồn kho từng biến thể, ghi lịch sử biến động IN và chuyển đơn nhập sang RECEIVED trong cùng một transaction. Trang kho hiển thị tồn hiện tại của mọi biến thể và lịch sử biến động IN/OUT/ADJUSTMENT; phiếu xuất kho phục vụ giao hàng sẽ trình bày ở lưu đồ bên dưới.

Hình 3. 34 Giao diện trang nhập hàng

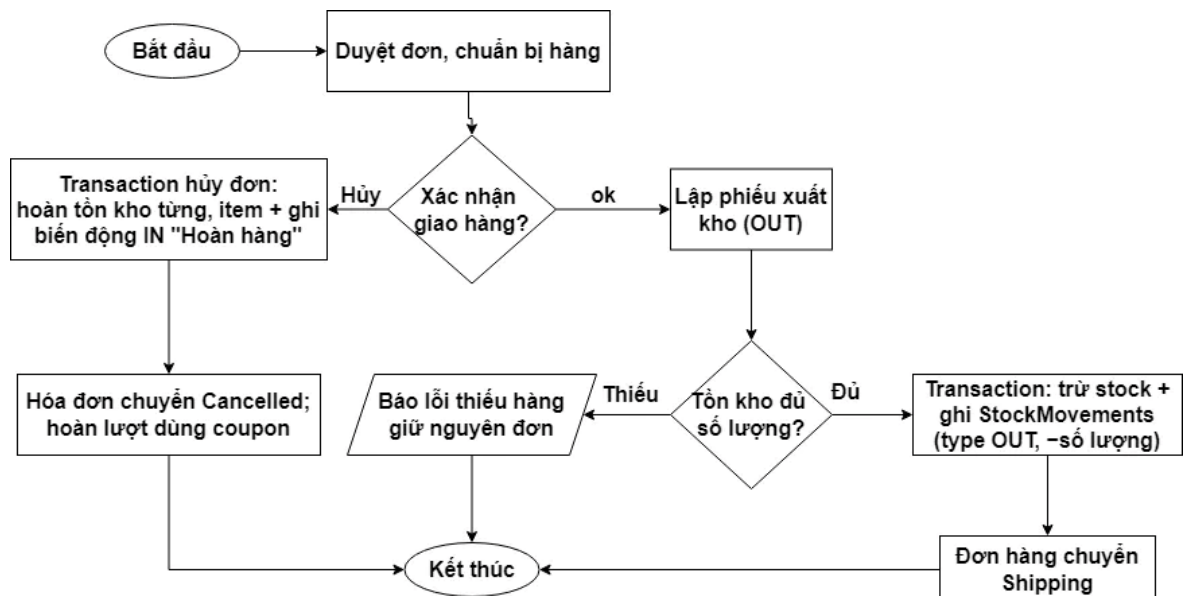
MÃ ĐỊNH DANH	SẢN PHẨM	THUỘC TÍNH	SỐ LƯỢNG TỒN
#VAR-885	Áo thun tập võ Trắng 222 Giá gốc: 3,000 đ	KÍCH THƯỚC: M - cm MÀU SẮC: Trắng -	66
#VAR-884	Áo thun tập võ Giá gốc: 230,000 đ	KÍCH THƯỚC: XL - cm MÀU SẮC: Trắng -	111
#VAR-883	Áo thun tập võ Giá gốc: 239,000 đ	KÍCH THƯỚC: L - cm MÀU SẮC: Trắng -	62
#VAR-882	Áo thun tập võ Giá gốc: 227,000 đ	KÍCH THƯỚC: M - cm MÀU SẮC: Trắng -	104
#VAR-881	Bọc tay quần Trắng Giá gốc: 138,000 đ	MÀU SẮC: Đỏ -	46

Hình 3. 35 Giao diện trang quản lý kho



Hình 3. 36 – Lưu đồ nhận hàng - nhập kho

Toàn bộ bước tăng tồn kho và ghi lịch sử biến động diễn ra trong một transaction: nếu một dòng hàng lỗi thì cả phiếu nhập bị quay lui, tồn kho không bị tăng dở dang. Khi có kèm mã đơn nhập, trạng thái đơn tự động chuyển sang RECEIVED ngay sau khi phiếu được ghi nhận.



Hình 3. 37 – Lưu đồ xuất kho, giao hàng và hủy đơn

Nhánh giao hàng: phiếu xuất kho chạy transaction kiểm tra tồn kho trước khi trừ — thiếu hàng là báo lỗi và không làm thay đổi dữ liệu; đủ hàng thì tồn kho giảm, lịch sử biến động OUT được ghi và đơn hàng tự chuyển sang Shipping. Nhánh hủy chỉ áp dụng cho đơn còn ở Processing: hệ thống hoàn tồn kho, ghi biến động IN hoàn hàng, hủy hóa đơn và hoàn lại lượt sử dụng mã giảm giá; nếu đơn đã thanh toán online thì trạng thái thanh toán chuyển sang hoàn tiền.

3.18.5 Xử lý đơn hàng và hóa đơn

Quản trị viên duyệt toàn bộ đơn hàng của khách tại một màn hình thống nhất: xem chi tiết từng đơn, lọc theo trạng thái – thời gian, cập nhật trạng thái xử lý – giao hàng và hủy đơn khi có yêu cầu. Hóa đơn không do quản trị viên lập tay mà được hệ thống sinh tự động ngay trong giao dịch đặt hàng; trạng thái hóa đơn luôn đồng bộ với đơn hàng (*đơn bị hủy thì hóa đơn chuyển sang đã hủy*), còn việc tra cứu và in hóa đơn thuộc về khách hàng tại trang cá nhân của họ.

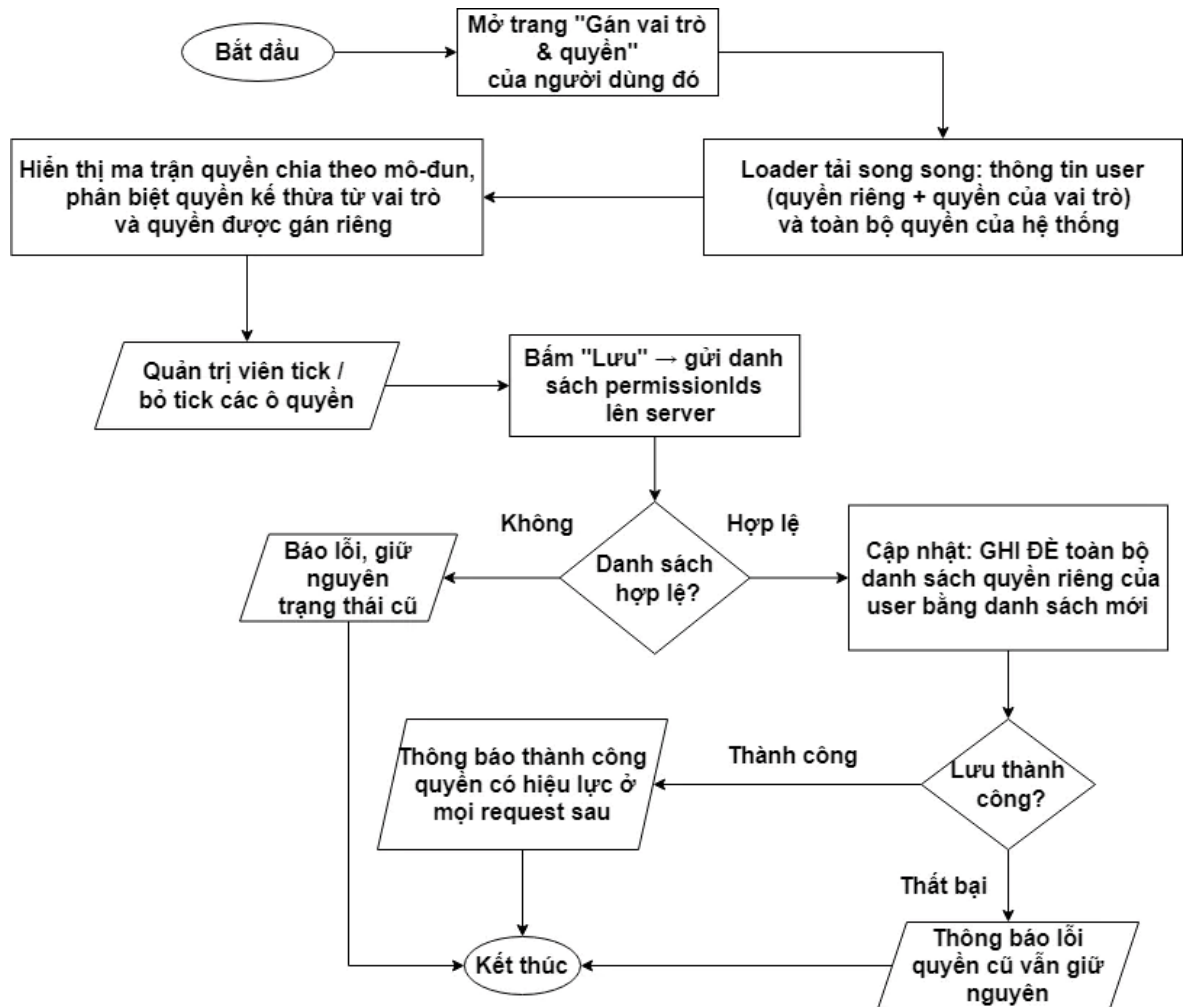
The screenshot shows the SPORTNEXUS order management system. The interface is in Vietnamese. On the left is a sidebar with navigation links: 'Hệ thống', 'Tổng quan', 'Hoạt động', 'KINH DOANH' (with sub-links like 'Đơn hàng', 'Khuyến mãi', 'Đánh giá & Phản hồi'), 'SẢN PHẨM & KHO' (with sub-links like 'Danh mục', 'Bộ sưu tập', 'Sản phẩm', 'Sản phẩm chi tiết'), 'Thuộc tính SP', 'Gán thuộc tính SP', 'Thương hiệu', 'Tồn kho', and 'CHUỖ CUNG ỨNG' (with sub-link 'Nhà cung cấp'). The main area is titled 'Quản lý kinh doanh > ĐƠN HÀNG'. It features a search bar and several filter buttons: 'Tìm mã đơn hàng, email khách hàng...', 'Q. Tìm', 'Bộ lọc', 'Tải mẫu', 'Xuất Excel', 'Nhập Excel', and '+ Tạo đơn hàng'. Below these are dropdown menus for 'Vận chuyển', 'Thanh toán', 'Phương thức', and 'Trạng thái'. There are also input fields for 'TỪ NGÀY' and 'ĐẾN NGÀY'. The main table, titled 'DANH SÁCH ĐƠN HÀNG', has columns: 'THÔNG TIN KHÁCH HÀNG', 'GIÁ TRỊ ĐƠN', 'TRẠNG THÁI & THỜI GIAN', and 'THAO TÁC'. It displays a list of orders for a customer named 'ngchinguyen2506@gmail.com'. Each order entry shows the email, address, order value (e.g., 'Gốc 2.000 đ', 'Giảm 150 đ', 'Thực thu 2.850 đ'), status (e.g., 'Lịch sử', 'Vận chuyển', 'CHUẨN BỊ HÀNG'), and a 'Thanh toán' button. The bottom of the screen shows the user's profile: 'Quản trị viên Nguyễn Chi Nguyễn'.

Hình 3. 38 Giao diện trang quản lý đơn hàng

3.18.6 Quản lý tài khoản, vai trò và phân quyền

Hình 3. 39 Giao diện trang quản lý người dùng

Hình 3. 40 Giao diện trang cấp quyền



Hình 3. 41 – Lưu đồ gán vai trò và quyền cho người dùng

Trang người dùng cho phép tra cứu, khóa/mở tài khoản, thêm mới và gán vai trò kèm quyền chi tiết cho từng nhân sự. Hệ thống vận hành theo mô hình RBAC: mỗi quyền là một bản ghi riêng, vai trò gom nhiều quyền, người dùng có thể mang nhiều vai trò; mọi route nhảy cảm đều kiểm tra quyền tương ứng trước khi thực thi. Trang nhật ký hệ thống (*logs*) lưu vết thao tác của người dùng nội bộ phục vụ truy cứu.

3.18.7 Khuyến mãi, chương trình thành viên và đánh giá

Quản trị viên cấu hình mã giảm giá (*thời gian chạy, giá trị tối thiểu, giới hạn lượt dùng*), quản lý chương trình thành viên: hạng khách hàng, quy tắc tích điểm, phần thưởng đổi điểm và điều chỉnh điểm từng hội viên. Đánh giá sản phẩm từ khách được duyệt tại đây: ẩn/hiện đánh giá vi phạm và phản hồi công khai dưới tên cửa hàng.

Mã giảm giá: Trang quản lý coupon cho phép tạo, chỉnh sửa và ẩn mã; mỗi mã khai báo loại giảm (*giảm cố định hoặc phần trăm*), giá trị giảm, trần giảm tối đa, giá trị đơn tối thiểu, thời hạn hiệu lực và tổng lượt dùng. Quản trị viên có thể tặng trực tiếp mã cho một

người dùng cụ thể (hệ thống ghi vào bảng `user\_coupons` với cờ `is\_gift`); khi khách lưu mã về ví, lượt dùng còn lại được kiểm tra theo cả `usage\_limit` toàn cục lẫn `max\_uses\_per\_user` trên từng tài khoản. Danh sách coupon hỗ trợ lọc theo trạng thái hoạt động/không, loại giảm giá và khoảng thời gian tạo.

The screenshot shows the 'THÊM MÃ GIẢM GIÁ MỚI' (Add New Discount Code) form in the SPORTNEXUS admin panel. The form is divided into several sections:

- Cấu hình coupon giá** (Price coupon configuration): Includes fields for 'Loại giảm giá' (Discount type), 'Giá trị giảm' (Discount value), and 'Mức giảm tối đa' (Maximum discount).
- KHUÔNG THỜI GIẠN** (Expiry date): Includes fields for 'Ngày bắt đầu' (Start date) and 'Ngày kết thúc' (End date).
- ĐIỀU KIỆN SỬ DỤNG** (Usage conditions): Includes fields for 'Đơn hàng tối thiểu' (Minimum order), 'Giới hạn lượt dùng chung' (Common usage limit), and 'Số lần dùng tối đa / khách' (Maximum usage per customer).
- THÔNG TIN MÃ** (Code information): Includes a field for 'Mã giảm giá (Code)' and checkboxes for 'NGỪNG HOẠT ĐỘNG' (Deactivate) and 'HIỂN THỊ TRÊN WEB' (Show on website).

At the bottom right, there are buttons for 'Quay lại' (Go back) and 'LƯU LẠI' (Save).

Hình 3. 42 Giao diện trang thêm khuyến mãi

The screenshot shows the 'DANH SÁCH MÃ GIẢM GIÁ' (Discount Code List) table in the SPORTNEXUS admin panel. The table has the following columns: ID, MÃ & HÌNH THỨC (Code & Format), THỜI GIẠN HIỆU LỰC (Expiry date), HẠN MỨC ĐƠN HÀNG (Usage limit), DÃ DÙNG / GIỚI HẠN (Used / Limit), and THAO TÁC (Actions). The table lists several discount codes, including SEED0006, SEED0100, SEED0005, SEED0004, SEED0001, and SEED0002. A modal window is open for the 'TẶNG MÃ GIẢM GIÁ: SEED0006' (Gift discount code: SEED0006) action, showing a search bar and a list of users to gift the code to.

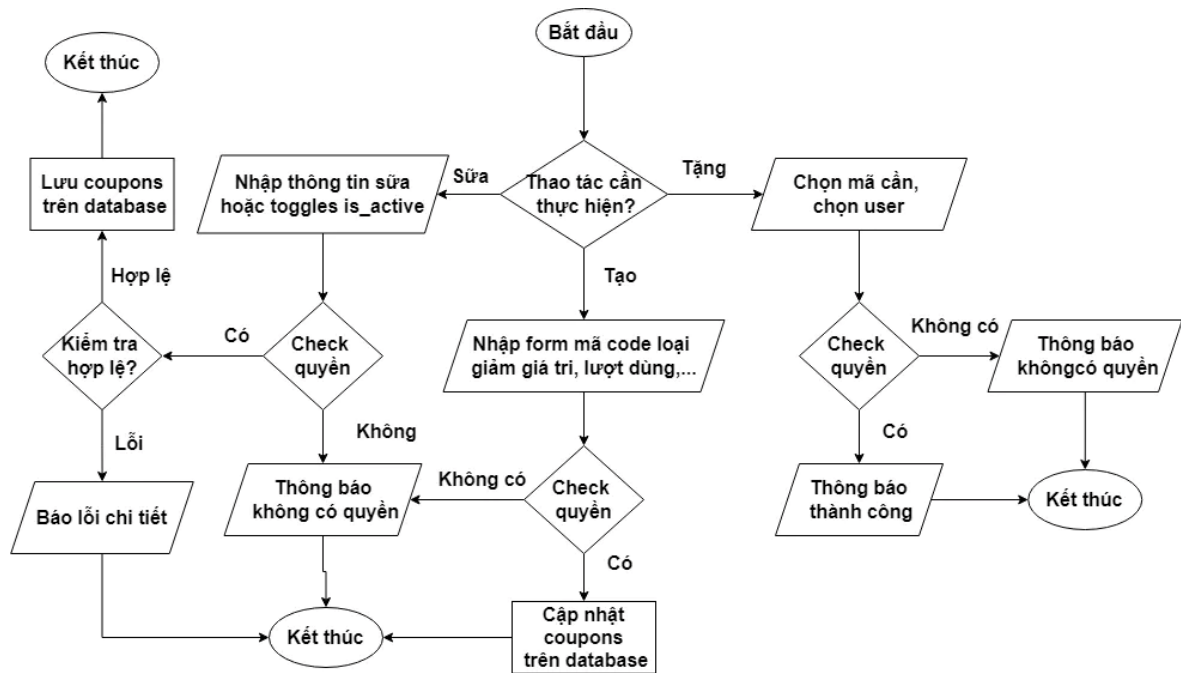
ID	MÃ & HÌNH THỨC	THỜI GIẠN HIỆU LỰC	HẠN MỨC ĐƠN HÀNG	DÃ DÙNG / GIỚI HẠN	THAO TÁC
113	Code: SEED0006 Tiền mặt 682.667 đ	Từ: 01/01/2025 Đến: 31/03/2025	Đơn tối thiểu: 384.807 đ	0 / 741	Tặng, Sửa, Xóa
113	Code: SEED0100 Tiền mặt 549.798 đ	Từ: ... Đến: ...			Tặng, Sửa, Xóa
118	Code: SEED0005 Tiền mặt 324.626 đ	Từ: ... Đến: ...			Tặng, Sửa, Xóa
117	Code: SEED0004 Tiền mặt 13.188 đ	Từ: ... Đến: ...			Tặng, Sửa, Xóa
114	Code: SEED0001 Tiền mặt 399.498 đ	Từ: ... Đến: ...			Tặng, Sửa, Xóa
115	Code: SEED0002 Tiền mặt 302.384 đ	Từ: ... Đến: ...			Tặng, Sửa, Xóa

The modal window for 'TẶNG MÃ GIẢM GIÁ: SEED0006' shows a search bar and a list of users to gift the code to:

- Nguyễn Chí Nguyễn (fb_3065376623105808@facebook.com)
- Đặng Huy Lan (customer10002@gmail.com)
- Vũ Hoàng Bằng (customer9997@gmail.com)
- Mai Hoàng My

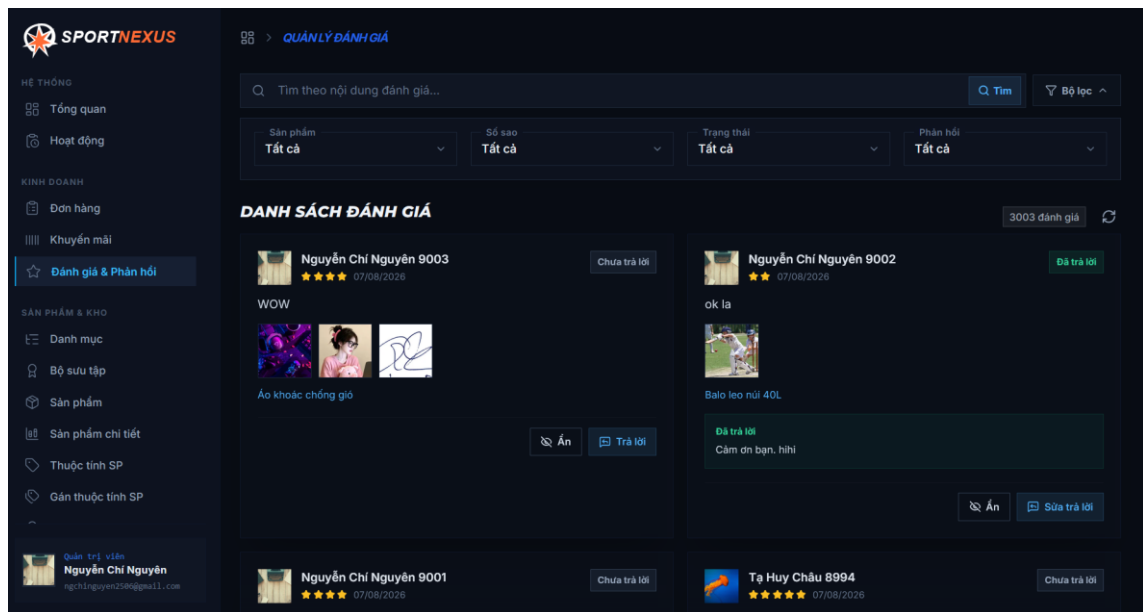
At the bottom of the modal, there is a button 'Gửi tặng' (Gift).

Hình 3. 43 Giao diện trang tặng khuyến mãi

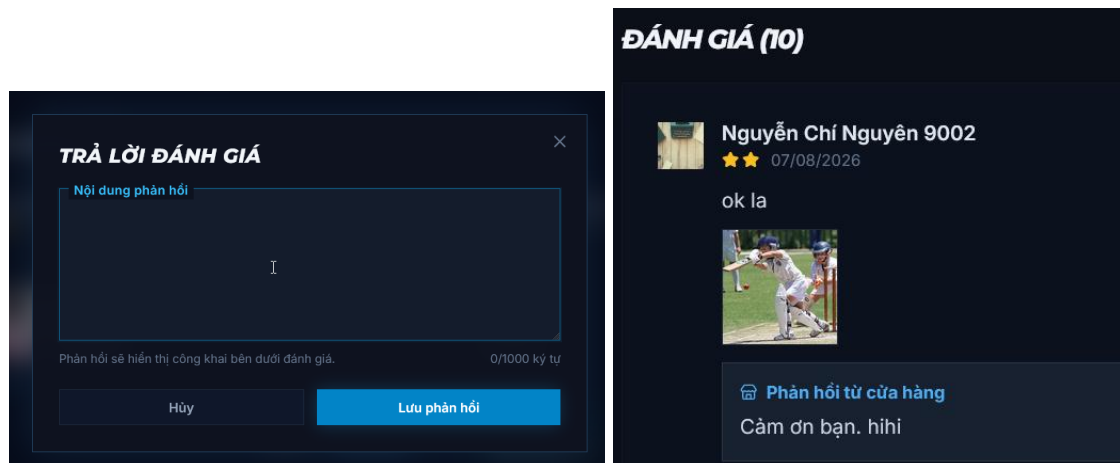


Hình 3. 44 – Lưu đồ tạo và tặng mã giảm giá

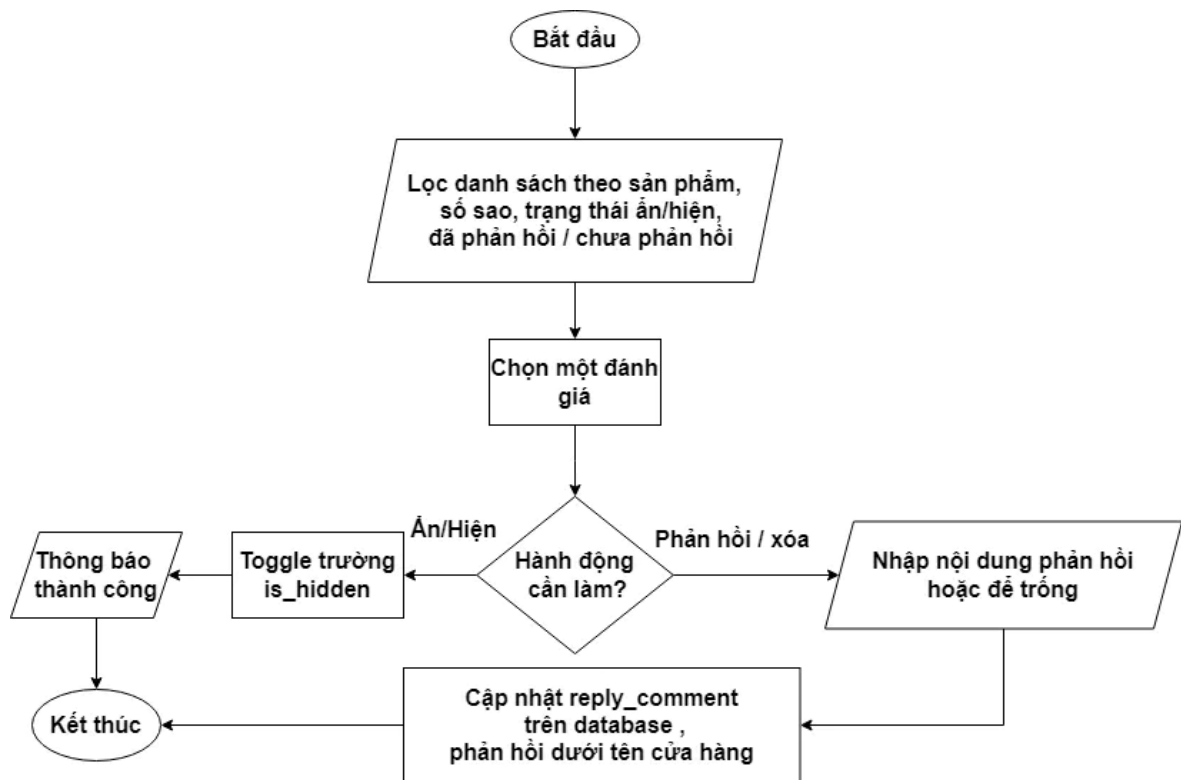
Đánh giá: Trang đánh giá (`/management/reviews`) hiển thị toàn bộ đánh giá từ khách trên mọi sản phẩm, hỗ trợ bộ lọc theo từ khóa tên sản phẩm, số sao, trạng thái ẩn/hiện và đã phản hồi hay chưa. Mỗi dòng hiển thị tên người đánh giá, số sao, nội dung, ảnh minh chứng (*nếu có*) và trạng thái hiện tại; đánh giá mới từ khách mặc định ở trạng thái ẩn (`is_hidden = true`) cho đến khi quản trị viên duyệt hiện lên website, cơ chế này giúp kiểm duyệt nội dung trước khi công khai. Quản trị viên có hai hành động chính: *ẩn/hiệ** đánh giá bằng cách toggle trường `is_hidden` — thay đổi có hiệu lực tức thì trên website khách hàng; và *phản hồi* dưới tên cửa hàng bằng cách nhập nội dung vào trường `reply_comment`, giúp khách khác thấy sự phản hồi chuyên nghiệp từ shop. Nếu phản hồi cần chỉnh sửa hoặc xóa, quản trị viên có thể cập nhật lại nội dung hoặc để trống để gỡ phản hồi.



Hình 3. 45 Giao diện trang quản lý đánh giá



Hình 3. 46 Giao diện model trả lời đánh giá



Hình 3. 47 – Lưu đồ duyệt và phản hồi đánh giá

Đánh giá mặc định ở trạng thái `is\_hidden = true` (ẩn) — chỉ hiện trên website sau khi quản trị viên chuyển sang `false`. Phản hồi (`reply\_comment`) được lưu riêng biệt, không ghi đè lên nội dung bình luận gốc của khách.

Chương trình thành viên: Quản trị quản lý bốn khía cạnh của chương trình khách hàng thân thiết tại các trang riêng biệt.

Thứ nhất, CRUD hạng thành viên (`membership\_tiers`): khai báo tên hạng, tổng chi tiêu tối thiểu để đạt hạng, tỷ lệ hoàn điểm trên mỗi giao dịch, phần trăm giảm giá độc quyền và thứ tự hiển thị; hạng không hoạt động có thể ẩn thay vì xóa để bảo toàn dữ liệu người dùng đang gắn.

Thứ hai, phần thưởng đổi điểm (`tier\_rewards`): mỗi hạng có bộ phần thưởng riêng gồm tên quà, số điểm cần đổi và mã giảm giá đi kèm (nếu phần thưởng là voucher); phần thưởng được gắn với đúng hạng nên khách hạng thấp không thấy phần thưởng hạng cao.

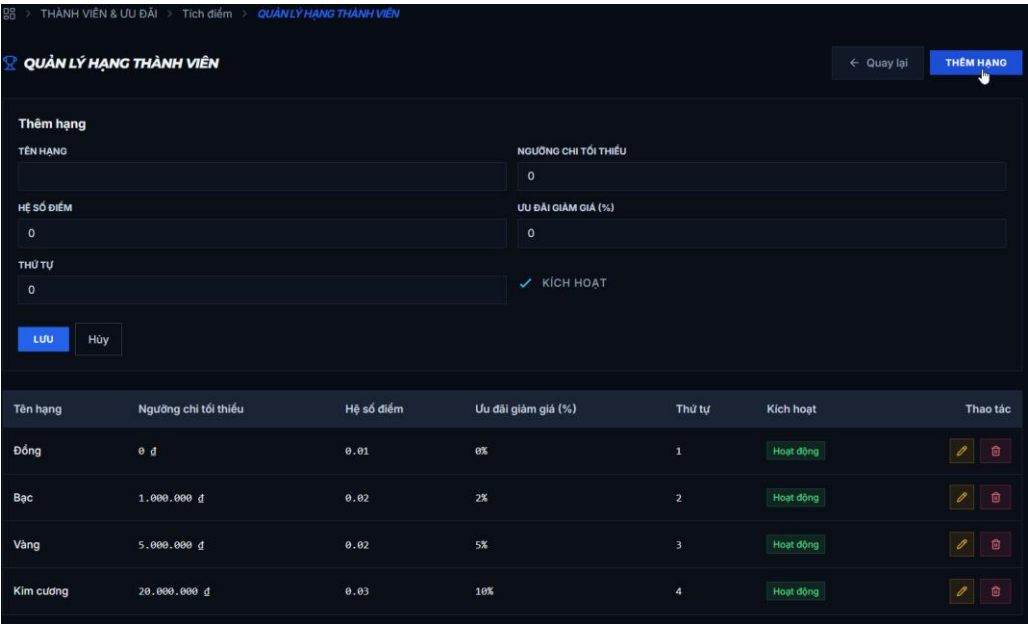
Thứ ba, trang quản lý hội viên (`loyalty/users`): hiển thị danh sách khách kèm hạng hiện tại, điểm tích lũy và tổng chi tiêu, cho phép tìm kiếm theo tên/email và lọc theo hạng; quản trị viên có thể nhấn vào một hội viên để xem lịch sử giao dịch điểm chi tiết.

Thứ tư, điều chỉnh điểm thủ công: nhập số điểm cộng hoặc trừ kèm lý do, hệ thống kiểm tra số dư không âm trước khi thực thi — toàn bộ thao tác chạy trong transaction, ghi

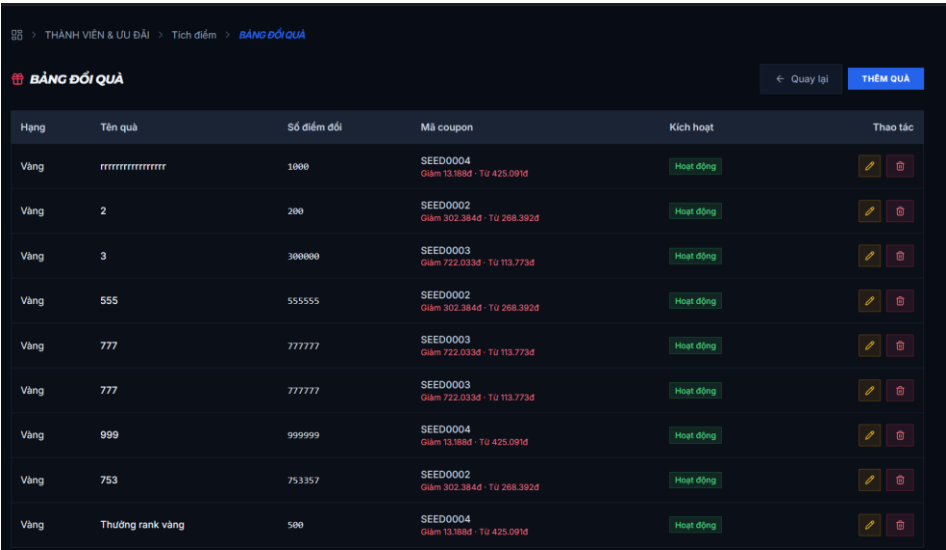
một bản ghi `point\_transactions` với số dư sau giao dịch (`balance\_after`), đảm bảo truy vết minh bạch từng bước.



Hình 3. 48 Giao diện menu tích điểm



Hình 3. 49 Giao diện trang quản lý hạng thành viên



Hình 3. 50 Giao diện trang quản lý đổi quà

THÀNH VIÊN & ƯU ĐÃI

Tích điểm

CẤU HÌNH

CẤU HÌNH

TỶ LỆ QUY ĐỔI (1 ĐIỂM = ? VND)

1000

Số VND quy đổi được từ 1 điểm

← Quay lại

LƯU

Hình 3. 51 Giao diện trang cấu hình điểm

THÀNH VIÊN & ƯU ĐÃI

Tích điểm

NGƯỜI DÙNG & ĐIỂM

← Quay lại

Tất cả hạng

Số dư điểm

Giảm dần

Q. Tìm người dùng...

Q. Tìm

Email	Tên	Hạng	Số dư điểm	Tổng chi	Thao tác
customer150@gmail.com	Lâm Mỹ Hồng	Kim cương	672750	22.425.000 đ	Xem chi tiết
customer786@gmail.com	Vũ Kim Duyên	Kim cương	616980	20.566.000 đ	Xem chi tiết
customer589@gmail.com	Đinh Tuấn Trí	Vàng	398320	19.916.000 đ	Xem chi tiết
customer509@gmail.com	Bùi Quốc Văn	Vàng	387200	19.360.000 đ	Xem chi tiết
customer2171@gmail.com	Phan Anh Lâm	Vàng	361860	18.093.000 đ	Xem chi tiết
customer216@gmail.com	Huỳnh Xuân Cường	Vàng	265040	13.252.000 đ	Xem chi tiết
customer2063@gmail.com	Dương Đình Cảnh	Vàng	210920	10.546.000 đ	Xem chi tiết
customer5361@gmail.com	Hà Bảo Lộc	Kim cương	204790	20.479.000 đ	Xem chi tiết
customer8843@gmail.com	Tô Thị Nga	Vàng	188400	9.420.000 đ	Xem chi tiết
customer827@gmail.com	Trần Thanh Phú	Vàng	179160	8.958.000 đ	Xem chi tiết

<

1

2

3

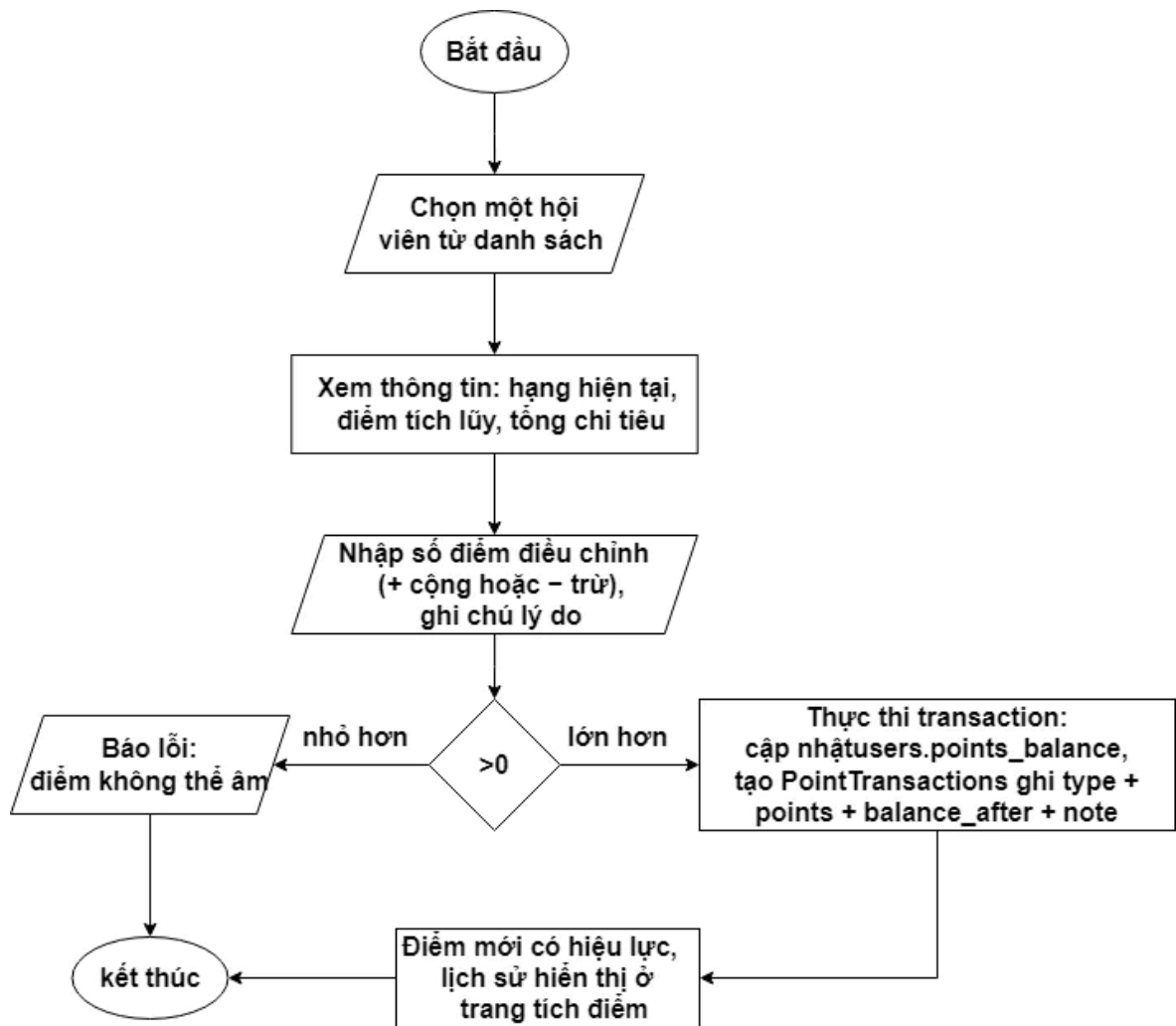
4

...

21

>

Hình 3. 52 Giao diện trang quản lý người dùng và điểm



Hình 3. 53 – Lưu đồ điều chỉnh điểm thành viên

Toàn bộ thao tác cộng/trừ điểm chạy trong một transaction: nếu số dư sau khi trừ bị âm thì giao dịch bị hủy, database giữ nguyên trạng thái cũ — cam kết điểm trong tài khoản không bao giờ bị sai lệch.

KẾT LUẬN – ĐÁNH GIÁ

Kết quả đạt được

Sau quá trình nghiên cứu và xây dựng, đề tài đã hoàn thành một hệ thống thương mại điện tử thể thao hoàn chỉnh gồm hai phần chính:

Website khách hàng:

- Trang chủ tổng hợp dữ liệu bằng một endpoint duy nhất, bên trong chạy song song 8 truy vấn (*sản phẩm mới, danh mục, thương hiệu, bán chạy tháng/toàn thời gian, đánh giá cao, sản phẩm theo danh mục, coupon đang chạy*).
- Danh sách sản phẩm hỗ trợ tìm kiếm theo từ khóa và lọc theo danh mục, thương hiệu, khoảng giá, thuộc tính biến thể (*kích cỡ, màu*), sắp xếp theo giá/mới nhất/bán chạy/đánh giá, kèm phân trang.
- Chi tiết sản phẩm với lựa chọn biến thể dựa trên tồn kho thật, hình ảnh, mô tả, đánh giá của người mua khác.
- Giỏ hàng hoạt động cả với khách vãng lai (*localStorage*) lẫn người đã đăng nhập (*đồng bộ lên server khi đăng nhập*).
- Đặt hàng trong một transaction nguyên tố: trừ tồn kho, ghi chi tiết đơn, tạo hóa đơn VAT và vận đơn cùng lúc — thất bại một bước là toàn bộ quay lui.
- Thanh toán COD và chuyển khoản trực tuyến (*liên kết PayOS, xác nhận tiền qua webhook Casso*); theo dõi vận đơn theo từng mốc trạng thái.
- Đánh giá sản phẩm sau mua, mã giảm giá (*xem, lưu, áp dụng có kiểm định điều kiện*), chương trình thành viên tích điểm và hạng khách hàng.
- Hồ sơ cá nhân, sổ địa chỉ, lịch sử đơn hàng – hóa đơn, yêu thích, lịch sử tìm kiếm, đổi mật khẩu, quên mật khẩu qua email; giao diện song ngữ Việt – Anh.

Website quản trị:

- Thống kê doanh thu, tồn kho, đơn hàng trên dashboard.
- Quản lý đầy đủ nghiệp vụ cửa hàng: sản phẩm – biến thể – thuộc tính – hình ảnh, danh mục, thương hiệu, nhà cung cấp, đơn nhập hàng, kho (*phiếu nhập/xuất/điều chỉnh*), đơn hàng, hóa đơn, vận chuyển, khách hàng, coupon, chương trình loyalty, yêu cầu hỗ trợ.
- Phân quyền theo vai trò và quyền riêng lẻ (*RBAC*); nhập/xuất Excel cho nhiều danh mục dữ liệu.

Về mặt kỹ thuật: cơ sở dữ liệu MySQL gồm 31 bảng được thiết kế qua Prisma với đầy đủ khóa ngoại và xóa mềm thống nhất; backend REST API tách rõ tầng Controller – Service; bảo mật bằng JWT (*cập access/refresh token*), bcrypt, kiểm tra dữ liệu vào bảng

Joi; gửi email qua Nodemailer; lưu trữ hình ảnh trên Supabase Storage.

Thu hoạch chuyên môn – Kinh nghiệm

Chuyên môn:

- Nắm vững cách tổ chức kiến trúc API tách tầng, giúp mã nguồn dễ bảo trì và mở rộng.
- Hiểu sâu cơ chế transaction trong cơ sở dữ liệu để bảo đảm tính toàn vẹn nghiệp vụ (*không bao giờ bán vượt tồn kho*).
- Thành thạo xác thực bằng JWT kết hợp refresh token, mã hóa mật khẩu bcrypt và mô hình phân quyền RBAC.
- Rèn luyện tư vấn thiết kế quan hệ dữ liệu: chuẩn hóa bảng, khóa ngoại, chỉ mục phục vụ truy vấn lọc – tìm kiếm.
- Làm quen tích hợp dịch vụ ngoài: cổng thanh toán PayOS và webhook đối soát Casso, email Nodemailer, lưu trữ đám mây Supabase.

Kinh nghiệm:

- Học cách chia bài toán lớn thành các luồng nhỏ, xử lý tuần tự từ cơ sở dữ liệu → API → giao diện.
- Biết đọc tài liệu chính thức (*Prisma, Express, React Router*) thay vì sao chép máy móc.
- Trải nghiệm quy trình làm việc thực tế: quản lý phiên bản Git, tự kiểm thử luồng trước khi hoàn tất tính năng.
- Rèn thói đặt câu hỏi "dữ liệu này nằm ở đâu, ai được sửa nó" trước khi viết mã — giúp giảm lỗi bảo mật và logic.

Ưu điểm, hạn chế – Nguyên nhân

Ưu điểm

- Đặt hàng dùng transaction nên dữ liệu đơn hàng và tồn kho luôn đồng nhất, không xảy ra tình trạng âm kho hay mất đơn.
- Mọi số tiền (*giá, phí ship, giảm giá, tổng cộng*) đều do server tự tính lại — client không thể gian lận bằng cách gửi sẵn con số.
- Phân quyền RBAC chặt chẽ, mọi route dữ liệu nhạy cảm đều bắt buộc đăng nhập và kiểm tra quyền.
- Xóa mềm thống nhất (*deleted_at*) giúp khôi phục dữ liệu và bảo toàn liên kết giữa các bản ghi.
- Trang chủ và trang danh sách lấy dữ liệu qua loader trước khi render nên không hiện màn hình trắng; backend gộp nhiều truy vấn thành một request giảm số lần

gọi mạng.

- Giao diện hiện đại, đáp ứng tốt trên máy tính lẫn điện thoại, hỗ trợ hai ngôn ngữ Việt – Anh.

Hạn chế

- **Chưa có bộ kiểm thử tự động** (*unit test, integration test*): việc kiểm tra chủ yếu thủ công. *Nguyên nhân*: thời gian thực hiện có hạn nên ưu tiên hoàn thiện tính năng; nhóm chưa có kinh nghiệm dựng hạ tầng test cho cả frontend lẫn backend.
- **Thanh toán trực tuyến mới chạy trên môi trường thử nghiệm** của PayOS. *Nguyên nhân*: cần hồ sơ doanh nghiệp để kích hoạt môi trường sản xuất và đối soát thật.
- **Tìm kiếm dạng chứa chuỗi** (*LIKE*) sẽ chậm khi dữ liệu lên đến hàng trăm nghìn sản phẩm. *Nguyên nhân*: chưa xây dựng chỉ mục full-text hoặc dùng công cụ tìm kiếm chuyên dụng.
- **Chưa triển khai lên máy chủ thật** với CI/CD. *Nguyên nhân*: chi phí hạ tầng và thời gian cấu hình nằm ngoài phạm vi đề tài.

Hướng phát triển

Giai đoạn ngắn hạn (1–2 tháng): Hoàn thiện nền tảng và Triển khai thực tế

Trong giai đoạn này, mục tiêu trọng tâm là củng cố chất lượng mã nguồn, tối ưu hóa các tính năng hiện có và đưa hệ thống lên môi trường vận hành thực tế (*Production Environment*) để sẵn sàng bàn giao và kiểm thử công khai. **Viết unit test cho service layer**: Thời gian dự kiến là **2 tuần** với mục tiêu đảm bảo logic nghiệp vụ không bị phá vỡ khi tiến hành refactor mã nguồn.

- **Tích hợp MySQL FULLTEXT index**: Thời gian dự kiến từ **2–3 ngày** nhằm nâng cấp tốc độ tìm kiếm sản phẩm và giảm sự phụ thuộc vào toán tử LIKE.
- **Đưa PayOS sang môi trường production**: Thời gian dự kiến là **1 tuần** để kích hoạt hệ thống thanh toán thật qua mã QR code ngân hàng.
- **Triển khai lên Vercel + Railway**: Thời gian dự kiến từ **2–3 ngày** nhằm đưa giao diện lên Vercel và backend lên Railway để có link demo thực tế.
- **Cấu hình GitHub Actions CI/CD**: Thời gian dự kiến từ **2–3 ngày** để tự động hóa quy trình lint, build và deploy mỗi khi push code lên nhánh main.

Giai đoạn dài hạn (4–6 tháng): Mở rộng quy mô, Đa nền tảng và Ứng dụng AI

Ở giai đoạn dài hạn, SportNexus sẽ chuyển mình thành một hệ sinh thái thương mại điện tử toàn diện, hỗ trợ đa nền tảng thiết bị và ứng dụng trí tuệ nhân tạo để cá nhân hóa trải nghiệm mua sắm của người dùng.

- **Tích hợp API GHN/GHTK:** Thời gian dự kiến là **2 tuần** nhằm tính phí ship real-time và theo dõi vận đơn tự động.
- **Tích hợp Elasticsearch:** Thời gian dự kiến từ **2–3 tuần** để xây dựng tính năng tìm kiếm full-text chuyên nghiệp và gợi ý sản phẩm liên quan.
- **Áp dụng Redis cache:** Thời gian dự kiến là **1 tuần** nhằm giảm thời gian phản hồi trang chủ và danh mục xuống dưới 50ms.
- **Viết integration test cho checkout flow:** Thời gian dự kiến từ **1–2 tuần** để đảm bảo các transaction đặt hàng hoạt động an toàn trong dài hạn.
- **Bổ sung VNPAY/MoMo:** Thời gian dự kiến từ **1–2 tuần** nhằm mở rộng thêm các phương thức thanh toán trực tuyến cho khách hàng.

TÀI LIỆU THAM KHẢO

Tài liệu viết (sách, giáo trình)

- [1]. Nguyễn Chí Cường. *_Lập trình căn bản_*. Khoa Kỹ thuật – Công nghệ, Đại Học Tây Đô, 2023.
- [2]. Nguyễn Chí Cường. *_Phân tích và thiết kế hệ thống thông tin_*. Khoa Kỹ thuật – Công nghệ, Đại Học Tây Đô, 2024.
- [3]. Ngô Thị Lan. *_Nhập môn Công nghệ phần mềm_*. Khoa Kỹ thuật – Công nghệ, Đại Học Tây Đô, 2024.
- [4]. Lâm Tấn Phương. *_Giáo trình Lý thuyết Thiết kế và lập trình Web_*. Khoa Kỹ thuật – Công nghệ, Đại Học Tây Đô, 2024.
- [5]. Lâm Tấn Phương. *_Chuyên đề ngôn ngữ lập trình_*. Khoa Kỹ thuật – Công nghệ, Đại Học Tây Đô, 2024.
- [6]. D. Nanci và B. Espinasse. *_Ingénierie des systèmes d'information – Merise_*, Paris: Masson.
- [7]. I. Sommerville. *_Software Engineering_*, 10th ed., Pearson Education, 2016.
- [8]. A. Silberschatz, H. F. Korth và S. Sudarshan. *_Database System Concepts_*, 7th ed., McGraw-Hill Education, 2019.

Tài liệu trên Internet

- [9]. PayOS. *_Tài liệu tích hợp cổng thanh toán PayOS_*, truy cập 2026
<https://docs.payos.vn>
- [10]. Meta Open Source. *_React Documentation – Learn React_*, 2026.
<https://react.dev/learn>
- [11]. Evan You và cộng sự. *_Vite – Next Generation Frontend Tooling_*, 2026.
<https://vite.dev/guide/>
- [12]. Remix Software. *_React Router Documentation_*, 2026 <https://reactrouter.com>
- [13]. TanStack. *_TanStack Query Documentation_*, 2026.
<https://tanstack.com/query/latest/docs>
- [14]. OpenJS Foundation. *_Express – Node.js Web Application Framework_*, 2026.
- [Online]. Hiện có tại: <https://expressjs.com>
- [15]. Prisma Data. *_Prisma ORM Documentation_*, <https://www.prisma.io/docs>
- [16]. Oracle Corporation. *_MySQL 8.4 Reference Manual_*, 2026
<https://dev.mysql.com/doc/>
- [17]. M. Jones, J. Bradley và N. Sakimura. *_JSON Web Token (JWT)_*, RFC 7519, IETF, tháng 5/2015. [Online]. Hiện có tại: <https://datatracker.ietf.org/doc/html/rfc7519>
- [18]. Kelektiv. *_bcrypt – Password Hashing Function for Node.js_*, 2026.

<https://github.com/kelektiv/node.bcrypt.js>

[19]. Sideway Inc. *_Joi – Data Validation Library_*, 2026. <https://joi.dev>

[20]. Nodemailer. *_Nodemailer Documentation_*, 2026. <https://nodemailer.com>

[21]. Supabase. *_Supabase Storage Documentation_*, 2026.

<https://supabase.com/docs/guides/storage>

[22]. i18next. *_i18next – Internationalization Framework_*, 2026.

<https://www.i18next.com>

[23]. ExcelJS. *_ExcelJS – Excel Workbook Manager_*,

2026<https://github.com/exceljs/exceljs>